Guia de implementação do trabalho final

Energia fotovoltaica sob sombreamento parcial: painel, conversor, controle e MPPT

DAS5109 – Modelagem e Simulação de Processos (UFSC)

Explicação passo a passo, com equações, diagramas e o código MATLAB real

10 de novembro de 2026

Resumo

Este guia explica, na ordem em que foi construído, tudo o que foi feito no trabalho final: do modelo elétrico do painel até a comparação de algoritmos de rastreamento do ponto de máxima potência (MPPT) num dia real. Cada capítulo traz a física, as equações, os diagramas e os trechos de código que as implementam, com a lógica explicada passo a passo. Os trechos de código são lidos diretamente dos arquivos de `matlab/` (não são cópias digitadas), então o que está aqui é o que roda. Os números vêm dos arquivos de `resultados/`.

Como usar este guia

Cada capítulo segue o mesmo roteiro: **(1)** o problema e a intuição física; **(2)** as equações; **(3)** um diagrama; **(4)** o código, com uma caixa *Lógica passo a passo* que explica linha a linha o que ele faz; **(5)** os resultados. As caixas coloridas têm papel fixo: verde = intuição, roxa = lógica do código, vermelha = atenção (uma armadilha ou limitação), amarela = resultado numérico, azul = resumo. Os comentários dentro do código estão sem acentos, como nos arquivos originais.

Sumário

1 Visão geral **5**
1.1 A pergunta central . . . 5
1.2 Por que sombra parcial é difícil . . . 5
1.3 O sistema modelado . . . 5
1.4 Mapa do repositório . . . 6
1.5 Ordem de construção . . . 6

2 Decisões de projeto (D1 a D17) **7**

3 Modelo do painel fotovoltaico **8**
3.1 O circuito equivalente de 2 diodos . . . 8
3.2 Dependência com irradiância e temperatura . . . 9
3.2.1 Por que R_p depende da irradiância (D10) . . . 10
3.3 Os dados de catálogo . . . 10
3.4 A curva P-V . . . 11

4 Extração de parâmetros com `fsolve` **12**
4.1 5 incógnitas, 5 condições . . . 12
4.1.1 Por que o segundo diodo é fixado (D9) . . . 13
4.1.2 Variáveis do solver e chute inicial . . . 13

4.2 Como o `fsolve` resolve o sistema . . . 13
4.3 Validação em duas etapas . . . 14
4.4 Identificabilidade do segundo diodo (E1) . . . 15

5 Sombreamento parcial e diodos de bypass 15
5.1 A física . . . 16
5.2 As equações . . . 16
5.3 Newton–Raphson com passo limitado . . . 17
5.4 O que sai: curvas P-V com vários picos . . . 18

6 Modelo térmico e calibração pelo NMOT 19
6.1 O balanço de energia . . . 19
6.2 Calibração pelo NMOT . . . 21

7 Conversor buck-boost 21
7.1 Como funciona o buck-boost inversor . . . 22
7.2 Dimensionamento . . . 22
7.3 Três modelos do mesmo conversor . . . 23
7.3.1 Modelo médio . . . 23
7.3.2 Modelo chaveado . . . 24
7.4 O equilíbrio com perdas . . . 25
7.5 CCM e DCM . . . 26
7.6 Validação chaveado × médio (E3) . . . 26
7.7 Malha aberta com DAE (`sim_medio_aberta`) . . . 27

8 Controle: linearização, resposta em frequência e projeto do PI 27
8.1 A estrutura em dois laços . . . 28
8.2 Linearização de pequenos sinais . . . 28
8.3 A planta G_{vd} em forma fechada . . . 29
8.4 Margens de estabilidade sem toolbox . . . 30
8.5 O projeto do PI (D12 e D13) . . . 32
8.6 Robustez . . . 33
8.7 Resultado e período do MPPT . . . 33
8.8 A lei de controle com saturação . . . 34

9 Algoritmos de MPPT 35
9.1 Perturba e Observa (P&O) . . . 36
9.2 Condutância Incremental (InC) . . . 37
9.3 Enxame de partículas (PSO) . . . 38
9.4 Configuração comum . . . 41
9.5 Rastreamento estático (E5) . . . 41

10 Simulação: dinâmica, quase-estática e oráculo 42
10.1 As escalas de tempo . . . 43
10.2 A hierarquia de modelos . . . 43
10.3 Simulação dinâmica . . . 43
10.4 Oráculo do máximo global . . . 46
10.5 Simulação quase-estática . . . 47
10.6 Validade do quase-estático contra a dinâmica (D16) . . . 49

11 Cenários: EN 50530 e o dia em Florianópolis 49
11.1 Cenário constante . . . 49
11.2 Perfil dinâmico da EN 50530 . . . 49
11.3 O dia em Florianópolis . . . 51
11.4 Nuvens e sombras sintéticas (D6) . . . 51

12 Experimentos e resultados 52
12.1 Visão geral . . . 52
12.2 E6: MPPT na EN 50530 (sem sombra) . . . 52
12.3 E6b: conferência no modelo dinâmico . . . 53
12.4 E7: dia em Florianópolis com sombras e ablação . . . 53
12.4.1 Três limitações que só o dia real revela (D17) . . . 54
12.5 E8: o trabalho anterior contra o modelo completo . . . 54

13 Testes, simplificação do código e verificação 55
13.1 O harness de testes . . . 56
13.2 A etapa de simplificação do código . . . 57
13.3 Resultado da simplificação . . . 58
13.4 Como se verificou que nada quebrou . . . 58

14 Conclusões, limitações, como rodar e glossário 59
14.1 Resposta à pergunta central . . . 59
14.2 Limitações . . . 59
14.3 Como rodar . . . 60
14.4 Glossário . . . 60

Lista de Figuras

1 Fluxo do sistema. Cada seta é uma variável trocada entre blocos. Os dois laços de controle estão em azul. . . . 6
2 Circuito equivalente de uma célula (ou de um grupo de células em série) no modelo de 2 diodos. . . . 8
3 Pipeline da extração. O `fsolve` chama os resíduos quantas vezes forem necessárias; a validação usa o coeficiente de P_{max}, que não entra no ajuste. . . . 12
4 E1. Esquerda: eficiência relativa $\eta(G)/\eta(\mathrm{STC})$ nos modelos de 1 e 2 diodos. Direita: eficiência em $200\,\mathrm{W/m^2}$ em função de I_{02}/I_{01}. . . . 15
5 Três submódulos em série, cada um com seu diodo de bypass em antiparalelo. Esquema conceitual. . . . 16
6 E2. Curvas I-V e P-V do ZTP-160M sob três padrões de sombra: aparecem de 1 a 3 picos. . . . 19
7 Fluxos de energia do modelo térmico. A radiação de onda longa da frente troca com o céu (frio); a do verso, com o solo (perto de T_{amb}). . . . 19
8 Conversor buck-boost inversor, entre o painel (com o capacitor de entrada C) e a bateria. . . . 22
9 As duas topologias que se alternam a cada período de chaveamento. . . . 22
10 E3. Modelo chaveado (com ripple), média móvel de um período e modelo médio. Em cima, STC (CCM); embaixo, $50\,\mathrm{W/m^2}$ (DCM). . . . 27
11 Malha de tensão do conversor. O ganho de malha é $L(s) = C(s)\,[-G_{vd}(s)]\,e^{-1{,}5sT_s}$; o sinal negativo da planta é absorvido na definição do erro. . . . 28
12 E4. Bode da planta e do ganho de malha com o PI. Preto: caso nominal; vermelho: o caso que limita a MG; cinza: os demais. . . . 34
13 E4. Degrau de V_{ref}: modelo linear (usado no projeto) contra o modelo médio não linear. . . . 34

14 Lógica do P&O em cada período T_a. 36
15 Lógica do InC. O ramo é escolhido pelo passo *comandado*, não pelo dV medido. . . . 37
16 Máquina de estados do PSO. Reiniciar significa voltar às posições iniciais, com uma nova semente. 39
17 Cada modelo mais simples é conferido contra o mais detalhado, no intervalo em que ambos rodam. 43
18 O laço de `sim_dinamica`, repetido a cada T_a. No fim, $\eta = E/E_g$. 43
19 Estrutura de cada linha da tabela: espera em G_{baixo} e depois um número de trapézios (rampa, patamar alto, rampa, patamar baixo). 50
20 E6b. Como cada algoritmo segue as rampas no modelo dinâmico. 53
21 E7. Potência ao longo do dia (modelo completo). Preto: máximo global (oráculo); demais: potência extraída por P&O, InC e PSO. Embaixo, o detalhe da sombra de poste. 55

1 Visão geral

O trabalho mede quanto de energia um painel fotovoltaico real perde quando uma parte dele fica na sombra e o algoritmo de rastreamento (MPPT) é simples, e quanto um algoritmo global recupera. Tudo foi feito em MATLAB (com a Optimization Toolbox só para o `fsolve`), com código escrito para ser lido por um júnior (laços e nomes claros, comentários sobre a física).

1.1 A pergunta central

Em resumo

Quanta energia o P&O deixa de captar sob sombreamento parcial, e quanto um MPPT global (PSO) recupera, considerando o aquecimento do painel e as perdas do conversor?

A última parte da pergunta é o diferencial. Muitos estudos comparam MPPT com o painel a temperatura fixa e um conversor ideal. Aqui o painel esquenta durante o dia e o conversor tem perdas; o relatório mede quanto isso muda a resposta.

1.2 Por que sombra parcial é difícil

Um módulo é uma fileira de células em série, dividida em grupos chamados *submódulos*. Cada submódulo tem um *diodo de bypass* em antiparalelo. Num circuito em série todos conduzem a mesma corrente. Quando um submódulo recebe menos luz, ele não consegue gerar essa corrente; o diodo de bypass então desvia a corrente em volta dele e o submódulo deixa de contribuir com tensão.

O efeito é que a curva potência × tensão deixa de ter um único pico e passa a ter **vários picos**, um para cada combinação de submódulos "ligados". Um algoritmo local, como o P&O (Perturba e Observa), sobe a ladeira do pico mais próximo e pode ficar preso num pico local, mais baixo que o global. Um algoritmo global, como o PSO (Otimização por Enxame de Partículas), explora a curva inteira.

1.3 O sistema modelado

O sistema tem quatro blocos em cadeia (cenário → painel → conversor → bateria), um modelo térmico que acopla a temperatura ao painel, e dois laços de controle. O laço **interno** (PI) mede a tensão do painel v_{pv} e ajusta a razão cíclica d do conversor para que v_{pv} siga uma referência V_{ref}. O laço **externo** (MPPT), mais lento, mede tensão e corrente médias a cada período T_{a} e escolhe uma nova V_{ref}. Um *oráculo* calcula o máximo global verdadeiro, só para medir a eficiência dos algoritmos.

Intuição

Pense no PI como o motorista que mantém o carro na velocidade pedida (a tensão V_{ref}), e no MPPT como o passageiro que, de tempos em tempos, diz "tente ir um pouco mais rápido" (muda V_{ref}) e observa se a viagem rende mais. O passageiro só pode falar depois que o motorista já estabilizou a velocidade, senão não consegue saber o efeito do pedido. Esse é o critério de Femia (Capítulo 8), que fixa T_{a}.

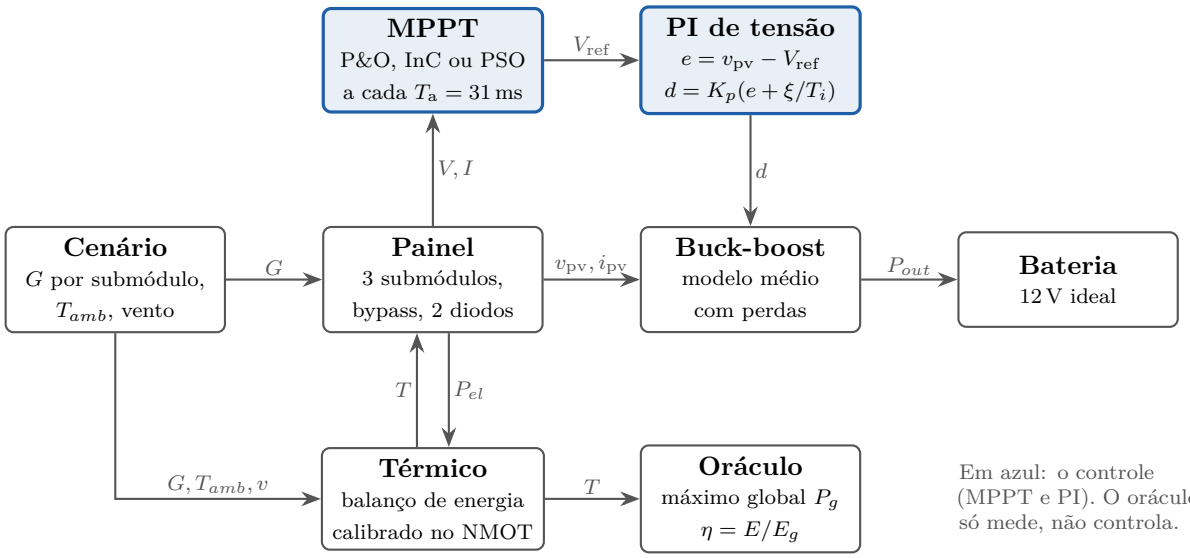


Figura 1: Fluxo do sistema. Cada seta é uma variável trocada entre blocos. Os dois laços de controle estão em azul.

1.4 Mapa do repositório

Pasta	O que contém	Cap.
`matlab/painel`	Modelo de 2 diodos, extração de parâmetros, sombreamento, validação	3, 4, 5
`matlab/termico`	Balanço térmico e calibração pelo NMOT	6
`matlab/conversor`	Buck-boost: ideal, médio com perdas, chaveado, equilíbrio, CCM/DCM	7
`matlab/controle`	Linearização, resposta em frequência, margens, projeto do PI	8
`matlab/mppt`	P&O, InC, PSO, duty fixo, rastreamento estático	9
`matlab/simulacao`	Simulação dinâmica, quase-estática, oráculo	10
`matlab/cenarios`	Perfil EN 50530, dia do PVGIS, sombras	11
`matlab/experimentos`	`exp01` a `exp08`, que geram `resultados/` e as figuras	12
`matlab/tests`	Testes automatizados (80)	13
`relatorio/`, `proposta/`, `guia/`	Documentos em LaTeX	—

1.5 Ordem de construção

O plano (`docs/superpowers/plans/2026-10-27-trabalho-final-pv-mppt.md`) tem 16 tarefas (0 a 15). Cada bloco só foi construído depois que o anterior estava testado:

1. Reorganização do código e harness de testes (tarefa 0).
2. Dependência da célula com G e T e KCL do módulo (tarefa 1).
3. Extração de parâmetros, validada no KC200GT (tarefa 2).
4. Painel real ZTP-160M e sua validação (tarefa 3).
5. Modelo térmico (tarefa 4).
6. Conversor: modelo médio, equilíbrio, CCM/DCM (tarefa 5) e modelo chaveado (tarefa 6).
7. Linearização e resposta em frequência (tarefa 7) e projeto do PI (tarefa 8).
8. Algoritmos de MPPT (tarefa 9).
9. Simulação dinâmica (tarefa 10) e quase-estática (tarefa 11).
10. Cenários (tarefa 12) e experimentos de resultados (tarefa 13).

11. Relatório (tarefa 14) e revisão final com publicação (tarefa 15).

Depois veio uma etapa extra de **simplificação do código** (Capítulo 13).

2 Decisões de projeto (D1 a D17)

O plano registra 17 decisões. As dez primeiras foram tomadas em 03/11/2026, ao validar o plano. As demais surgiram durante a implementação, quando um teste ou experimento mostrou que a ideia inicial não funcionava. Entender o *porquê* de cada uma é o que permite defender o trabalho.

#	Tema	Decisão	Por quê
D1	Painel	ZTROON ZTP-160M; o KC200GT fica só como benchmark da extração	Valida o método contra um resultado publicado (Villalva, 2009) e aplica ao painel real
D2	Ferramentas	MATLAB base; depois, também a Optimization Toolbox só para o `fsolve`	Inicialmente sem toolboxes (LM próprio). Revisto em 10/11/2026: o `fsolve` em $\log_{10}$ de R_s e R_p dá os mesmos parâmetros
D3	Bateria	12 V constante (fonte ideal)	Sem sombra $V_{mp} \approx 19{,}6\,\text{V} > 12\,\text{V}$: o conversor abaixa. Com sombra o pico global cai para 7 a 13 V: precisa elevar. O buck-boost é necessário justamente para o MPPT global
D4	Conversor	$f_s = 20\,\text{kHz}$, $L \approx 57\,\mu\text{H}$, $C \approx 1{,}3\,\text{mF}$, perdas de catálogo	20 kHz é inaudível e tem perdas de chaveamento pequenas nessa potência (o trabalho anterior usou 2 kHz, com indutor 10 vezes maior)
D5	EN 50530	Tabelas B.2 a B.4 transcritas de um relatório de ensaio TÜV Rheinland	O texto da norma é pago; os valores conferem com as durações impressas
D6	Dia real	PVGIS horário de Florianópolis, com nuvens e sombra de poste sintéticas e declaradas	Dados horários não têm transitórios rápidos
D7	Relatório	LaTeX, estilo do trabalho anterior	Continuidade com a disciplina
D8	Validação do painel	Sem curvas I-V (o datasheet não tem): STC, coeficiente térmico de P_{max}, NMOT e KC200GT	Usa só informação publicada e declara a limitação
D9	2 diodos	Convenção de Ishaque: $a_1 = 1$, $a_2 = 1{,}2$, $I_{01} = I_{02}$; lei térmica efetiva com E_g ajustado pelo K_v	Com $E_g = 1{,}12\,\text{eV}$ o modelo dá $K_v = -0{,}106\,\text{V/K}$ contra $-0{,}123$ medido; e o datasheet não identifica I_{02}
D10	R_p	$R_p(G) = R_{p,ref}\, G_n/G$ (De Soto)	Com R_p constante o ajuste previa perda de 26 % em $200\,\text{W/m}^2$, contra 3 a 5 % típicos
D11	Térmico	Convecção de Watmuff, radiação frente→céu e verso→solo, capacidade pela massa do datasheet	McAdams já inclui radiação e a contava duas vezes. Com as correções: $k_h = 0{,}79$ e $\tau \approx 7\,\text{min}$
D12	Projeto do PI	Varrer f_c e a posição do zero $r = \omega_z/\omega_c$, com MF$\geq 45°$, MG$\geq 6\,\text{dB}$; escolher a maior f_c	A regra inicial (zero em $\omega_c/5$) falha: a ressonância LC ($\sim$240 Hz, $\zeta \approx 0{,}16$) cruza 0 dB com fase $\approx -150°$
D13	Robustez e T_a	Margens em todos os pontos × cantos de L e C (± 20 %) e ponto quente; $T_a = 1{,}5\,t_s$ no pior caso	Com $L+20$ % a MG caiu a 4,6 dB; o MPPT tem de esperar o transitório do PI
D14	Parâmetros do MPPT	InC: $\varepsilon = \frac{1}{2}\lvert ds/dV\rvert\delta$. PSO: partículas em $k\,V_{mp}/N_{sub}$ e $0{,}9V_{oc}$	$\varepsilon = 0{,}002\,\text{S}$ nunca era atingido (InC oscilava como P&O); com partículas espaçadas linearmente o PSO errava o pico em $\sim$1/3 das sementes
D15	Dinâmica	EDO reduzida de 4 estados, com a corrente do painel resolvida por Newton a cada avaliação	A DAE com matriz de massa falhou em 9 de 36 casos; a EDO: 0 falhas, mesma eficiência em 4 dígitos, 2 a 4 vezes mais lenta

#	Tema	Decisão	Por quê
D16	Quase-estático	Aceito com erro $\leq 0{,}4\,\%$ (P&O, InC) e $-0{,}9$ a $-1{,}1\,\%$ (PSO); oráculo com $dt = 1\,\mathrm{s}$	Com $dt = T_\mathrm{a}$ o oráculo custa $\sim$2500 s de CPU por hora simulada; com 1 s, $\sim$65 s
D17	Limitações	Documentar como resultado, sem alterar os algoritmos	Os três efeitos só aparecem com o modelo térmico e o dia real; corrigir exigiria rodar E6 e E7 de novo (horas)

Em resumo

D1 a D8 são escolhas de escopo. D9 a D17 são *correções de rumo*: o plano dizia uma coisa, o teste mostrou outra, e a decisão registra o que foi trocado e por quê. O padrão se repete: primeiro a hipótese mais simples, depois a medição que a derruba, depois a correção documentada.

3 Modelo do painel fotovoltaico

Arquivos: `painel/celula_params.m`, `kcl_modulo.m`, `parametros_painel.m`, `datasheet_painel.m`, `curva_pv.m`. O painel é modelado como um circuito elétrico equivalente.

3.1 O circuito equivalente de 2 diodos

Uma célula solar se comporta como uma fonte de corrente (a luz gera elétrons) em paralelo com dois diodos (perdas por recombinação), uma resistência em paralelo R_p (fuga) e uma resistência em série R_s (contatos e metalização).

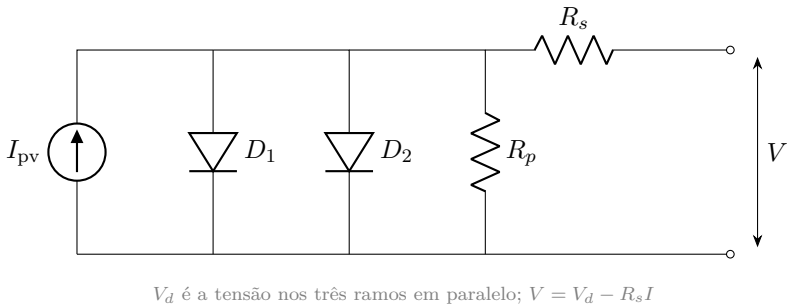


Figura 2: Circuito equivalente de uma célula (ou de um grupo de células em série) no modelo de 2 diodos.

A lei das correntes (KCL) dá a equação do módulo:

$$I = I_\mathrm{pv} - I_{01}\left(e^{V_d/(a_1 V_t)} - 1\right) - I_{02}\left(e^{V_d/(a_2 V_t)} - 1\right) - \frac{V_d}{R_p}, \qquad V_d = V + R_s I. \tag{1}$$

- I_pv: fotocorrente, proporcional à luz; é o que o painel "gera".
- I_{01}, I_{02}: correntes de saturação. O diodo 1 representa a difusão de portadores ($a_1 = 1$); o diodo 2, a recombinação na região de depósito ($a_2 = 1{,}2$).
- $V_t = kT/q$: tensão térmica ($\approx 25{,}7\,\mathrm{mV}$ por célula a 25 °C). No módulo multiplica-se pelo número de células em série.
- V_d/R_p: corrente de fuga pela resistência paralela.

O código devolve o *resíduo* $F = 0$ quando o ponto (V, I) está sobre a curva (Código 1). Essa forma *explícita* evita resolver a equação implícita $I(V)$ em todo lugar: basta testar se um ponto a satisfaz. Isso torna a extração de parâmetros (Capítulo 4) simples.

Código 1: `painel/kcl_modulo.m`: resíduo da equação do módulo e inclinação dI/dV.

```
function [F, g, dIdV] = kcl_modulo(V, I, G, T, p)
%KCL_MODULO Residuo explicito da equacao do modulo inteiro (sem sombra) em (V, I).
%   V, I, G, T sao escalares (G = irradiancia uniforme do modulo).
%   F = 0 quando (V, I) esta sobre a curva. g = -dIc/dVd.
%   dIdV = inclinacao dI/dV da curva I-V no ponto (V, I).

% Modulo inteiro tratado como um unico grupo de Ns celulas em serie
q = p;  q.Ncel = p.Ns;
c = celula_params(G, T, q);

% Tensao na juncao dos diodos = tensao do terminal + queda na resistencia serie
Vd = V + c.Rs_s * I;
e1 = exp(Vd / (p.a1 * c.Vt_s));
e2 = exp(Vd / (p.a2 * c.Vt_s));

% G e escalar aqui, entao c.Rp_s tambem e (Rp(G) de De Soto, ver celula_params);
% ./ por coerencia com residuo_painel.
% KCL: fotocorrente - diodo 1 - diodo 2 - corrente em Rp - corrente de saida = 0
F = c.Ipv - c.I01 * (e1 - 1) - c.I02 * (e2 - 1) - Vd ./ c.Rp_s - I;

% g = -dIc/dVd: derivada dos diodos mais condutancia de Rp
g = c.I01 * e1 / (p.a1 * c.Vt_s) + c.I02 * e2 / (p.a2 * c.Vt_s) + 1 ./ c.Rp_s;

% Derivacao implicita de F(V, I) = 0: dI/dV = -(dF/dV)/(dF/dI)
% com dF/dV = -g e dF/dI = -(1 + Rs_s*g), logo dI/dV = -g / (1 + Rs_s*g)
dIdV = -g / (1 + c.Rs_s * g);
end
```

Lógica passo a passo

1. O módulo todo é tratado como *um* grupo de N_s células em série (`q.Ncel = p.Ns`). As grandezas dependentes de G e T vêm de `celula_params`.
2. `Vd = V + Rs_s*I`: tensão na junção = tensão do terminal mais a queda em R_s.
3. `F` é a eq. (1) passada toda para um lado: fotocorrente menos os dois diodos menos a fuga menos a corrente de saída. Se $F = 0$, o ponto está na curva.
4. `g` é a condutância total da junção, $g = -\partial I_c/\partial V_d$.
5. `dIdV`: por derivação implícita de $F(V, I) = 0$, $\dfrac{dI}{dV} = -\dfrac{\partial F/\partial V}{\partial F/\partial I} = -\dfrac{g}{1 + R_s g}$. Essa inclinação é usada na condição $dP/dV = 0$ da extração.

3.2 Dependência com irradiância e temperatura

`celula_params(G,T,p)` concentra tudo o que muda com G (W/m^2) e T (K); a física fica num único lugar.

Código 2: `painel/celula_params.m`: grandezas do grupo de células em função de G e T.

```
function c = celula_params(G, T, p)
%CELULA_PARAMS Grandezas dependentes de G e T de um grupo de p.Ncel celulas.
%   G pode ser vetor (uma irradiancia por submodulo); c.Ipv tem o mesmo tamanho.
%   As saidas valem para um grupo de p.Ncel celulas em serie (Ncel = 1 da
%   os valores por celula).
%   Campos de saida:
%       Vt, Vt_s  tensao termica de uma celula e do grupo [V]
%       Rs_s  resistencia serie do grupo [ohm]
%       Rp_s  resistencia paralela do grupo [ohm], uma por elemento de G
%       Ipv  fotocorrente [A]; I01, I02  correntes de saturacao dos diodos [A]

% Tensao termica Vt = k*T/q (uma celula); o grupo tem Ncel celulas em serie
c.Vt   = p.k * T / p.q;
c.Vt_s = p.Ncel * c.Vt;

% Resistencias de celulas em serie somam
```

```
c.Rs_s = p.Ncel * p.Rs;

% Resistencia paralela dependente da irradiancia (De Soto, Klein e Beckman,
% 2006, Solar Energy 80:78-88): Rp(G) = Rp_ref * Gn/G, um valor por submodulo.
% Com Rp constante, a fuga V/Rp quase nao muda com a luz enquanto a
% fotocorrente cai com G; em baixa irradiancia a fuga vira uma fracao enorme
% da corrente (no ajuste do ZTP-160M, perda de 26 % a 200 W/m^2, contra os
% 3-5 % tipicos de modulos monocristalinos). A lei empirica de De Soto faz a
% fuga escalar com a irradiancia. Em G = Gn nada muda, entao a extracao de
% parametros (toda feita em Gn) nao e afetada.
% G = 0 (submodulo no escuro): Gn/0 = Inf, logo Rp = Inf e a fuga Vd/Rp = 0
% (sem NaN, pois Vd/Inf = 0 e 1/Inf = 0).
c.Rp_s = p.Ncel * p.Rp * (p.Gn ./ G(:));

% Fotocorrente: proporcional a irradiancia, corrigida linearmente pela temperatura
c.Ipv  = (G(:) / p.Gn) * (p.Ipv_n + p.Ki * (T - p.Tn));

% Correntes de saturacao: lei EFETIVA, a mesma para os dois diodos,
%   I0k(T) = I0k_n * (T/Tn)^3 * exp(Eg/k * (1/Tn - 1/T)).
% Modulos reais se desviam da teoria da juncao ideal; por isso p.Eg e um gap
% EFETIVO, calibrado pelo coeficiente Kv de Voc do datasheet, que absorve
% efeitos que a lei simples ignora. Com o gap do silicio (1,12 eV) o modelo
% da Kv = -0,106 V/K para o KC200GT, contra -0,123 V/K medido.
c.I01  = p.I01_n * (T / p.Tn)^3 * exp(p.q * p.Eg / p.k * (1 / p.Tn - 1 / T));
c.I02  = p.I02_n * (T / p.Tn)^3 * exp(p.q * p.Eg / p.k * (1 / p.Tn - 1 / T));
end
```

Grandeza	Lei usada	Intuição
Fotocorrente	$I_{\mathrm{pv}} = \dfrac{G}{G_n}\left(I_{pv,n} + K_i(T - T_n)\right)$	Mais luz, mais corrente; calor aumenta um pouco a corrente
Saturação	$I_{0k} = I_{0k,n}\left(\dfrac{T}{T_n}\right)^3 \exp\left[\dfrac{E_g}{k}\left(\dfrac{1}{T_n} - \dfrac{1}{T}\right)\right]$	Diodos "vazam" mais a quente; por isso V_{oc} cai com T
Tensão térmica	$V_t = kT/q$	Cresce com T
Resistência paralela	$R_p(G) = R_{p,ref}\, G_n/G$	Ver abaixo
Resistência série	$N_{cel} R_s$	Células em série somam resistências

Atenção: gap efetivo

Na teoria da junção, E_g é o gap do silício (1,12 eV). Com esse valor o modelo previa $K_v = -0{,}106\,\mathrm{V/K}$ para o KC200GT, contra $-0{,}123\,\mathrm{V/K}$ do datasheet. Por isso E_g virou um parâmetro *efetivo*, ajustado pelo K_v medido. Ele absorve efeitos que a lei simples ignora.

3.2.1 Por que R_p depende da irradiância (D10)

Com R_p constante, a fuga V/R_p quase não muda com a luz, mas a fotocorrente cai proporcionalmente a G. Em $200\,\mathrm{W/m^2}$ a fuga vira uma fração enorme da corrente gerada: no ajuste do ZTP-160M ($R_p \approx 41\,\Omega$) isso previa **26 % de perda de eficiência em 200 W/m²**, contra 3 a 5 % típicos de módulos monocristalinos. A lei de De Soto, Klein e Beckman (2006) faz a fuga escalar com a luz. Em $G = G_n$ nada muda, então a extração (toda em STC) não é afetada. Um submódulo no escuro ($G = 0$) dá $R_p = \infty$ e fuga zero, sem NaN, pois $V_d/\infty = 0$.

3.3 Os dados de catálogo

Código 3: `painel/datasheet_painel.m`, linhas 23 a 41: dados do ZTP-160M, com as conversões de %/K para A/K e V/K.

```
    case 'ztp160m'
        % ZTROON ZTP-160M -- datasheet v4.3 (dez/2024).
        % Coeficientes do catalogo em %/K convertidos para A/K e V/K.
        ds.nome = 'ZTP-160M';
```

```
        ds.Isc = 9.02;   ds.Voc = 22.64;
        ds.Imp = 8.20;   ds.Vmp = 19.57;  ds.Pmax = 160;
        ds.Ki = 0.0005 * ds.Isc;          % +0,05 %/K de Isc
        ds.Kv = -0.0034 * ds.Voc;         % -0,34 %/K de Voc
        ds.gamaP = -0.004;                % -0,4 %/K de Pmax
        ds.Ns = 33;                       % "Associacao das celulas 3x11"
        ds.Nsub = 3;                      % HIPOTESE: 1 bypass por coluna de 11
        ds.A = 0.82;                      % area [m^2]
        ds.NMOT = 45;                     % [C]
        ds.massa = 8.9;                   % massa do modulo [kg] (datasheet v4.3)
        % Modulo monocristalino com fator de forma alto (0,78): a = 1,3
        % (escolha de Villalva para o KC200GT, policristalino) nao o alcanca
        % sem Rs <= 0. a = 1,1 e o maior valor, em passos de 0,1, que ajusta.
        % A literatura trata o a do modelo de 1 diodo como escolha em [1; 1,5].
        ds.a_1d = 1.1;
```

Dado (ZTP-160M, datasheet v4.3)	Valor
I_{sc} / V_{oc}	9,02 A / 22,64 V
I_{mp} / V_{mp}	8,20 A / 19,57 V
P_{max}	160 W (mas $V_{mp}I_{mp} = 160{,}47$ W: o catálogo arredonda)
K_i	+0,05 %/K de I_{sc}
K_v	−0,34 %/K de V_{oc}
γ_P (coef. de P_{max})	−0,4 %/K (usado só para validar)
Arranjo	33 células, 3 submódulos de 11 (hipótese: 1 bypass por coluna)
Área, massa, NMOT	0,82 m^2, 8,9 kg, 45 °C

O fator de idealidade do modelo de 1 diodo é $a = 1{,}1$. Para o KC200GT, Villalva usa 1,3, mas o ZTP-160M é monocristalino com fator de forma 0,78 e 1,3 não o alcança sem $R_s \leq 0$. O valor 1,1 é o maior, em passos de 0,1, que ajusta.

3.4 A curva P-V

`curva_pv(G,T,p,nPts)` varre a tensão de 0 até uma estimativa folgada de V_{oc} e, a cada tensão, resolve a corrente do módulo (com sombra, por `painel_sombreado`, Capítulo 5). Cada ponto usa a solução do anterior como chute inicial (*warm start*), o que acelera o Newton. O resultado traz as curvas I-V e P-V, o máximo global (V_{mp}, P_{mp}) e a tabela de todos os máximos locais (`islocalmax`).

Código 4: `painel/curva_pv.m`, linhas 12 a 33: varredura de tensão com *warm start* e detecção dos picos.

```
% Voc estimado folgado: soma dos Voc nominais por submodulo, com margem
Voc_est = p.Nsub * p.Ncel * 0.75;
V = linspace(0, Voc_est, nPts);
I = zeros(size(V));

z = [];
for n = 1:nPts
    [I(n), s] = painel_sombreado(V(n), G, T, p, z);
    z = s.z;                      % warm start para o proximo ponto
end

ok = I >= 0;                      % descarta a regiao alem de Voc
c.V = V(ok);  c.I = I(ok);  c.P = c.V .* c.I;

[c.Pmp, k] = max(c.P);
c.Vmp = c.V(k);
c.Imp = c.I(k);

loc = find(islocalmax(c.P));
c.picos = table(c.V(loc).', c.I(loc).', c.P(loc).', ...
    'VariableNames', {'V', 'I', 'P'});
end
```

4 Extração de parâmetros com `fsolve`

O datasheet não dá R_s, R_p, I_0 nem E_g. Dá só pontos de operação. Este capítulo mostra como transformar esses pontos nos parâmetros do modelo. Arquivos: `painel/extrair_parametros.m`, `painel/identificabilidade_2d.m`, `painel/validar_painel.m`.

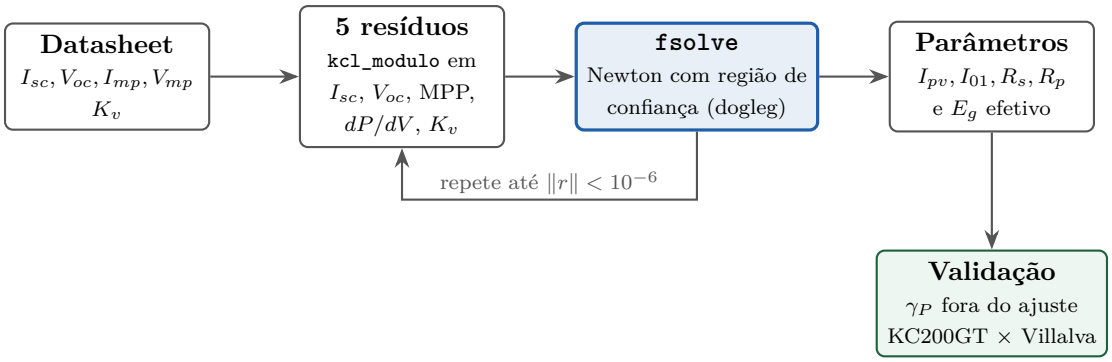


Figura 3: Pipeline da extração. O `fsolve` chama os resíduos quantas vezes forem necessárias; a validação usa o coeficiente de P_{max}, que não entra no ajuste.

4.1 5 incógnitas, 5 condições

As incógnitas são $x = [I_{\mathrm{pv}},\ \log_{10} I_{01},\ R_s,\ R_p,\ E_g]$ (com R_s e R_p do módulo inteiro). As cinco condições vêm do datasheet; cada uma vira um resíduo calculado com `kcl_modulo`:

$$F_1 = F(0, I_{\mathrm{sc}}) = 0 \qquad \text{(curto-circuito)} \tag{2}$$

$$F_2 = F(V_{\mathrm{oc}}, 0) = 0 \qquad \text{(circuito aberto)} \tag{3}$$

$$F_3 = F(V_{\mathrm{mp}}, I_{\mathrm{mp}}) = 0 \qquad \text{(o MPP está na curva)} \tag{4}$$

$$F_4 = I_{\mathrm{mp}} + V_{\mathrm{mp}} \left.\frac{dI}{dV}\right|_{mp} = 0 \qquad \text{(o MPP é máximo: } dP/dV = 0) \tag{5}$$

$$F_5 = F\big(V_{\mathrm{oc}} + K_v \Delta T,\ 0\big) \text{ em } T_n + \Delta T = 0,\ \Delta T = 25\,\mathrm{K} \qquad \text{(coeficiente de } V_{oc}) \tag{6}$$

Todos os resíduos são divididos por I_{sc}, para ficarem na mesma escala. Como há 5 equações e 5 incógnitas, o ajuste é na verdade a solução de um sistema; se tudo der certo, o resíduo final é praticamente zero. O código exige menos de 10^{-6} e emite `warning` se não convergir. E_g é incógnita porque a condição 5 é a única que enxerga a temperatura, e E_g controla como I_0 varia com T.

Código 5: `painel/extrair_parametros.m`, linhas 87 a 109: os cinco resíduos, um por condição do datasheet.

```
87  end
88
89  function r = residuos_extracao(x, ds, p0)
90  % Residuos das condicoes do datasheet; todos zero na solucao.
91  q = aplicar_incognitas(x, p0);
92  Gn = q.Gn;  Tn = q.Tn;
93
94  % 1) Curto-circuito: (V, I) = (0, Isc) esta sobre a curva
95  F1 = kcl_modulo(0, ds.Isc, Gn, Tn, q);
96  % 2) Circuito aberto: (V, I) = (Voc, 0) esta sobre a curva
97  F2 = kcl_modulo(ds.Voc, 0, Gn, Tn, q);
98  % 3) MPP: (Vmp, Imp) esta sobre a curva
99  [F3, ~, dIdV] = kcl_modulo(ds.Vmp, ds.Imp, Gn, Tn, q);
100 % 4) MPP e maximo: dP/dV = I + V*dI/dV = 0 em (Vmp, Imp)
101 F4 = ds.Imp + ds.Vmp * dIdV;
102 % 5) Coeficiente termico: a Tn + dT o Voc deve ser Voc + Kv*dT.
103 %    Esta condicao calibra o gap efetivo Eg. dT = 25 K leva a 50 C, dentro
104 %    da faixa de operacao real, e e grande o bastante para que o efeito
105 %    de Eg sobre Voc domine o erro numerico.
```

```
dT = 25;
F5 = kcl_modulo(ds.Voc + ds.Kv * dT, 0, Gn, Tn + dT, q);

% Divide por Isc para que todos os residuos sejam correntes relativas
```

Lógica passo a passo

1. `aplicar_incognitas(x,p0)` converte o vetor x num *struct* de parâmetros por célula: $I_{01} = 10^{x_2}$, $I_{02} = \text{razão} \cdot I_{01}$, $R_s = x_3/N_s$, $R_p = x_4/N_s$.
2. As linhas 1 a 3 apenas perguntam: "o ponto (V, I) do datasheet satisfaz a equação do módulo?" A resposta é o resíduo de `kcl_modulo`.
3. `F4` usa a inclinação devolvida por `kcl_modulo` (terceira saída) para impor $dP/dV = 0$ no MPP.
4. `F5` refaz a pergunta a 50 °C, com V_{oc} deslocado de $K_v \cdot 25$. Como T muda, quem responde é a lei de $I_0(T)$, ou seja, o E_g.
5. O vetor final é normalizado por I_{sc}.

4.1.1 Por que o segundo diodo é fixado (D9)

Se I_{02} é fixado em qualquer valor entre 10^{-12} e 10^{-5} A, as quatro condições de STC continuam fechando exatamente: falta informação. Adota-se então a hipótese publicada de Ishaque et al. (2011): $a_1 = 1$, $a_2 = 1{,}2$ e $\boldsymbol{I_{01} = I_{02}}$. O modelo de 1 diodo (`'1d'`, com $a = 1{,}1$ e $I_{02} = 0$) também existe, para comparação.

4.1.2 Variáveis do solver e chute inicial

O `fsolve` não aceita limites. As variáveis do solver são $y = [I_{\text{pv}},\ \log_{10} I_{01},\ \log_{10} R_s,\ \log_{10} R_p,\ E_g]$, e o código converte de volta com $R_s = 10^{y_3}$ e $R_p = 10^{y_4}$ antes de calcular os resíduos.

Variável	Chute inicial	Motivo
I_{pv}	I_{sc}	No curto $I_{\text{sc}} = I_{\text{pv}}$−perdas, então I_{pv} fica um pouco acima de I_{sc}
$\log_{10} I_{01}$	-8	I_{01} varia muitas décadas; em log as variáveis ficam na mesma ordem
$\log_{10} R_s$	$\log_{10} 0{,}2$	$R_s = 10^y$ é sempre positivo
$\log_{10} R_p$	$\log_{10} 300$	Idem
E_g [eV]	1,12	Gap do silício como ponto de partida

Atenção: por que o log em R_s e R_p

Testado: com as variáveis diretas e o mesmo chute, o `fsolve` **divergiu no ZTP-160M** (1 e 2 diodos): R_p foi para $-4{,}5 \times 10^{12}\,\Omega$ e o resíduo parou em 4×10^{-2} após 500 iterações. No KC200GT funcionava. Em $\log_{10}$, a divergência para valores negativos some, e o `fsolve` convergiu nos quatro casos (ZTP-160M e KC200GT, 1 e 2 diodos) para os mesmos parâmetros do método anterior (LM próprio com limites), com resíduo de $\sim 10^{-15}$.

4.2 Como o `fsolve` resolve o sistema

Um sistema de 5 equações e 5 incógnitas, $r(y) = 0$, é resolvido por **Newton**: em cada iteração aproxima-se r por uma reta (em várias variáveis, pela matriz jacobiana J) e salta-se para onde ela cruza o zero:

$$J\,\Delta y = -r, \qquad y \leftarrow y + \Delta y. \tag{7}$$

Newton puro converge muito rápido perto da solução, mas longe dela um passo pode lançar uma variável para um valor absurdo (aqui, com exponenciais, isso estoura). O algoritmo padrão do `fsolve` é uma *região de confiança* (*dogleg*): ele limita o tamanho do passo a uma região onde confia na aproximação linear, e só aceita o passo se o resíduo realmente diminuir; senão encolhe a região e tenta de novo. O jacobiano é estimado por diferenças finitas, então não precisamos derivar `kcl_modulo` em relação a cada parâmetro.

Código 6: `painel/extrair_parametros.m`, linhas 46 a 64: variáveis em log, chamada do `fsolve` e montagem do resultado.

```
46 % Variaveis do fsolve: y = [Ipv, log10 I01, log10 Rs, log10 Rp, Eg].
47 % O fsolve NAO tem limites. Sem eles, no ZTP-160M ele divergia (Rp indo
48 % para -4e12 ohm e residuo parado em 4e-2). Resolvemos em log10 de Rs e Rp:
49 % 10^y e sempre positivo, entao a divergencia para valores negativos some.
50 % I01 ja era ajustada em log10 porque varia muitas decadas (o solver ve
51 % variaveis da mesma ordem de grandeza).
52 %     Ipv    log10 I01   log10 Rs [ohm]   log10 Rp [ohm]   Eg [eV]
53 y0 = [ds.Isc; -8;       log10(0.2);      log10(300);      1.12];
54 fun = @(y) residuos_extracao([y(1); y(2); 10^y(3); 10^y(4); y(5)], ds, p0);
55
56 % Tolerancias apertadas: o sistema 5x5 tem solucao exata (residuo ~ 0)
57 opcoes = optimoptions('fsolve', 'Display', 'off', ...
58     'FunctionTolerance', 1e-14, 'StepTolerance', 1e-14, ...
59     'OptimalityTolerance', 1e-14, 'MaxIterations', 500, ...
60     'MaxFunctionEvaluations', 5000);
61 [y, r, ~, saida] = fsolve(fun, y0, opcoes);
62 x = [y(1); y(2); 10^y(3); 10^y(4); y(5)];
63 info = struct('iter', saida.iterations, 'residuo', norm(r), 'r', r);
64 p = aplicar_incognitas(x, p0);
```

Lógica passo a passo

1. `y0`: chute inicial das variáveis do solver (tabela acima).
2. `fun = @(y) residuos_extracao([...])`: um *handle* que converte y em $x = [I_{\text{pv}}, \log I_{01}, R_s, R_p, E_g]$ (com $R_s = 10^{y_3}$, $R_p = 10^{y_4}$) e devolve os 5 resíduos.
3. `optimoptions('fsolve',...)`: tolerâncias de 10^{-14} e até 500 iterações e 5000 avaliações, porque o sistema tem solução exata (resíduo ≈ 0) e queremos ela, não só uma aproximação.
4. `fsolve(fun, y0, opcoes)` devolve a solução `y`, o vetor de resíduos `r` e informações do solver (`saida.iterations`).
5. O resultado volta para x e vira o *struct* de parâmetros por célula com `aplicar_incognitas`. `info.residuo = norm(r)` é conferido depois: abaixo de 10^{-6} é aceito, senão há `warning`.

Em resumo

O `fsolve` é da Optimization Toolbox. O projeto passa a depender dela, e não roda mais só com MATLAB base. Antes, um Levenberg–Marquardt próprio (com limites e 50 linhas) fazia o mesmo papel; foi removido junto com o seu teste.

4.3 Validação em duas etapas

Primeiro no KC200GT. Antes de confiar no método, ele foi aplicado a um painel cujo resultado é publicado (Kyocera KC200GT, Villalva 2009); o teste está em `tests/test_extracao.m`.

Depois no ZTP-160M, sem curvas. Como o datasheet não tem curvas I-V, os pontos de STC só confirmam que o ajuste fechou. A validação *independente* é o coeficiente térmico de P_{max}, que não entra no ajuste:

Resultado

Modelo	Erro P_{max} (STC)	Erro V_{oc}	γ_P do modelo	γ_P datasheet
1 diodo	+0,296 %	−0,032 %	−0,366 %/K	−0,4 %/K
2 diodos	+0,296 %	−0,032 %	−0,363 %/K	−0,4 %/K

O erro de 0,3 % em P_{max} não é do modelo: o datasheet é inconsistente ($V_{mp}I_{mp} = 160{,}47\,\mathrm{W}$, mas $P_{max} = 160\,\mathrm{W}$). O ajuste impõe (V_{mp}, I_{mp}), e contra $V_{mp}I_{mp}$ o erro é de 8×10^{-5} %. O γ_P previsto é cerca de 9 % menor em módulo que o do datasheet; o relatório declara essa diferença como limitação.

4.4 Identificabilidade do segundo diodo (E1)

Para mostrar que o datasheet não fixa I_{02}, refaz-se a extração com $I_{02}/I_{01} \in \{0{,}01;\ 0{,}1;\ 1;\ 10;\ 100\}$. Todas fecham, e o que muda é a previsão fora de STC:

I_{02}/I_{01}	Resíduo	E_g efetivo [eV]	$\eta(200)/\eta(\mathrm{STC})$	γ_P [%/K]
0,01	2×10^{-11}	1,307	94,83 %	−0,3623
0,1	3×10^{-11}	1,307	94,82 %	−0,3624
1	5×10^{-11}	1,304	94,79 %	−0,3628
10	5×10^{-8}	1,274	94,45 %	−0,3664
100	3×10^{-8}	1,164	93,08 %	−0,3743

Todos os resíduos são muito menores que 10^{-6}: todos os modelos satisfazem o datasheet. Mas a eficiência em baixa luz vai de 94,8 % a 93,1 %. Essa faixa de menos de 2 pontos percentuais é a **incerteza que o datasheet deixa** em baixa irradiância, pequena frente aos efeitos estudados depois (9 %, 11 % e 22 % na energia). Por isso a escolha da razão 1 não contamina as conclusões.

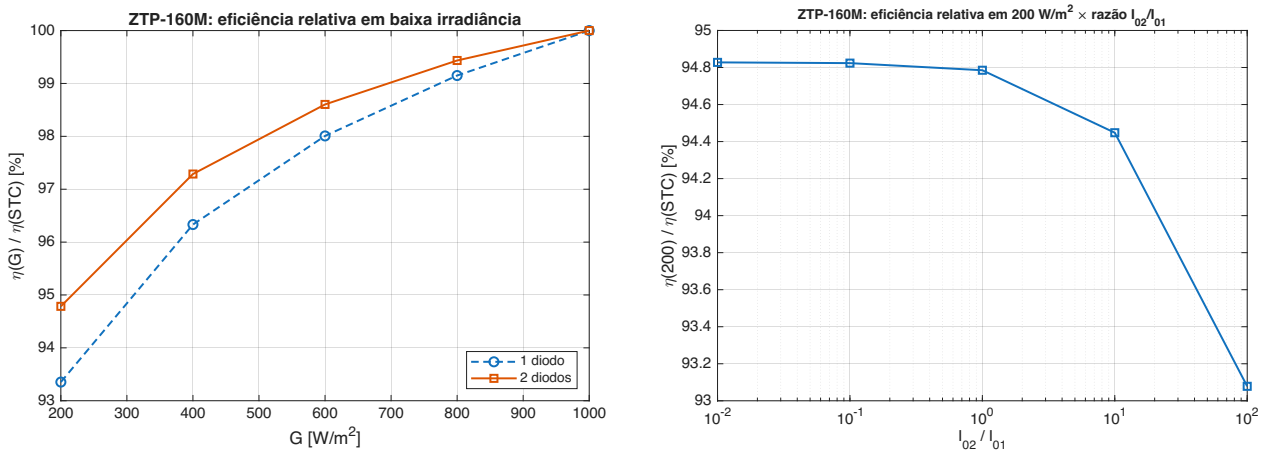


Figura 4: E1. Esquerda: eficiência relativa $\eta(G)/\eta(\mathrm{STC})$ nos modelos de 1 e 2 diodos. Direita: eficiência em $200\,\mathrm{W/m^2}$ em função de I_{02}/I_{01}.

5 Sombreamento parcial e diodos de bypass

Arquivos: `painel/painel_sombreado.m`, `painel/residuo_painel.m`, `cenarios/aplicar_sombras.m`. Aqui o módulo deixa de ser um bloco único e vira três submódulos em série, cada um com a sua irradiância.

5.1 A física

Em série, todos os submódulos conduzem a mesma corrente I e as tensões somam. Se um submódulo recebe pouca luz, não consegue gerar essa corrente; sem proteção seria forçado a tensão negativa e esquentaria (ponto quente). O **diodo de bypass**, em antiparalelo, conduz quando a tensão do submódulo cai para cerca de $-0{,}4\,\mathrm{V}$: a corrente passa por ele e o submódulo sombreado deixa de contribuir com tensão. O código modela o bypass como um Schottky ($I_{0bp} = 1{,}4 \times 10^{-6}\,\mathrm{A}$, queda de $\sim 0{,}4\,\mathrm{V}$ a $8\,\mathrm{A}$).

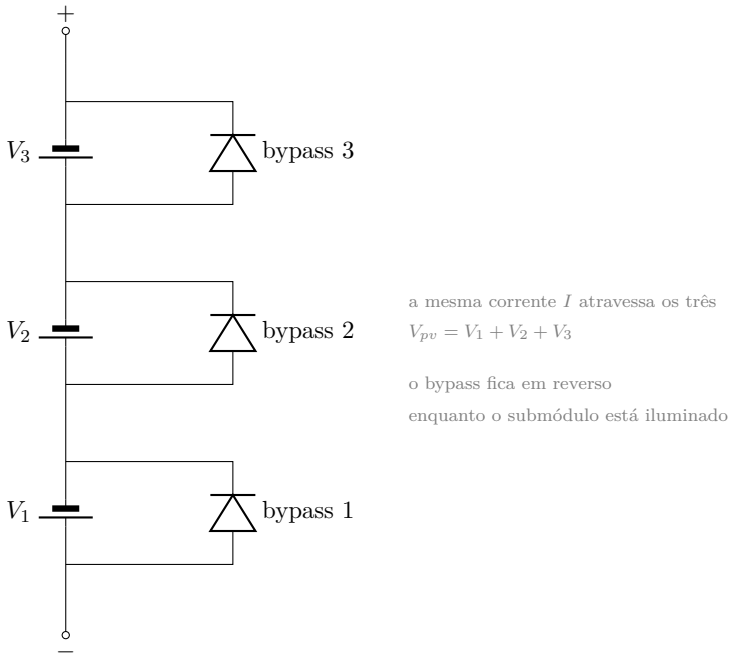


Figura 5: Três submódulos em série, cada um com seu diodo de bypass em antiparalelo. Esquema conceitual.

5.2 As equações

O vetor de incógnitas é $z = [I;\, V_{d,1}; \ldots; V_{d,N}]$, com $V_{d,j}$ a tensão de junção do submódulo j. Dado $V_{d,j}$, tudo no submódulo é explícito:

$$I_{c,j} = I_{pv,j} - I_{01}\big(e^{V_{d,j}/(a_1 V_t)} - 1\big) - I_{02}\big(e^{V_{d,j}/(a_2 V_t)} - 1\big) - \frac{V_{d,j}}{R_{p,j}} \tag{8}$$

$$V_j = V_{d,j} - R_s I_{c,j} \tag{9}$$

$$I_{bp,j} = I_{0bp}\big(e^{-V_j/V_{t,bp}} - 1\big) \tag{10}$$

O sistema tem $N+1$ equações:

$$R_j = I_{c,j} + I_{bp,j} - I = 0 \qquad \text{(KCL: células + bypass = corrente da fileira)} \tag{11}$$

$$R_{N+1} = \textstyle\sum_j V_j - V_{pv} = 0 \qquad \text{(KVL: a tensão do terminal é a soma)} \tag{12}$$

Código 7: `painel/residuo_painel.m`: resíduo $R(z)$ e jacobiano analítico $J = dR/dz$.

```
function [R, J, Vj, Ic, Ibp] = residuo_painel(z, Vpv, G, T, p)
%RESIDUO_PAINEL Equacoes algebricas do modulo com submodulos e bypass.
%   z = [I; Vd_1; ...; Vd_N], com Vd_j = tensao de juncao do submodulo j.
%   Com Vd_j conhecido, tudo no submodulo e explicito:
%       Ic_j  = Ipv_j - I01(e1-1) - I02(e2-1) - Vd_j/Rp_j   (celulas)
%       V_j   = Vd_j - Rs*Ic_j                              (terminais)
%       Ibp_j = I0bp*(exp(-V_j/(nbp*Vt)) - 1)               (bypass)
%   Equacoes (N+1):
```

```
%       R_j     = Ic_j + Ibp_j - I = 0      (KCL em cada submodulo)
%       R_{N+1} = sum(V_j) - Vpv   = 0      (KVL na serie)
%   J = dR/dz (jacobiano analitico).

N = p.Nsub;
G = G(:);

% Parametros do submodulo (dependentes de G e T), calculados em celula_params
c = celula_params(G, T, p);
Vt_s = c.Vt_s;  Rs_s = c.Rs_s;
Rp_s = c.Rp_s;                % vetor N x 1: Rp depende da G de cada submodulo
Ipv = c.Ipv;  I01 = c.I01;  I02 = c.I02;
Vt_bp = p.nbp * c.Vt;                       % tensao termica do diodo de bypass

I  = z(1);
Vd = z(2:end);
e1  = exp(Vd / (p.a1 * Vt_s));
e2  = exp(Vd / (p.a2 * Vt_s));
Ic  = Ipv - I01 * (e1 - 1) - I02 * (e2 - 1) - Vd ./ Rp_s;   % ./ : um Rp por submodulo
g   = I01 * e1 / (p.a1 * Vt_s) + I02 * e2 / (p.a2 * Vt_s) + 1 ./ Rp_s; % -dIc/dVd
Vj  = Vd - Rs_s * Ic;
dVj = 1 + Rs_s * g;                                                    % dVj/dVd
ebp = exp(-Vj / Vt_bp);
Ibp = p.I0bp * (ebp - 1);
h   = p.I0bp * ebp / Vt_bp;                                            % -dIbp/dVj

R = [Ic + Ibp - I; sum(Vj) - Vpv];

J = zeros(N + 1);
J(1:N, 1)       = -1;
J(1:N, 2:end)   = diag(-g - h .* dVj);
J(N + 1, 2:end) = dVj.';
end
```

Lógica passo a passo

1. `celula_params` entrega, para cada submódulo, R_p, I_{pv}, I_{01}, I_{02} (um valor por elemento de G).
2. `Ic` é a corrente das células; `Vj` a tensão nos terminais do submódulo; `Ibp` a do bypass. Se `Vj` é negativo, `exp(-Vj/Vt_bp)` dispara e o bypass conduz muito.
3. `R` empilha as N equações de KCL e a de KVL.
4. **Jacobiano analítico:** $\partial R_j/\partial I = -1$; $\partial R_j/\partial V_{d,j} = -g - h\,dV_j$ (com $g = -dI_c/dV_d$, $h = -dI_{bp}/dV_j$, $dV_j = 1 + R_s g$); a última linha é dV_j (derivada da soma de tensões).

5.3 Newton–Raphson com passo limitado

Código 8: `painel/painel_sombreado.m`: corrente do módulo sob sombra por Newton.

```
function [I, sol] = painel_sombreado(Vpv, G, T, p, z0)
%PAINEL_SOMBREADO Corrente do modulo FV sob sombreamento parcial.
%   [I, sol] = painel_sombreado(Vpv, G, T, p)
%   [I, sol] = painel_sombreado(Vpv, G, T, p, z0)
%
%   Vpv : tensao imposta nos terminais do modulo [V] (vem do conversor)
%   G   : vetor 1 x Nsub com a irradiancia de cada submodulo [W/m^2]
%   T   : temperatura das celulas [K]
%   p   : struct de parametros_painel()
%   z0  : chute inicial [I; Vd_1..Vd_Nsub] (warm start), opcional
%
%   Resolve residuo_painel(z) = 0 por Newton-Raphson com passo limitado.

N = p.Nsub;
c = celula_params(G, T, p);
Vt_s = c.Vt_s;

```

```
if nargin < 5 || isempty(z0)
    z0 = [p.Ipv_n * min(G) / p.Gn; repmat(max(Vpv, 0) / N, N, 1) + 0.5 * Vt_s];
end
z = z0(:);

tol = 1e-9;
for it = 1:60
    [R, J] = residuo_painel(z, Vpv, G, T, p);
    if norm(R, inf) < tol
        break
    end
    dz = -J \ R;
    % limita o passo: evita overflow das exponenciais longe da solucao
    dz(1)     = max(min(dz(1), 2), -2);
    dz(2:end) = max(min(dz(2:end), 2 * Vt_s), -2 * Vt_s);
    z = z + dz;
end

[R, ~, Vj, Ic, Ibp] = residuo_painel(z, Vpv, G, T, p);
I = z(1);
sol.z = z;  sol.Vj = Vj;  sol.Ic = Ic;  sol.Ibp = Ibp;
sol.iter = it;  sol.res = norm(R, inf);

% avisa se o Newton nao chegou na tolerancia
if sol.res > tol
    warning('painel_sombreado:naoConvergiu', ...
        'Newton nao convergiu em Vpv = %.3f V (residuo = %.2e)', Vpv, sol.res);
end
end
```

Lógica passo a passo

1. **Chute inicial.** Sem *warm start*: $I_0 = I_{pv,n}\min(G)/G_n$ e $V_d = V_{pv}/N + 0{,}5V_t$. Com *warm start*, usa-se a solução do ponto anterior.
2. A cada iteração calcula-se R e J; para se $\|R\|_\infty < 10^{-9}$; senão $\Delta z = -J^{-1}R$.
3. **Limite de passo:** a corrente muda no máximo 2 A e cada V_d no máximo $2V_t$ por iteração. Sem isso, um passo grande de exponencial estoura (overflow).
4. Máximo de 60 iterações; se não convergir, avisa com `warning`.
5. Saídas auxiliares (`sol.Vj`, `sol.Ibp`…) servem aos testes de bypass.

5.4 O que sai: curvas P-V com vários picos

O experimento E2 mede os picos de três cenários do ZTP-160M a 25 °C:

Resultado

Cenário	Irradiância relativa	V [V]	P [W]	Global?
Sem sombra	[1 1 1]	19,57	160,5	sim
1 submódulo a 40 %	[1 1 0,4]	12,68	103,8	**sim**
1 submódulo a 40 %	[1 1 0,4]	20,69	70,2	não
Degradê 100/60/30 %	[1 0,6 0,3]	5,80	47,3	não
Degradê 100/60/30 %	[1 0,6 0,3]	13,08	65,9	**sim**
Degradê 100/60/30 %	[1 0,6 0,3]	20,49	52,2	não

No caso de 1 submódulo a 40 %, o pico mais próximo de V_{oc} (20,7 V, 70 W) é local. Um algoritmo que parta de V_{oc} e suba a ladeira termina ali e perde 33 W (32 %) em relação ao pico global de 12,7 V. No degradê, o global está no meio. Os picos globais sombreados ficam em 12,7 e 13,1 V, perto da bateria de 12 V, e outros ficam abaixo: por isso o conversor precisa operar como buck *e* como boost (D3).

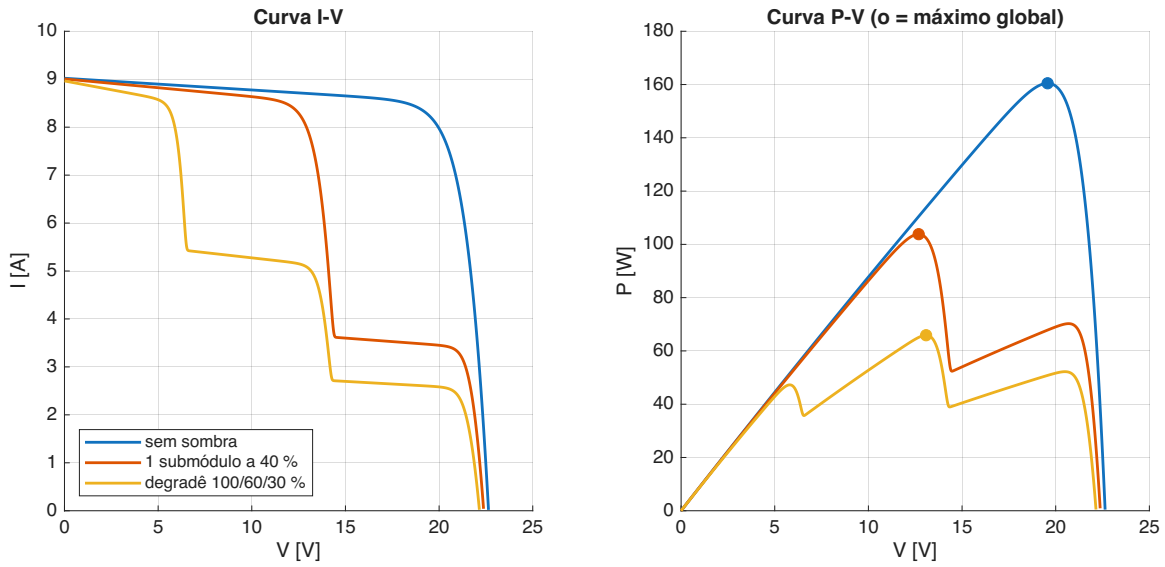


Figura 6: E2. Curvas I-V e P-V do ZTP-160M sob três padrões de sombra: aparecem de 1 a 3 picos.

6 Modelo térmico e calibração pelo NMOT

Arquivos: `termico/termico_dTdt.m`, `parametros_termicos.m`, `calibrar_nmot.m`. A temperatura do painel muda a curva I-V (Capítulo 3) e portanto muda onde está o ponto de máxima potência. Ignorá-la superestima a energia em cerca de 9 % no dia simulado.

6.1 O balanço de energia

O módulo é tratado como um único corpo à temperatura T (parâmetros concentrados). A energia que entra menos a que sai aquece o módulo:

$$mc_p\frac{dT}{dt} = \underbrace{\alpha A G}_{\text{sol absorvido}} - \underbrace{P_{el}}_{\text{eletricidade}} - \underbrace{h_c\,2A\,(T-T_{amb})}_{\text{convecção}} - \underbrace{\varepsilon\sigma A\,(T^4-T_{céu}^4)}_{\text{radiação, frente}} - \underbrace{\varepsilon\sigma A\,(T^4-T_{amb}^4)}_{\text{radiação, verso}} \qquad (13)$$

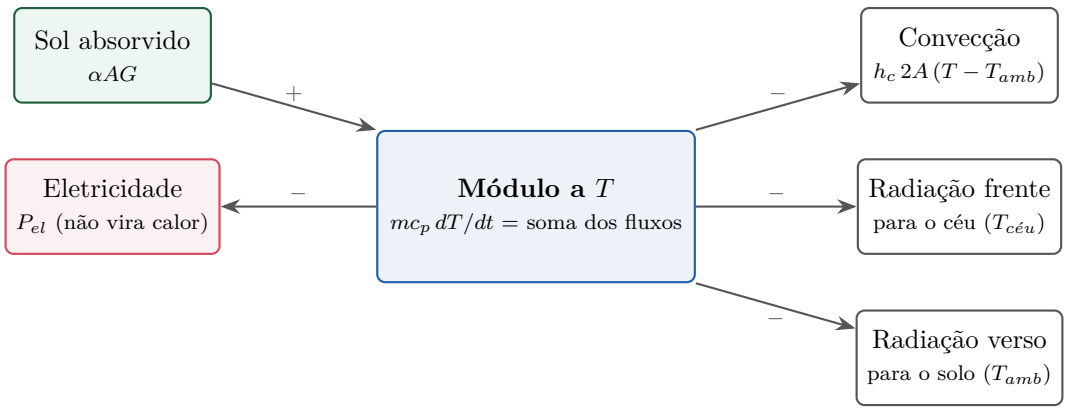


Figura 7: Fluxos de energia do modelo térmico. A radiação de onda longa da frente troca com o céu (frio); a do verso, com o solo (perto de T_{amb}).

Termo	Significado físico	Valor
α	Fração da luz absorvida	0,9
P_{el}	Energia que sai como eletricidade, não aquece	vem da simulação
h_c	Convecção de Watmuff: $k_h(2{,}8 + 3{,}0\,v)$	k_h calibrado
$2A$	A convecção age nas duas faces	$A = 0{,}82\,\mathrm{m}^2$
ε	Emissividade do vidro	0,85
$T_{céu}$	Temperatura efetiva do céu (Swinbank): $0{,}0552\,T_{amb}^{1,5}$	$\approx 277\,\mathrm{K}$ com $T_{amb} = 293\,\mathrm{K}$
mc_p	Capacidade térmica por área: massa · $840\,\mathrm{J/(kg\,K)}$ / área	$\approx 9{,}1\,\mathrm{kJ/(m^2K)}$

Código 9: `termico/termico_dTdt.m`: o balanço de energia como função dT/dt.

```
function dTdt = termico_dTdt(T, G, Tamb, vw, Pel, pt)
%TERMICO_DTDT Balanco de energia do modulo como um unico corpo [K/s].
%   m*cp*dT/dt = sol absorvido - eletricidade - conveccao - radiacao
%   T, Tamb [K]; G [W/m^2] (media dos submodulos); vw = vento [m/s]; Pel [W].
%   vw: a correlacao de Watmuff usa o vento perto do modulo; o PVGIS fornece o
%   vento a 10 m (WS10m), convertido no codigo do cenario (Tarefa 12).
%   Integracao: com vento de 10 m/s a menor constante de tempo e ~2,3 min; no
%   Euler explicito o passo precisa ser bem menor que 1 min (estabilidade: dt < 2*tau).
sigma = 5.670374419e-8;              % constante de Stefan-Boltzmann [W/(m^2 K^4)]
At = 2 * pt.A;                       % area de conveccao: frente e verso
% Conveccao pura (Watmuff et al., 1977, via Duffie e Beckman): h = 2.8 + 3.0*vw.
% Nao usamos h = 5.7 + 3.8*vw (McAdams) porque esse valor e um coeficiente
% COMBINADO que ja inclui a radiacao; como calculamos a radiacao a parte
% (Q_rad), usa-lo contaria a radiacao duas vezes. kh e o fator calibrado.
hc = pt.kh * (2.8 + 3.0 * vw);       % coeficiente de conveccao [W/(m^2 K)]
Tceu = 0.0552 * Tamb.^1.5;            % temperatura efetiva do ceu (Swinbank) [K]

Q_sol  = pt.alpha * pt.A * G;                    % radiacao solar absorvida [W]
Q_conv = hc * At * (T - Tamb);                   % calor levado pelo ar [W]
% Radiacao de onda longa (hipotese usual de modulo concentrado, ver Duffie e
% Beckman; Jones e Underwood, 2001): a face da frente enxerga o ceu (Tceu) e a
% face de tras, num modulo em estrutura, enxerga o solo, que esta perto de Tamb.
Q_rad_frente = pt.eps * sigma * pt.A * (T.^4 - Tceu.^4);   % frente -> ceu [W]
Q_rad_tras   = pt.eps * sigma * pt.A * (T.^4 - Tamb.^4);   % verso -> solo [W]
Q_rad = Q_rad_frente + Q_rad_tras;                       % radiacao total perdida [W]

% O que sobra do balanco aquece o modulo; dividir pela capacidade termica total [J/K]
dTdt = (Q_sol - Pel - Q_conv - Q_rad) / (pt.mcp_area * pt.A);
end
```

Lógica passo a passo

1. `hc = kh*(2.8 + 3.0*vw)`: coeficiente de convecção; `At = 2*A` porque o ar toca as duas faces.
2. `Tceu = 0.0552*Tamb^1.5`: temperatura efetiva do céu; é a que a face da frente "enxerga".
3. `Q_sol`, `Q_conv`, `Q_rad_frente`, `Q_rad_tras`: cada fluxo em watts.
4. `dTdt = (Q_sol - Pel - Q_conv - Q_rad)/(mcp_area*A)`: o saldo dividido pela capacidade térmica total em J/K.

Duas escolhas merecem explicação (D11):

- **Watmuff, não McAdams.** O coeficiente de McAdams $(5{,}7 + 3{,}8v)$ já inclui a radiação. Como o código calcula a radiação à parte, usar McAdams contaria a radiação duas vezes.
- **Frente vê o céu, verso vê o solo.** Com tudo irradiando para o céu, a calibração pelo NMOT exigia $k_h = 0{,}33$, valor irreal.

A capacidade térmica vem da massa do datasheet (8,9 kg): o vidro é 70 a 75 % da massa e tem $c_p \approx 840\,\mathrm{J/(kg\,K)}$; alumínio (∼900) e polímeros (1200 a 1500) são frações menores.

6.2 Calibração pelo NMOT

O único parâmetro livre é k_h. Ele é ajustado para reproduzir o **NMOT**, publicado no datasheet como 45 °C, nas condições $G = 800\,\mathrm{W/m^2}$, $T_{amb} = 20\,°\mathrm{C}$, vento de 1 m/s, módulo no MPP.

Código 10: `termico/calibrar_nmot.m`: busca do k_h que zera dT/dt em 45 °C.

```
function pt = calibrar_nmot(pt, ds)
%CALIBRAR_NMOT Ajusta pt.kh para que o regime permanente reproduza o NMOT.
%   Condicao NMOT: G = 800 W/m^2, Tamb = 20 C, vento 1 m/s, modulo no MPP.
%   A potencia eletrica no MPP e estimada pelo datasheet (Pmax e gamaP).
T_alvo = ds.NMOT + 273.15;
Pel = ds.Pmax * 0.8 * (1 + ds.gamaP * (T_alvo - 298.15));

% No regime permanente dT/dt = 0. Procuramos o kh que zera dT/dt em T_alvo.
% Em MATLAB, struct passada por valor: copiamos pt e trocamos so o campo kh.
balanco = @(kh) balanco_com_kh(kh, pt, T_alvo, Pel);
pt.kh = fzero(balanco, [0.1 10]);
end

function dTdt = balanco_com_kh(kh, pt, T_alvo, Pel)
% dT/dt na condicao NMOT para um dado kh (usa uma copia de pt com esse kh).
pt.kh = kh;
dTdt = termico_dTdt(T_alvo, 800, 293.15, 1, Pel, pt);
end
```

Lógica passo a passo

1. Potência elétrica no NMOT estimada pelo datasheet: $P_{el} = P_{max} \cdot 0{,}8 \cdot (1 + \gamma_P(45 - 25)) = 160 \cdot 0{,}8 \cdot 0{,}92 \approx 117{,}8\,\mathrm{W}$.
2. Em regime permanente $dT/dt = 0$. `fzero` procura em $[0{,}1;\ 10]$ o k_h que zera `balanco_com_kh` em $T = 45\,°\mathrm{C}$.
3. Resultado: $\boldsymbol{k_h = 0{,}79}$.

Resultado

Conferindo à mão. Com $k_h = 0{,}79$ e vento de 1 m/s, $h_c \approx 4{,}6\,\mathrm{W/(m^2K)}$. Em $T = 45\,°\mathrm{C}$: sol absorvido ≈ 590 W; elétrico ≈ 118 W; convecção ≈ 188 W; radiação frente ≈ 172 W; radiação verso ≈ 113 W. Saldo: $590 - 118 - 188 - 172 - 113 \approx 0$.
Constante de tempo. Capacidade total $9{,}1\ \mathrm{kJ/(m^2K)} \times 0{,}82\ \mathrm{m^2} \approx 7{,}5\,\mathrm{kJ/K}$. A perda cresce ≈ 17,7 W por kelvin. Logo $\tau \approx 7{,}5/17{,}7 \approx 7\,\mathrm{min}$, na faixa típica de 5 a 7 min.

Atenção: cuidado numérico

Com vento de 10 m/s a menor constante de tempo cai para ∼2,3 min. No Euler explícito o passo precisa ser menor que 2τ (na prática bem menor que 1 min). A simulação quase-estática usa passo $T_a \approx 31\,\mathrm{ms}$ (estável de sobra); a dinâmica usa `ode15s`. O vento do PVGIS é a 10 m de altura e é convertido para o vento junto ao módulo no código do cenário.

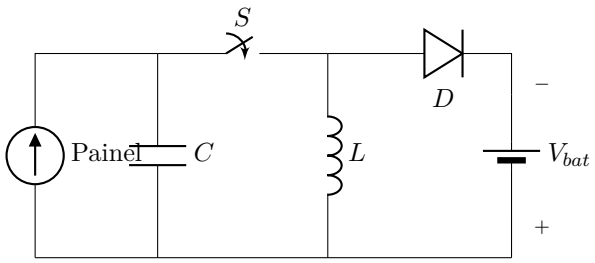
7 Conversor buck-boost

Arquivos: `conversor/parametros_conversor.m`, `conversor_ideal.m`, `conversor_medio_rhs.m`, `equilibrio_conversor.m`, `verifica_ccm.m`, `sim_chaveado.m`, `sim_medio_aberta.m`. O conversor é quem impõe ao painel a tensão de operação. Sem ele não há como o MPPT "escolher" o ponto da curva.

7.1 Como funciona o buck-boost inversor

O conversor tem uma chave (MOSFET), um indutor L, um diodo Schottky e um capacitor C na entrada, em paralelo com o painel. Em cada período $T_s = 1/f_s$:

- **Chave ligada** (fração d do período, a *razão cíclica*): o indutor fica ligado ao painel e *carrega*; a corrente i_L sobe.
- **Chave desligada** (fração $1-d$): o indutor é desligado do painel e *descarrega* na bateria pelo diodo; i_L cai.



Polaridade invertida: a bateria é carregada pela corrente do diodo.

Figura 8: Conversor buck-boost inversor, entre o painel (com o capacitor de entrada C) e a bateria.

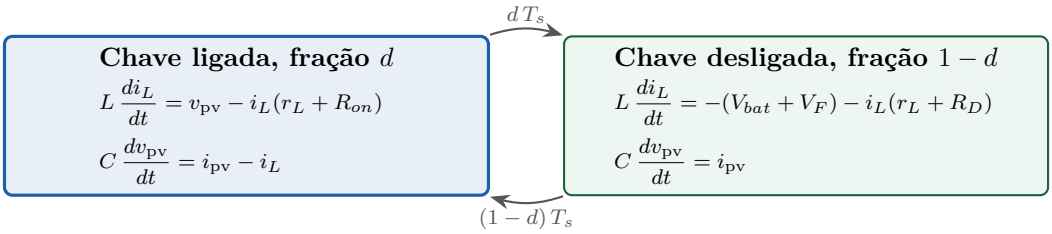


Figura 9: As duas topologias que se alternam a cada período de chaveamento.

No regime permanente a tensão média no indutor é zero (o fluxo não pode crescer sem limite). Sem perdas isso dá $d\,v_{\mathrm{pv}} = (1-d)\,V_{bat}$, ou

$$d = \frac{V_{bat}}{V_{bat} + v_{\mathrm{pv}}}, \qquad i_L = \frac{i_{\mathrm{pv}}}{d}. \tag{14}$$

A razão cíclica é a "alavanca" do MPPT: mudar d muda v_{pv}. Com $V_{bat} = 12\,\mathrm{V}$ e $v_{\mathrm{pv}} = 19{,}57\,\mathrm{V}$ (MPP de STC), $d = 0{,}38 < 0{,}5$: o conversor trabalha como buck. Se o pico global cai para 7 V, $d = 0{,}63$ e trabalha como boost.

7.2 Dimensionamento

Código 11: `conversor/parametros_conversor.m`: L e C pelo ripple no MPP de STC, perdas típicas.

```
function pc = parametros_conversor(p, Vbat)
%PARAMETROS_CONVERSOR Buck-boost inversor dimensionado pelo ripple no MPP em STC.
%   Chave ligada (fracao d): o indutor ve v_pv. Chave desligada: o indutor
%   descarrega na bateria. No equilibrio de tensao do indutor (sem perdas),
%   d*Vmp = (1-d)*Vbat, logo d = Vbat/(Vbat+Vmp) e iL = Imp/d.
if nargin < 2, Vbat = 12; end
pc.Vbat = Vbat;
pc.fs = 20e3;  pc.Ts = 1 / pc.fs;

% Ponto de operacao de projeto: MPP em STC (G = 1000 em todos os submodulos)
c = curva_pv(1000 * ones(1, p.Nsub), p.Tn, p, 1000);
D = Vbat / (Vbat + c.Vmp);
```

```
IL = c.Imp / D;

% A meta de ripple de 30 % e nominal (projeto com o D sem perdas); o ripple
% real fica em ~34-35 % porque o d real, com perdas, e maior.
% Indutor: com a chave ligada, di = v_pv*d*Ts/L. Impomos ripple de 30 % de iL.
pc.L = c.Vmp * D / (0.30 * IL * pc.fs);
% Capacitor de entrada: com a chave ligada o indutor puxa iL do capacitor, e a
% carga trocada por ciclo e IL*D*(1-D)*Ts (aprox.); ripple de tensao de 1 % de Vmp.
pc.C = IL * D * (1 - D) / (0.01 * c.Vmp * pc.fs);

% Perdas (valores tipicos de catalogo, ver decisao D4)
pc.rL = 0.020;                  % resistencia serie do indutor [ohm]
pc.Ron = 0.018;                 % resistencia do MOSFET ligado [ohm]
pc.VF = 0.45;                   % queda direta do diodo Schottky [V]
pc.RD = 0.010;                  % resistencia dinamica do diodo [ohm]
pc.ttr = 100e-9;                % tempo de subida + descida da chave [s]
% Dmin/Dmax/atraso sao usados pelo controlador (Tarefa 8) e Dmin/Dmax tambem
% pela checagem de viabilidade em equilibrio_conversor.
pc.Dmin = 0.02;  pc.Dmax = 0.95;
pc.atraso = 1.5 * pc.Ts;        % atraso do PWM digital
end
```

Lógica passo a passo

1. Ponto de projeto: MPP de STC (`curva_pv` com $G = 1000$ em todos os submódulos).
2. $D = V_{bat}/(V_{bat} + V_{mp}) \approx 0{,}380$ e $I_L = I_{mp}/D \approx 21{,}6\,\text{A}$ (eq. 14).
3. **Indutor**: com a chave ligada, $\Delta i = v_{\text{pv}} D T_s / L$. Impondo ripple de 30 % de I_L: $L = V_{mp} D/(0{,}30\, I_L f_s) \approx 57\,\mu\text{H}$.
4. **Capacitor**: com a chave ligada o indutor puxa i_L do capacitor; a carga trocada por ciclo é $I_L D(1-D)T_s$. Com ripple de tensão de 1 %: $C = I_L D(1-D)/(0{,}01\, V_{mp} f_s) \approx 1{,}3\,\text{mF}$.
5. Perdas de catálogo: $r_L = 20\,\text{m}\Omega$, $R_{on} = 18\,\text{m}\Omega$, $V_F = 0{,}45\,\text{V}$, $R_D = 10\,\text{m}\Omega$, $t_r + t_f = 100\,\text{ns}$; limites $d \in [0{,}02;\ 0{,}95]$; atraso do PWM $1{,}5T_s = 75\,\mu\text{s}$.

O ripple real fica em ∼34 a 35 %, não 30 %, porque o d real (com perdas) é maior que o ideal usado no projeto. A MOSFET é de classe 60 V porque a chave vê $v_{\text{pv}} + V_{bat} \approx 37\,\text{V}$. As perdas são típicas de catálogo, e o E7 mede o efeito delas (com e sem perdas), então a conclusão não depende dos valores exatos.

7.3 Três modelos do mesmo conversor

Modelo	O que faz	Quando usar
Ideal (`conversor_ideal`)	Zera todas as perdas	Comparação: "quanto as perdas custam?"
Médio com perdas (`conversor_medio_rhs`)	Média ponderada pelo tempo das duas topologias; sem ripple	Simulações longas (dia inteiro)
Chaveado (`sim_chaveado`)	Alterna as topologias de fato, 200 passos por período	Só para validar o médio, em janelas de ms

7.3.1 Modelo médio

$$C\frac{dv_{\text{pv}}}{dt} = i_{\text{pv}} - d\,i_L, \qquad L\frac{di_L}{dt} = d\,v_{\text{pv}} - (1-d)(V_{bat} + V_F) - i_L\big(r_L + d\,R_{on} + (1-d)R_D\big) \tag{15}$$

Código 12: `conversor/conversor_medio_rhs.m`: lado direito do modelo médio.

```
function dx = conversor_medio_rhs(vpv, iL, ipv, d, pc)
%CONVERSOR_MEDIO_RHS Derivadas do modelo medio com perdas: [dv_pv/dt; di_L/dt].
%   Valido apenas em conducao continua (CCM); use verifica_ccm para sinalizar DCM.
```

```
%   Em DCM as equacoes de CCM subestimam as perdas de conducao (corrente de
%   pico/RMS bem maior que a media), entao a eficiencia obtida e otimista.
%   O modelo medio nao inclui a perda de chaveamento Psw.
%   O modelo medio e a media ponderada pelo tempo das duas topologias:
%     chave ligada  (fracao d):   L di/dt = v_pv - iL*(rL + Ron)
%     chave desligada (1-d):      L di/dt = -(Vbat + VF) - iL*(rL + RD)
%   Capacitor de entrada: C dv/dt = i_pv - d*iL (o indutor so puxa corrente
%   do capacitor enquanto a chave esta ligada).
dv = (ipv - d * iL) / pc.C;
di = (d * vpv - (1 - d) * (pc.Vbat + pc.VF) ...
      - iL * (pc.rL + d * pc.Ron + (1 - d) * pc.RD)) / pc.L;
dx = [dv; di];
end
```

Lógica passo a passo

Cada termo da eq. (15) é a média ponderada pelo tempo das duas topologias da Figura 9: o que ocorre com a chave ligada multiplica-se por `d`; o que ocorre com ela desligada, por `(1-d)`. O capacitor só é descarregado pelo indutor durante a fração `d`.

7.3.2 Modelo chaveado

Código 13: `conversor/sim_chaveado.m`, linhas 44 a 85: o laço de Euler com 200 passos por período.

```
for k = 1:n
    t = r.t(k);

    % Guarda o estado ANTES de avancar: r.v(k), r.iL(k) valem em r.t(k)
    r.v(k) = v;  r.iL(k) = iL;

    % Indice do passo dentro do periodo (inteiro: evita erro de mod em ponto flutuante)
    j = mod(k - 1, passos_por_periodo);

    % PWM real: a razao ciclica e amostrada no inicio de cada periodo e fica
    % fixa ate o proximo (uma mudanca de d no meio do periodo so vale no seguinte)
    if j == 0
        d_per = dfun(t);
    end

    % Fracao do passo j com a chave ligada. A chave liga em t = 0 do periodo e
    % desliga em d*Ts, ou seja, no passo d*200. Os passos antes disso ficam
    % ligados inteiros (f = 1), os depois desligados (f = 0) e o passo onde
    % ocorre a comutacao fica ligado so em parte (0 < f < 1). Sem isso a chave
    % ficaria ligada ceil(200*d) passos inteiros, e o d efetivo erraria em ate 1/200.
    f = min(max(d_per * passos_por_periodo - j, 0), 1);

    % Corrente do painel na tensao atual do capacitor (warm start em z)
    [ipv, s] = painel_sombreado(v, G, T, p, z);
    z = s.z;

    % Chave ligada: indutor carrega a partir do painel/capacitor
    di_on = (v - iL * (pc.rL + pc.Ron)) / pc.L;
    dv_on = (ipv - iL) / pc.C;
    % Chave desligada: indutor descarrega na bateria pelo diodo;
    % o capacitor so recebe a corrente do painel
    di_off = (-(pc.Vbat + pc.VF) - iL * (pc.rL + pc.RD)) / pc.L;
    dv_off = ipv / pc.C;

    % Derivada media no passo, ponderada pelo tempo em cada topologia
    di = f * di_on + (1 - f) * di_off;
    dv = f * dv_on + (1 - f) * dv_off;

    % Euler explicito; o diodo bloqueia corrente negativa (i_L >= 0)
    v = v + dv * dt;
    iL = max(iL + di * dt, 0);
end
```

Lógica passo a passo

1. A cada passo guarda-se o estado *antes* de avançar; `j` é o índice do passo dentro do período.
2. **PWM real:** d é amostrado no início de cada período (`j == 0`) e fica fixo até o próximo.
3. `f` é a fração do passo com a chave ligada: 1 antes de $d \cdot 200$, 0 depois, e fracionário no passo da comutação. Sem isso o d efetivo erraria em até 1/200.
4. O painel é resolvido por Newton (com *warm start*) na tensão atual do capacitor.
5. As derivadas das duas topologias são combinadas por `f` e o estado avança por Euler.
6. O diodo bloqueia corrente reversa: `iL = max(..., 0)`. Quando i_L chega a zero, entra-se em DCM.

A condição inicial parte do instante em que a chave liga (i_L no vale, v_{pv} no pico): partir da média deslocaria o resultado em meio ripple (∼17 % em STC) e excitaria uma oscilação LC que não existe no modelo médio.

7.4 O equilíbrio com perdas

Dado $(v_{\text{pv}}, i_{\text{pv}})$, `equilibrio_conversor` acha d e a eficiência. Em regime, $dv_{\text{pv}}/dt = 0$ dá $i_L = i_{\text{pv}}/d$. Substituindo em $di_L/dt = 0$ e multiplicando por d, chega-se a uma equação do 2.º grau:

$$(v_{\text{pv}} + V_t)\, d^2 - \big(V_t + i_{\text{pv}}(R_{on} - R_D)\big)\, d - i_{\text{pv}}(r_L + R_D) = 0, \qquad V_t = V_{bat} + V_F. \tag{16}$$

O coeficiente de d^2 é positivo e o termo constante é negativo: o produto das raízes é negativo e há **exatamente uma raiz positiva**. Não precisa de `fzero`.

Código 14: `conversor/equilibrio_conversor.m`: raiz da quadrática, perdas e eficiência.

```
function eq = equilibrio_conversor(vpv, ipv, pc)
%EQUILIBRIO_CONVERSOR Regime permanente do modelo medio para (vpv, ipv) impostos.
%   Campos de saida: d, iL, Pcond, Psw, Pout, eta, viavel.
%
%   d = NaN e viavel = false significam "sem transferencia de potencia neste
%   ponto" (ipv <= 0, ou o controlador nao alcanca o ponto: d fora de
%   [Dmin, Dmax], por exemplo com vpv muito baixo).
%
%   ATENCAO: so vale em conducao continua (CCM). Use verifica_ccm para
%   sinalizar DCM. Em DCM as equacoes de CCM subestimam as perdas de conducao
%   (corrente de pico/RMS bem maior que a media), entao eta em DCM e otimista.
%
%   Psw entra apenas no balanco de potencia (e descontada de Pout); nao entra
%   no modelo medio / EDO (e pequena, < 1 % em STC).
%
%   Derivacao de d (regime: dv/dt = 0 e di/dt = 0):
%   1) dv/dt = 0  ->  ipv - d*iL = 0  ->  iL = ipv/d
%   2) di/dt = 0  ->  d*vpv - (1-d)*Vt - iL*(rL + d*Ron + (1-d)*RD) = 0,
%      com Vt = Vbat + VF.  Trocando iL por ipv/d:
%        d*vpv - (1-d)*Vt - (ipv/d)*(rL + d*Ron + (1-d)*RD) = 0
%   3) Multiplicando tudo por d (d > 0):
%        d^2*vpv - d*(1-d)*Vt - ipv*(rL + d*Ron + RD - d*RD) = 0
%   4) Abrindo os produtos e juntando as potencias de d:
%        d^2*vpv + d^2*Vt - d*Vt - ipv*d*(Ron - RD) - ipv*(rL + RD) = 0
%        (vpv + Vt)*d^2 - (Vt + ipv*(Ron - RD))*d - ipv*(rL + RD) = 0
%   5) E uma equacao do 2o grau a*d^2 + b*d + c = 0 com
%        a = vpv + Vt  (> 0),  b = -(Vt + ipv*(Ron - RD)),  c = -ipv*(rL + RD) (< 0)
%      Como a > 0 e c < 0, o produto das raizes c/a e negativo: ha exatamente
%      uma raiz positiva, d = (-b + sqrt(b^2 - 4*a*c)) / (2*a).
semtransf = struct('d', NaN, 'iL', 0, 'Pcond', 0, 'Psw', 0, 'Pout', 0, ...
                   'eta', 0, 'viavel', false);
if ipv <= 0
    eq = semtransf;
    return
end
```

```
Vt = pc.Vbat + pc.VF;
a = vpv + Vt;
b = -(Vt + ipv * (pc.Ron - pc.RD));
c = -ipv * (pc.rL + pc.RD);
d = (-b + sqrt(b^2 - 4 * a * c)) / (2 * a);

% Se o ponto exige d fora da faixa do PWM, o controlador nao o alcanca:
% nao ha transferencia de potencia util, entao devolvemos tudo zerado.
if d > pc.Dmax || d < pc.Dmin
    eq = semtransf;
    return
end

eq.d = d;
eq.viavel = true;
eq.iL = ipv / eq.d;
% Perda de conducao: I^2*R nas resistencias + queda fixa do diodo (so quando conduz)
eq.Pcond = eq.iL^2 * (pc.rL + eq.d * pc.Ron + (1 - eq.d) * pc.RD) ...
           + (1 - eq.d) * eq.iL * pc.VF;
% Perda de chaveamento: sobreposicao tensao x corrente nas transicoes
eq.Psw = 0.5 * (vpv + pc.Vbat) * eq.iL * pc.ttr * pc.fs;
% Potencia entregue a bateria = media da corrente de saida x Vbat, menos chaveamento
eq.Pout = (1 - eq.d) * eq.iL * pc.Vbat - eq.Psw;
eq.eta = eq.Pout / (vpv * ipv);
end
```

Lógica passo a passo

1. Se $i_{\text{pv}} \leq 0$ (noite): sem transferência, devolve tudo zerado e `viavel = false`.
2. `a`, `b`, `c` são os coeficientes de (16); `d` é a raiz positiva.
3. Se d sai de $[D_{min}, D_{max}]$ o controlador não alcança o ponto: também sem transferência.
4. **Perda de condução:** $P_{cond} = i_L^2(r_L + dR_{on} + (1-d)R_D) + (1-d)i_L V_F$.
5. **Perda de chaveamento:** $P_{sw} = \frac{1}{2}(v_{\text{pv}} + V_{bat})\, i_L\, (t_r + t_f)\, f_s$ (sobreposição tensão × corrente).
6. **Potência na bateria:** $P_{out} = (1-d)\, i_L V_{bat} - P_{sw}$; $\eta = P_{out}/(v_{\text{pv}} i_{\text{pv}})$.

P_{sw} entra só no balanço de potência, não na EDO média ($< 1\,\%$ em STC).

7.5 CCM e DCM

Código 15: `conversor/verifica_ccm.m`: teste de condução contínua.

```
function ccm = verifica_ccm(vpv, d, iL, pc)
%VERIFICA_CCM true se a corrente no indutor nao zera (conducao continua).
%   O ripple pico a pico e vpv*d/(L*fs); a corrente minima e iL - ripple/2.
%   Conducao continua exige iL > ripple/2.
%   O ripple usa a inclinacao ideal vpv/L da chave ligada (ignora as quedas
%   resistivas), o que e levemente conservador.
ccm = iL > vpv * d / (pc.L * pc.fs) / 2;
end
```

O ripple pico a pico é $v_{\text{pv}}\, d/(L f_s)$ e a condução é contínua (**CCM**) se $i_L >$ ripple/2. Em baixa luz ($50\,\text{W/m}^2$) a corrente média ($\sim$1 A) é menor que meio ripple ($\sim$3 A): o conversor entra em **condução descontínua (DCM)**. Ali o modelo médio de CCM deixa de valer e a eficiência calculada fica *otimista*, pois as perdas de condução dependem da corrente RMS, bem maior que a média.

7.6 Validação chaveado × médio (E3)

O experimento aplica um degrau de +0,02 em d, em $t = 2\,\text{ms}$, partindo do MPP, e compara o modelo médio com a média móvel de um período do chaveado.

- **Em STC (CCM):** o médio segue o chaveado. O teste de regressão exige desvio máximo $< 0{,}5\,\%$ na tensão e $< 1{,}5\,\%$ na corrente, e ripple de i_L dentro de $15\,\%$ do valor teórico.
- **Em $50\,\mathrm{W/m^2}$ (DCM):** as curvas se separam, e isso é o resultado esperado. Em DCM a energia puxada por ciclo é fixada por $(v_{\mathrm{pv}} dT_s)^2/(2L)$, muito maior que a potência do painel nessa luz; o médio chega a dar $i_L < 0$, impossível fisicamente. O experimento mostra o *limite de validade* do modelo médio.

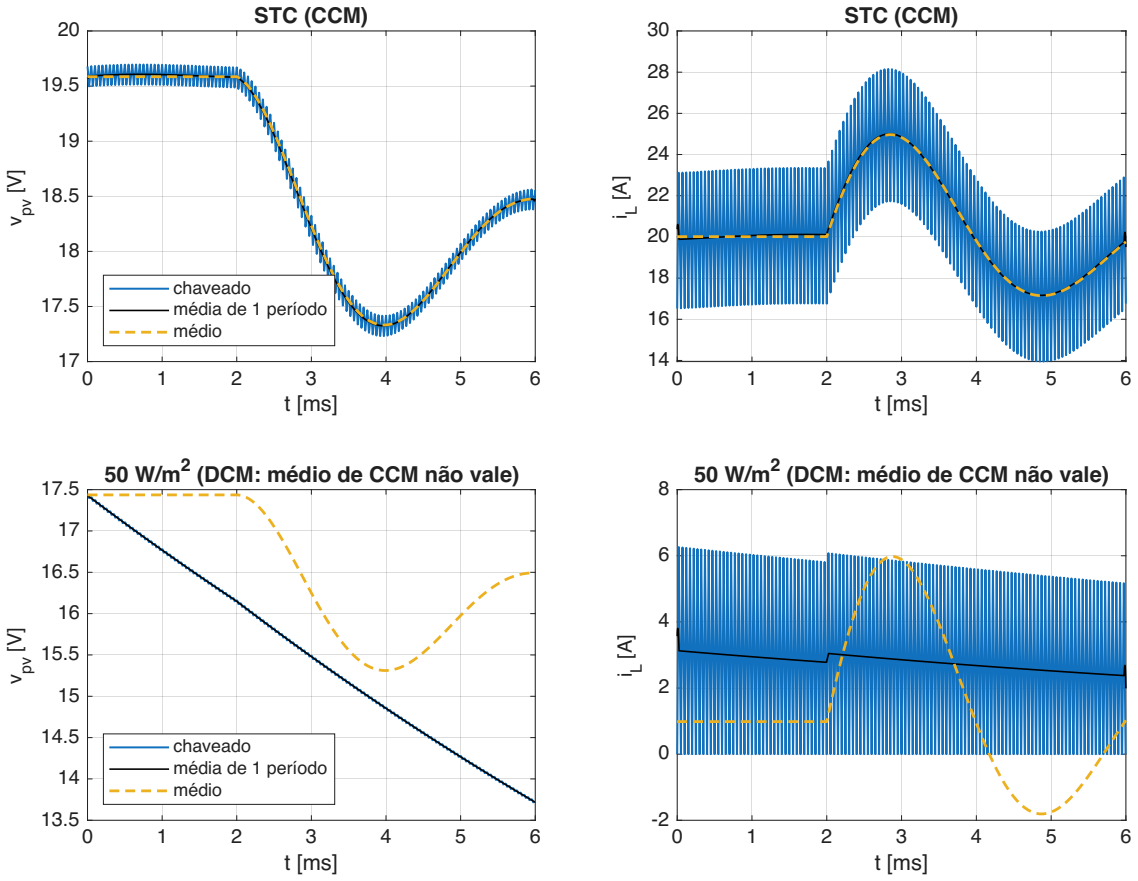


Figura 10: E3. Modelo chaveado (com ripple), média móvel de um período e modelo médio. Em cima, STC (CCM); embaixo, $50\,\mathrm{W/m^2}$ (DCM).

7.7 Malha aberta com DAE (`sim_medio_aberta`)

Integra o modelo médio mais o painel como DAE (equações diferenciais e algébricas) com `ode15s` e matriz de massa $M = \mathrm{diag}(1, 1, 0, \ldots, 0)$: as duas primeiras linhas são EDOs (v_{pv} e i_L) e as demais são as restrições algébricas do painel. O jacobiano analítico é obrigatório, pois com diferenças finitas a exponencial do bypass faz o `ode15s` falhar. Ela só é usada no E3; a malha fechada usa a EDO reduzida (Capítulo 10), porque a DAE falhou em 9 de 36 casos quando o MPPT dava degraus grandes.

8 Controle: linearização, resposta em frequência e projeto do PI

Arquivos: `controle/linearizar_planta.m`, `gvd_analitico.m`, `resposta_freq.m`, `margens.m`, `projeto_pi.m`, `resposta_degrau_mf.m`, `lei_pi.m`. O MPPT não mexe na razão cíclica d diretamente: ele escolhe uma tensão de referência V_{ref}, e um **PI de tensão** ajusta d para que v_{pv} siga V_{ref}. Este capítulo mostra como esse PI foi projetado.

8.1 A estrutura em dois laços

- **Laço interno (rápido):** PI que mede v_{pv} e ajusta d para seguir V_{ref}; escala de ms.
- **Laço externo (lento):** MPPT que, a cada T_{a}, escolhe um novo V_{ref}; escala de dezenas de ms.

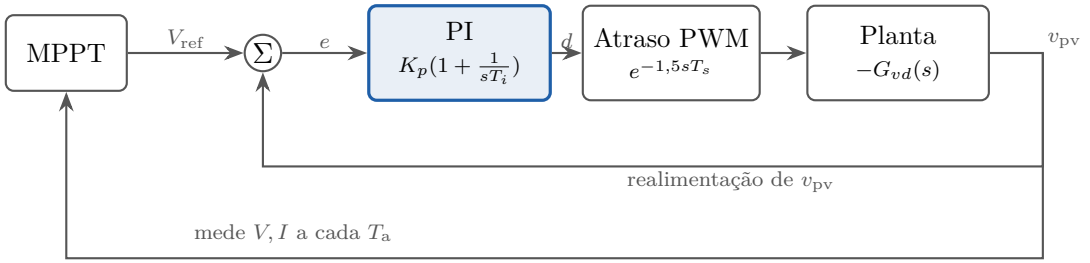


Figura 11: Malha de tensão do conversor. O ganho de malha é $L(s) = C(s)\,[-G_{vd}(s)]\,e^{-1,5sT_s}$; o sinal negativo da planta é absorvido na definição do erro.

O laço externo só funciona se o interno já tiver acomodado quando o MPPT medir a potência: esse é o critério que fixa T_{a} (Seção 8.7).

8.2 Linearização de pequenos sinais

O modelo médio é não linear (há o produto $d\,i_L$ e a curva I-V do painel). Para projetar um controlador clássico, ele é linearizado em torno de um equilíbrio (x_0, d_0), onde $f(x_0, d_0) = 0$. Escreve-se $x = x_0 + \tilde{x}$ e $d = d_0 + \tilde{d}$ com $\tilde{x}$ e $\tilde{d}$ pequenos; a série de Taylor de 1.º ordem dá

$$\dot{\tilde{x}} = A\tilde{x} + B\tilde{d}, \qquad A = \frac{\partial f}{\partial x}\Big|_{(x_0,d_0)}, \quad B = \frac{\partial f}{\partial d}\Big|_{(x_0,d_0)}, \quad \tilde{v}_{pv} = C\tilde{x}. \tag{17}$$

Código 16: `controle/linearizar_planta.m`, linhas 34 a 80: equilíbrio, matrizes A e B por diferenças centrais e r_{pv}.

```
34 % 1) Equilibrio: corrente do painel em vpv e razao ciclica que mantem vpv
35 [ipv0, sol] = painel_sombreado(op.vpv, op.G, op.T, p);
36 eq = equilibrio_conversor(op.vpv, ipv0, pc);
37 % Ponto inviavel: diz ao usuario qual das duas causas aconteceu
38 if ~eq.viavel
39     if ipv0 <= 0
40         causa = 'vpv acima de Voc (sem corrente do painel)';
41     else
42         causa = sprintf('d fora de [Dmin, Dmax] = [%.3g, %.3g]', pc.Dmin, pc.Dmax);
43     end
44     error('linearizar_planta:inviavel', ...
45         'linearizar_planta: ponto inviavel (vpv = %.4g V, ipv = %.4g A): %s.', ...
46         op.vpv, ipv0, causa);
47 end
48 x0 = [op.vpv; eq.iL];
49 d0 = eq.d;
50 z0 = sol.z;   % solucao interna do painel, usada como chute (warm start)
51
52 % 2) Matriz A: derivada de f em relacao a cada estado (coluna j <-> estado j)
53 A = zeros(2);
54 for j = 1:2
55     h = zeros(2, 1);
56     h(j) = 1e-5 * max(1, abs(x0(j)));   % passo relativo ao tamanho do estado
57     f_mais  = derivadas(x0 + h, d0, op, p, pc, z0);
58     f_menos = derivadas(x0 - h, d0, op, p, pc, z0);
59     A(:, j) = (f_mais - f_menos) / (2 * h(j));
60 end
61
62 % 3) Vetor B: derivada de f em relacao a d
63 hd = 1e-6;
64 B = (derivadas(x0, d0 + hd, op, p, pc, z0) ...
65    - derivadas(x0, d0 - hd, op, p, pc, z0)) / (2 * hd);
```

```
66
67 % 4) Resistencia dinamica do painel: r_pv = -dV/dI = -1 / (dI/dV).
68 %    O sinal "-" deixa r_pv positiva, pois a corrente do painel cai quando a
69 %    tensao sobe. No MPP, dP/dV = d(V*I)/dV = I + V*dI/dV = 0, logo
70 %    dI/dV = -I/V e r_pv = Vmp/Imp: a resistencia dinamica iguala a estatica.
71 %    E o principio do metodo da condutancia incremental (InC).
72 hv = 1e-4;
73 i_mais  = painel_sombreado(op.vpv + hv, op.G, op.T, p, z0);
74 i_menos = painel_sombreado(op.vpv - hv, op.G, op.T, p, z0);
75 rpv = -2 * hv / (i_mais - i_menos);
76
77 sys = struct('A', A, 'B', B, 'C', [1 0], 'D', 0, 'x0', x0, 'd0', d0, ...
78     'ipv0', ipv0, 'rpv', rpv);
79 % 5) Conducao continua? (o modelo linear so vale em CCM)
80 sys.ccm = verifica_ccm(x0(1), d0, x0(2), pc);
```

Lógica passo a passo

1. **Equilíbrio:** resolve o painel em `op.vpv` e acha (d_0, i_{L0}) com `equilibrio_conversor`. Se o ponto for inviável, dá erro dizendo se foi $v_{\text{pv}} > V_{oc}$ ou d fora de $[D_{min}, D_{max}]$.
2. **Matriz A:** perturba cada estado ($h = 10^{-5}\max(1, |x_j|)$) e deriva por $(f(x_0 + h) - f(x_0 - h))/(2h)$, com erro $O(h^2)$. Assim a mesma função serve com ou sem perdas e para qualquer padrão de sombra, sem derivar o painel à mão.
3. **Vetor B:** idem, perturbando d ($h = 10^{-6}$).
4. **Resistência dinâmica:** $r_{pv} = -1/(dI/dV)$. No MPP, $dP/dV = I + V\,dI/dV = 0$ e $r_{pv} = V_{mp}/I_{mp}$: a dinâmica iguala a estática; esse é o princípio do método InC.
5. **CCM?** O modelo linear só vale em condução contínua (`sys.ccm`).

8.3 A planta G_{vd} em forma fechada

Sem perdas, com o painel virando um resistor r_{pv}, as equações de pequenos sinais são

$$C\dot{\tilde{v}} = -\frac{\tilde{v}}{r_{pv}} - D\,\tilde{i}_L - I_L\,\tilde{d}, \qquad L\dot{\tilde{i}}_L = D\,\tilde{v} + (V + V_{bat})\,\tilde{d}. \tag{18}$$

Eliminando $\tilde{i}_L$ na transformada de Laplace:

$$G_{vd}(s) = \frac{\tilde{v}_{pv}}{\tilde{d}} = -\frac{LI_L\,s + D\,(V + V_{bat})}{LC\,s^2 + (L/r_{pv})\,s + D^2} \tag{19}$$

Código 17: `controle/gvd_analitico.m`: $G_{vd}(j\omega)$ sem perdas, para conferir a planta numérica.

```
1  function H = gvd_analitico(sys, pc, w)
2  %GVD_ANALITICO G_vd(jw) = v~_pv / d~ do buck-boost SEM perdas (expressao da proposta).
3  %   H = gvd_analitico(sys, pc, w)
4  %
5  %   sys : saida de linearizar_planta (usa d0, x0 = [Vpv; IL] e rpv)
6  %   pc  : parametros do conversor (L, C, Vbat)
7  %   w   : frequencias angulares [rad/s]
8  %
9  %   Derivacao (sem perdas: rL = Ron = RD = VF = 0):
10 %   1) Equacoes de estado do modelo medio:
11 %        C dv/dt = i_pv(v) - d*iL
12 %        L di/dt = d*v - (1-d)*Vbat
13 %   2) Pequenos sinais em torno de (V, IL, D). O painel vira um resistor:
14 %      i~_pv = -v~/r_pv. Os produtos d*iL e d*v dao dois termos cada:
15 %        C dv~/dt = -v~/r_pv - D*i~L - IL*d~
16 %        L di~/dt =  D*v~ + (V + Vbat)*d~
17 %   3) Laplace (condicao inicial nula):
18 %        C s v~ = -v~/r_pv - D i~L - IL d~
19 %        L s i~L = D v~ + (V + Vbat) d~   ->   i~L = (D v~ + (V+Vbat) d~)/(L s)
```

```
%   4) Eliminando i~L na primeira e multiplicando tudo por L s:
%        (L C s^2 + (L/r_pv) s + D^2) v~ = -(L IL s + D (V + Vbat)) d~
%      portanto
%        G_vd(s) = -(L IL s + D (V+Vbat)) / (L C s^2 + (L/r_pv) s + D^2)
%
%   Sinal negativo: aumentar d deixa a chave mais tempo ligada, o indutor
%   puxa mais corrente do capacitor de entrada e v_pv cai.
%   Polos: w0 = D/sqrt(L C) e amortecimento zeta = sqrt(L/C) / (2 D r_pv)
%   (o unico "resistor" sem perdas e o proprio painel).
s = 1j * w;
D = sys.d0;
IL = sys.x0(2);
Vsum = sys.x0(1) + pc.Vbat;
H = -(pc.L * IL * s + D * Vsum) ./ (pc.L * pc.C * s.^2 + (pc.L / sys.rpv) * s + D^2);
end
```

Três pontos de leitura:

- **Sinal negativo:** mais d = chave mais tempo ligada; o indutor puxa mais corrente do capacitor e v_{pv} cai.
- **Polos:** $\omega_0 = D/\sqrt{LC}$ e $\zeta = \sqrt{L/C}/(2D\,r_{pv})$. Com os valores do projeto isso dá a ressonância LC em ∼240 Hz, pouco amortecida (ζ de 0,16 a 0,3).
- **Amortecimento:** o único "resistor" do caso sem perdas é o próprio painel.

Código 18: `controle/resposta_freq.m`: $H(j\omega) = C(j\omega I - A)^{-1}B + D$, no lugar de `bode`/`freqresp`.

```
function H = resposta_freq(sys, w)
%RESPOSTA_FREQ Resposta em frequencia H(jw) de um sistema em espaco de estados.
%   H = resposta_freq(sys, w)
%
%   sys : struct com .A, .B, .C, .D (x' = A x + B u, y = C x + D u)
%   w   : vetor de frequencias angulares [rad/s]
%
%   Aplicando Laplace com condicao inicial nula: s X = A X + B U, logo
%   X = (sI - A)^-1 B U e Y/U = C (sI - A)^-1 B + D. A resposta em frequencia
%   e essa funcao avaliada em s = jw. Substitui bode/freqresp (Control System
%   Toolbox, nao disponivel). Usa "\" em vez de inv() por ser mais preciso.
H = zeros(size(w));
n = size(sys.A, 1);
for k = 1:numel(w)
    s = 1j * w(k);
    H(k) = sys.C * ((s * eye(n) - sys.A) \ sys.B) + sys.D;
end
end
```

8.4 Margens de estabilidade sem toolbox

`margens(w,L)` substitui `margin()`. Duas definições:

- **Margem de fase (MF):** na frequência ω_c onde $|L| = 1$, quanto falta para a fase chegar a $-180°$: $\text{MF} = 180° + \angle L(j\omega_c)$. Se a fase chegasse a $-180°$ com $|L| = 1$, $L = -1$ e $1 + L = 0$: polo de malha fechada sobre o eixo imaginário (oscilação sustentada). O usual é MF $\geq 45°$.
- **Margem de ganho (MG):** na frequência onde a fase vale $-180°$, quanto o ganho poderia crescer até $|L| = 1$: $\text{MG} = -20\log_{10}|L(j\omega_{180})|$. MG ≥ 6 dB significa que o ganho da planta pode dobrar sem instabilizar.

Código 19: `controle/margens.m`: margens de fase e ganho a partir de $L(j\omega)$ amostrado.

```
function m = margens(w, L)
%MARGENS Margens de fase e de ganho a partir de L(jw) amostrado em w.
%   m = margens(w, L)
%
%   w : vetor de frequencias angulares [rad/s], crescente
%   L : vetor com o ganho de malha L(jw) nessas frequencias
%
%   Substitui margin(), que e da Control System Toolbox (nao disponivel).
%
```

```
%   O que as margens significam (criterio de Nyquist simplificado):
%   - Margem de fase (MF): na frequencia de cruzamento de ganho wc, onde
%     |L| = 1, quanto falta para a fase chegar a -180 graus. Se a fase
%     chegasse a -180 com |L| = 1, L = -1 e 1 + L = 0: polo em malha fechada
%     sobre o eixo imaginario (oscilacao sustentada). MF = 180 + fase(wc).
%     MF baixa = resposta muito oscilatoria; MF >= 45 graus e o usual.
%   - Margem de ganho (MG): na frequencia w180, onde a fase vale -180 graus,
%     quanto o ganho poderia crescer ate |L| = 1. MG = -20 log10|L(w180)| dB.
%     MG >= 6 dB: o ganho da planta pode dobrar sem instabilizar.
%
%   m.wc, m.PM   : primeiro cruzamento de ganho [rad/s] e a menor MF [graus]
%                  entre todos os cruzamentos de ganho
%   m.w180, m.GM : primeiro cruzamento de fase [rad/s] e a menor MG [dB]
%                  entre todos os cruzamentos de fase
%   Quando o cruzamento nao existe na faixa de w: wc (ou w180) = NaN e a
%   margem correspondente = Inf.
%
%   Para L estavel em malha aberta, so importam os cruzamentos acima de wc;
%   aqui todos entram no minimo, o que e conservador (nunca otimista).
%
%   Por que olhar todos os cruzamentos: a ressonancia LC do conversor pode
%   fazer |L| voltar a passar de 1 depois do primeiro cruzamento. A margem
%   que vale e a pior delas.
magn = abs(L);
fase = unwrap(angle(L)) * 180 / pi;   % unwrap: fase continua, sem saltos de 360

% Cruzamento de ganho: |L| passa de >= 1 para < 1 (ou volta de < 1 para >= 1)
m.wc = NaN;  m.PM = Inf;
for k = 1:numel(w) - 1
    if (magn(k) >= 1) ~= (magn(k + 1) >= 1)
        % interpola em log|L|: no Bode |L| varia como potencia de w
        wck = interp1(log(magn(k:k + 1)), w(k:k + 1), 0);
        fase_wc = interp1(w(k:k + 1), fase(k:k + 1), wck);
        PMk = mod(180 + fase_wc + 180, 360) - 180;    % leva para [-180, 180)
        if isnan(m.wc), m.wc = wck; end
        m.PM = min(m.PM, PMk);
    end
end

% Cruzamento de fase: (fase + 180), levada para [-180, 180), muda de sinal.
% O teste abs(diff) < 180 descarta o salto artificial de +-180 da funcao mod.
% Usa "<= 0" para nao perder uma amostra caindo exatamente em -180 graus; a
% condicao fr(j+1) ~= 0 evita contar esse mesmo cruzamento duas vezes (ele
% fica com o par que comeca na amostra nula).
fr = mod(fase + 180 + 180, 360) - 180;
m.w180 = NaN;  m.GM = Inf;
for j = 1:numel(w) - 1
    if fr(j) * fr(j + 1) <= 0 && fr(j + 1) ~= 0 && abs(fr(j + 1) - fr(j)) < 180
        w180j = interp1(fr(j:j + 1), w(j:j + 1), 0);
        magn180 = exp(interp1(w(j:j + 1), log(magn(j:j + 1)), w180j));
        GMj = -20 * log10(magn180);
        if isnan(m.w180), m.w180 = w180j; end
        m.GM = min(m.GM, GMj);
    end
end
end
```

Lógica passo a passo

1. `fase = unwrap(angle(L))*180/pi`: fase contínua, sem saltos de 360°.
2. **Cruzamentos de ganho:** varre as amostras e detecta onde $|L|$ passa de ≥ 1 para < 1 (ou volta). Interpola em $\log|L|$, pois no Bode o módulo varia como potência de ω.
3. **Cruzamentos de fase:** detecta onde fase + 180° muda de sinal (descartando o salto artificial de ±180°).
4. O código olha **todos** os cruzamentos, não só o primeiro, e fica com a pior margem: a ressonância LC pode fazer $|L|$ voltar a passar de 1 depois do primeiro cruzamento.

8.5 O projeto do PI (D12 e D13)

O controlador é $C(s) = K_p(1+1/(sT_i))$, aplicado como $d = K_p(e+\xi/T_i)$ com $e = v_{\text{pv}} - V_{\text{ref}}$ e $\dot{\xi} = e$.

Atenção: por que a regra clássica falha

A regra usual põe o zero do PI uma década abaixo do cruzamento ($\omega_z = \omega_c/5$, $r = 0{,}2$). Aqui a planta é plana (40 a 85 V por unidade de d) até a ressonância LC. Com o zero abaixo de ω_c, acima dele o PI vira praticamente só o ganho K_p, e $|L|$ fica perto de 1 desde ω_c até a ressonância: o pico cruza 0 dB de novo com fase $\approx -150°$, e a MF cai para 25 a 40°. Subir f_c acima da ressonância também não adianta: ali a fase da planta já está perto de $-180°$.

O que funciona: zero *acima* do cruzamento ($r > 1$). Abaixo do zero o PI age como integrador puro, $|L|$ cai 20 dB/década, o cruzamento fica bem abaixo da ressonância e $|L| \ll 1$ no pico LC. A fase em ω_c é $\approx -90°$, então a MF é alta.

Código 20: `controle/projeto_pi.m`, linhas 114 a 170: varredura de f_c e r, escolha do maior f_c que passa e cálculo de T_a.

```
ctl = [];
fcs = logspace(log10(5), log10(pc.fs / 10), 80);    % bandas candidatas [Hz]
for i = 1:numel(fcs)
    fc = fcs(i);
    wc = 2 * pi * fc;
    % planta nominal em wc (nao depende de r)
    Gwc = -resposta_freq(sysn, wc) * exp(-1j * wc * pc.atraso);
    for r = razoes_zero
        Ti = 1 / (r * wc);                          % zero do PI em wz = r*wc
        % Kp: ganho de malha unitario em wc no ponto nominal
        Kp = 1 / abs(Gwc * (1 + 1 / (1j * wc * Ti)));
        Cw = Kp * (1 + 1 ./ (1j * w * Ti));         % PI na grade de frequencias
        PM = zeros(numel(Gplanta), 1);  GM = PM;
        for k = 1:numel(Gplanta)
            m = margens(w, Cw .* Gplanta{k});
            PM(k) = m.PM;  GM(k) = m.GM;
        end
        cand = struct('Kp', Kp, 'Ti', Ti, 'fc', fc, 'r', r);

        % Acomodacao em todos os casos (so se as margens passaram: e o
        % calculo mais caro). Femia vale no PIOR caso: o P&O e o PSO operam
        % justamente em baixa irradiancia e nos picos com sombra, onde a
        % malha e mais lenta.
        margens_ok = all(PM >= 45) && all(GM >= 6);
        ts = NaN(numel(sys_ccm), 1);
        ts_pior = NaN;
        if margens_ok
            for k = 1:numel(sys_ccm)
                [~, ~, ts(k)] = resposta_degrau_mf(sys_ccm{k}, cand, t_fim);
            end
            ts_pior = max(ts);
        end

        passa = margens_ok && ts_pior < ts_max_limite;
        % fcs e crescente: substitui o escolhido se a banda for maior, ou se
        % for a mesma banda com MF maior
        if passa && (isempty(ctl) || fc > ctl.fc || (fc == ctl.fc && min(PM) > rel.PMmin))
            ctl = cand;  ctl.ts = ts_pior;
            [~, k_pior] = max(ts);
            rel = struct('PMmin', min(PM), 'GMmin', min(GM), 'PM', PM, 'GM', GM, ...
                'ts', ts, 'ts_max', ts_pior, 'k_ts_max', k_pior);
        end
    end
end
if isempty(ctl)
    error('projeto_pi:inviavel', ...
        'Nenhum par (fc, r) atende MF >= 45, MG >= 6 dB e ts < %g s.', ts_max_limite);
end

% Criterio de Femia: o P&O compara a potencia antes e depois de cada passo
% de V_ref. Se o periodo Ta for menor que o tempo de acomodacao da malha de
% tensao, a potencia medida ainda inclui o transitorio do conversor e o P&O
```

```
% confunde essa dinamica com uma mudanca real de potencia (pode andar para o
% lado errado). Por isso Ta > ts no pior caso; usamos 1,5 ts_max
% arredondado para cima em ms.
ctl.Ta = 1e-3 * ceil(1.5 * rel.ts_max / 1e-3);
rel.ops = ops_ccm;  rel.ops_dcm = ops_dcm;
```

Lógica passo a passo

1. Antes deste trecho, o código monta os casos de robustez (próxima subseção) e linearia cada um uma só vez: a planta não depende do PI.
2. Para cada f_c (80 valores, de 5 Hz a $f_s/10$, espaçados em log) e cada $r \in \{0{,}2; 0{,}5; 1; 2; 5\}$: `Ti = 1/(r*wc)` e `Kp` tal que $|L(j\omega_c)| = 1$ no ponto nominal.
3. Calcula MF e MG em todos os casos. Se alguma ficar abaixo de 45° ou 6 dB, descarta o par.
4. Para os que passam, simula o degrau de V_{ref} em cada caso (`resposta_degrau_mf`) e calcula t_s (faixa de 2 %). Exige $t_{s,max} < 50$ ms, só para rejeitar malhas absurdamente lentas.
5. Entre os aprovados, fica o de **maior** f_c (empate: maior MF): banda maior = malha mais rápida = passos de MPPT mais curtos.
6. No fim, $T_{\text{a}} = 1{,}5\, t_{s,max}$ arredondado para cima em ms.

8.6 Robustez

Indutores e capacitores eletrolíticos têm tolerância típica de ±20 %. Por isso cada ponto de operação é verificado com L e C nominais e nos cantos $[f_L \; f_C] = [1{,}2 \; 1], [1 \; 1{,}2], [1{,}2 \; 1{,}2], [0{,}8 \; 0{,}8]$. São 12 pontos de operação:

- 1000, 600 e 200 W/m² uniformes, cada um no MPP e em $0{,}8V_{mp}$ e $1{,}1V_{mp}$ (o MPPT passeia em torno do MPP e o ganho muda com r_{pv});
- os dois picos da curva com um submódulo a 400 W/m²;
- o módulo quente (60 °C, 1000 W/m²) no seu MPP.

São $12 \times 5 = 60$ casos. Em 4 deles, em 200 W/m², o conversor cai em DCM e eles são excluídos (o modelo linear de CCM não vale ali); são conferidos depois na simulação não linear. O PI projetado só com L e C nominais tinha MG de 4,6 dB com $L + 20$ %, e por isso a robustez entrou no projeto.

8.7 Resultado e período do MPPT

Resultado

Grandeza	Valor
Banda de cruzamento f_c	56,6 Hz
Posição do zero $r = \omega_z/\omega_c$	5
K_p	$2{,}5 \times 10^{-3}$ V^{-1}
T_i	0,562 ms
Casos verificados em CCM	56 de 60
Pior MG nos 56 casos	6,2 dB (limita a banda; caso crítico: 200 W/m², $L + 20$ %)
Pior t_s (2 %)	20,2 ms
Período do MPPT $T_{\text{a}} = 1{,}5\, t_{s,max}$	31 ms

A MG de 6,2 dB é o que limita a banda. O pior t_s vem do pico de baixa tensão com sombra: o ganho DC da planta é ∼0,6 do nominal e, como K_p é fixado no nominal, o cruzamento cai na

mesma proporção. Escalonar o ganho (um K_p por ponto) tiraria essa limitação, mas ficou fora do escopo.

Intuição: critério de Femia

O P&O compara a potência antes e depois de cada passo de V_{ref}. Se T_{a} for menor que o tempo de acomodação da malha, a potência medida ainda inclui o transitório e o P&O confunde essa dinâmica com uma mudança real de potência, podendo andar para o lado errado. Por isso $T_{\text{a}} > t_s$ no pior caso; o código usa $1{,}5\,t_{s,max}$.

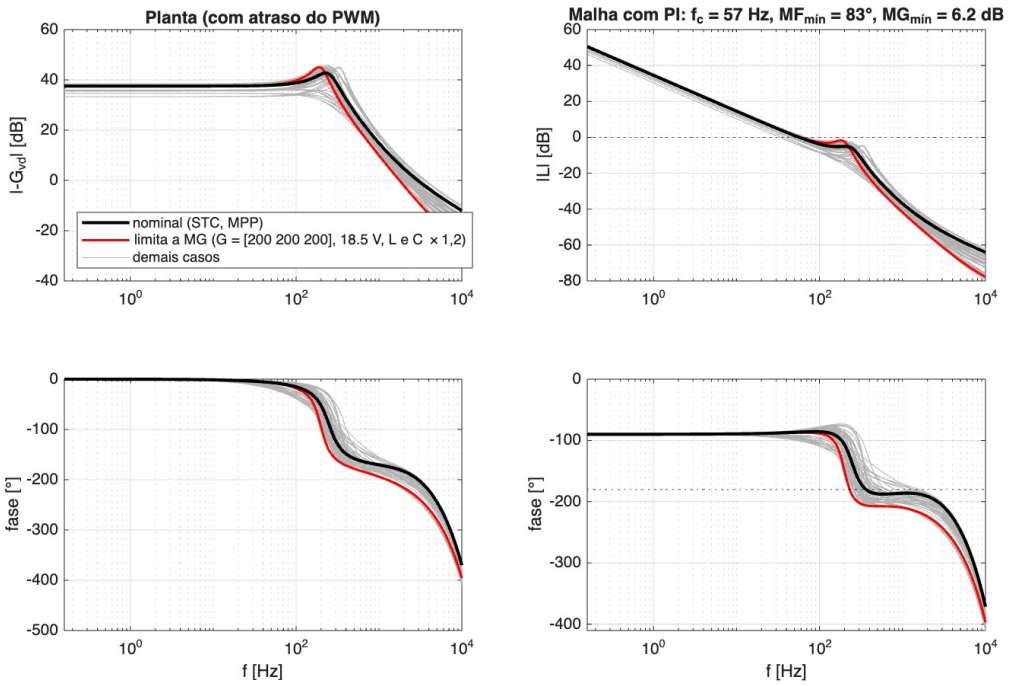


Figura 12: E4. Bode da planta e do ganho de malha com o PI. Preto: caso nominal; vermelho: o caso que limita a MG; cinza: os demais.

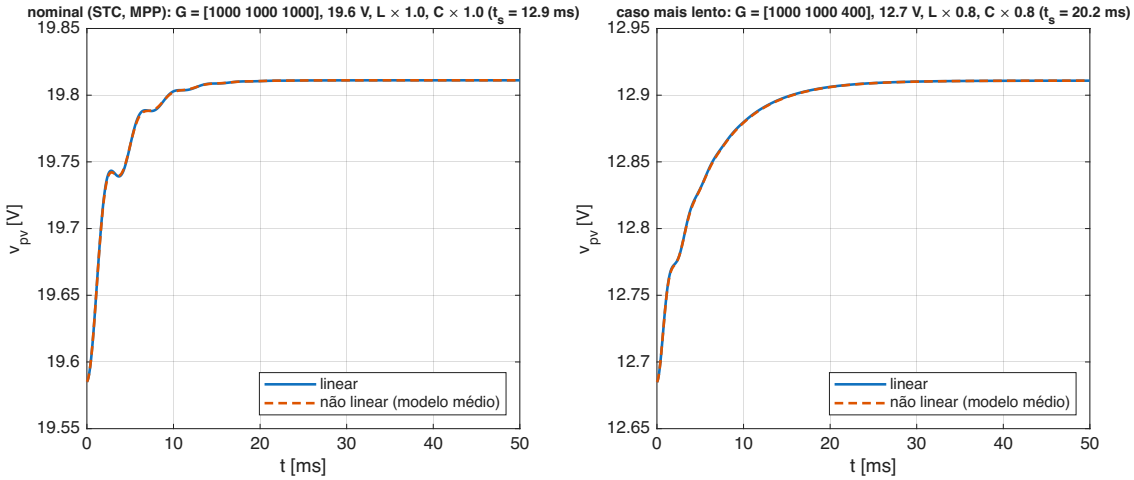


Figura 13: E4. Degrau de V_{ref}: modelo linear (usado no projeto) contra o modelo médio não linear.

8.8 A lei de controle com saturação

Código 21: `controle/lei_pi.m`: PI de tensão com saturação e anti-windup.

```
function [d, dxi, modo] = lei_pi(v, xi, Vref, ctl, pc, modo)
%LEI_PI Lei do PI de tensao com saturacao e anti-windup (modelo medio).
%   [d, dxi, modo] = lei_pi(v, xi, Vref, ctl, pc)
%   [d, dxi]       = lei_pi(v, xi, Vref, ctl, pc, modo)
%
%   Fica num arquivo proprio para que a simulacao dinamica (sim_dinamica) e
%   a quase-estatica usem exatamente a mesma lei de controle.
%
%   d = Kp*(e + xi/Ti), e = v - Vref (sinal invertido: mais d tira mais
%   corrente do painel e a tensao dele cai), saturado em [Dmin, Dmax].
%   dxi = derivada do integrador: e, ou 0 com o integrador congelado.
%   modo = [lado, integra]: lado = +1 (d em Dmax), -1 (d em Dmin) ou 0
%   (linear); integra = 1 se o integrador esta ativo, 0 se congelado.
%   Sem o argumento modo, ele e decidido pelo estado atual. Passar modo
%   congela a logica de saturacao (usado nos jacobianos por diferencas
%   finitas, para o passo de diferenca nao atravessar a descontinuidade).
err = v - Vref;
du = ctl.Kp * (err + xi / ctl.Ti);        % comando antes da saturacao
if nargin < 6
    lado = 0;
    if du > pc.Dmax
        lado = 1;
    elseif du < pc.Dmin
        lado = -1;
    end
    % Anti-windup: com d saturado, para de integrar se o erro empurraria
    % du ainda mais para fora da faixa (err > 0 aumenta du). Sem isso xi
    % cresceria sem limite e a malha demoraria a sair da saturacao.
    integra = ~((lado == 1 && err > 0) || (lado == -1 && err < 0));
    modo = [lado, integra];
end
if modo(1) == 1
    d = pc.Dmax;
elseif modo(1) == -1
    d = pc.Dmin;
else
    d = du;
end
dxi = modo(2) * err;
end
```

Lógica passo a passo

1. $d = K_p(\mathrm{err} + \xi/T_i)$, saturado em $[D_{min}, D_{max}] = [0{,}02;\ 0{,}95]$.
2. **Anti-windup:** com d saturado, o integrador para se o erro empurrar o comando ainda mais para fora da faixa. Sem isso, ξ cresceria sem limite e a malha demoraria a sair da saturação.
3. O argumento `modo` permite congelar a lógica de saturação; é usado nos jacobianos por diferenças finitas, para o passo não atravessar a descontinuidade.
4. A mesma função serve à simulação dinâmica e à quase-estática: as duas usam exatamente o mesmo controle.

9 Algoritmos de MPPT

Arquivos: `mppt/config_mppt.m`, `mppt_po.m`, `mppt_inc.m`, `mppt_pso.m`, `mppt_fixo.m`, `rastrear_estatico.m`. Os três algoritmos têm a mesma interface, `[Vref, e] = alg(V, I, e, cfg)`: recebem a tensão e a corrente médias do último período T_a, guardam o estado em `e` (vazio na primeira chamada) e devolvem a próxima tensão de referência. Isso permite trocar um pelo outro sem mudar a simulação.

9.1 Perturba e Observa (P&O)

O P&O é uma subida de encosta na curva $P(V)$. A cada período, a referência anda um passo δ. Se a potência subiu, continua no mesmo sentido; se caiu, inverte.

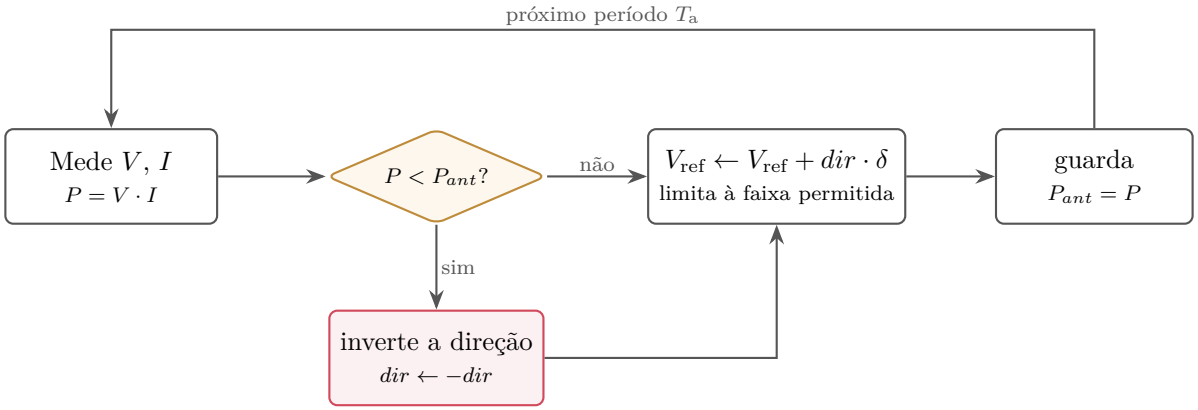


Figura 14: Lógica do P&O em cada período T_a.

- **Vantagem:** simples e rápido.
- **Oscila no MPP:** nunca para. Perto do pico fica alternando entre $V_{mp} - \delta$, V_{mp} e $V_{mp} + \delta$, o que custa um pouco de energia (∼0,1 % em STC no E5).
- **Preso em pico local:** só vê a inclinação local. Partindo de perto de V_{oc}, para no primeiro pico que encontra.

Código 22: `mppt/mppt_po.m`: Perturba e Observa.

```
function [Vref, e] = mppt_po(V, I, e, cfg)
%MPPT_PO Perturba e Observa: se a potencia caiu, inverte a direcao.
%   [Vref, e] = mppt_po(V, I, e, cfg)
%
%   V, I : media medida da tensao e corrente do painel no ultimo periodo Ta
%   e    : estado do algoritmo ([] na primeira chamada)
%   cfg  : struct de config_mppt()
%
%   E uma "subida de encosta" (hill climbing) na curva P(V): a cada periodo
%   a referencia anda um passo delta; se a potencia subiu, continua no mesmo
%   sentido, se caiu, volta. Consequencias:
%   - Nunca para: perto do MPP fica oscilando em 3 niveis de tensao
%     (Vmp - delta, Vmp, Vmp + delta), o que custa um pouco de energia.
%   - So enxerga a inclinacao local: sob sombra parcial a curva P-V tem
%     varios picos, e o P&O para no primeiro que encontra (partindo de perto
%     de Voc, o pico de tensao mais alta), mesmo que nao seja o global.

P = V * I;
if isempty(e)
    % Primeira chamada: nao ha potencia anterior para comparar. Comeca
    % descendo a tensao (partida perto de Voc, o MPP esta a esquerda).
    e = struct('Vref', cfg.Vref0, 'Pant', P, 'dir', -1);
else
    % A potencia caiu: o ultimo passo foi no sentido errado, inverte
    if P < e.Pant
        e.dir = -e.dir;
    end
    e.Pant = P;
end

% Da o passo e limita a faixa permitida. Ao bater num limite, a direcao
% passa a apontar para dentro da faixa: evita uma trava teorica no limite
% (referencia saturada e potencia constante, sem nunca inverter).
e.Vref = e.Vref + e.dir * cfg.delta;
if e.Vref <= cfg.Vmin
    e.Vref = cfg.Vmin;  e.dir = +1;
elseif e.Vref >= cfg.Vmax
```

```
    e.Vref = cfg.Vmax;  e.dir = -1;
end
Vref = e.Vref;
end
```

Lógica passo a passo

1. Na primeira chamada não há potência anterior: o estado começa em $V_{\text{ref},0} = 0{,}85V_{oc}$, descendo ($dir = -1$), pois o MPP está à esquerda de V_{oc}.
2. Nas seguintes: se `P < e.Pant`, o último passo foi no sentido errado e `e.dir` inverte. Depois guarda `P`.
3. Dá o passo `e.Vref + e.dir*cfg.delta` e limita à faixa permitida. Ao bater num limite, a direção passa a apontar para dentro, evitando uma trava teórica (referência saturada e potência constante, sem nunca inverter).

9.2 Condutância Incremental (InC)

No MPP, $dP/dV = 0$. Como $P = VI$, isso dá $I + V\,dI/dV = 0$, ou $dI/dV = -I/V$. Define-se

$$s = \frac{dI}{dV} + \frac{I}{V} = \frac{1}{V}\frac{dP}{dV} \tag{20}$$

- $s > 0$: está à esquerda do MPP, aumenta a tensão;
- $s < 0$: está à direita do MPP, diminui a tensão;
- $|s| < \varepsilon$: está no MPP, mantém a referência.

Por isso o InC não oscila, ao contrário do P&O. Como ele, só enxerga a inclinação local e também para num pico local.

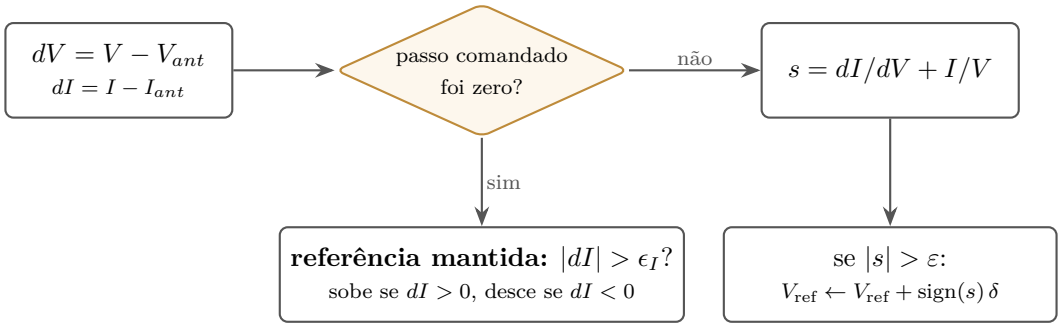


Figura 15: Lógica do InC. O ramo é escolhido pelo passo *comandado*, não pelo dV medido.

Código 23: `mppt/mppt_inc.m`: Condutância Incremental.

```
function [Vref, e] = mppt_inc(V, I, e, cfg)
%MPPT_INC Condutancia incremental: dI/dV = -I/V no MPP.
%   [Vref, e] = mppt_inc(V, I, e, cfg)   (mesma interface de mppt_po)
%
%   No MPP dP/dV = 0. Como P = V*I:
%       dP/dV = I + V*dI/dV = 0   =>   dI/dV = -I/V
%   Definindo s = dI/dV + I/V (= (dP/dV)/V):
%       s > 0 : a esquerda do MPP, aumenta a tensao
%       s < 0 : a direita do MPP, diminui a tensao
%       |s| < eps : esta no MPP, mantem a referencia (nao oscila, ao
%                   contrario do P&O)
%   Como o P&O, so enxerga a inclinacao local: tambem para num pico local
%   sob sombra parcial.

if isempty(e)
```

```
    % Primeira chamada: da um passo para ter dois pontos e calcular dI/dV
    e = struct('Vref', cfg.Vref0 - cfg.delta, 'dVref', -cfg.delta, 'Vant', V, 'Iant', I);
    e.Vref = min(max(e.Vref, cfg.Vmin), cfg.Vmax);
    Vref = e.Vref;
    return
end

dV = V - e.Vant;
dI = I - e.Iant;
VrefAnt = e.Vref;

% O ramo e escolhido pelo passo COMANDADO (e.dVref), nao pelo dV medido:
% em simulacao dinamica (ou com ruido), uma referencia mantida ainda gera
% dV de ~1e-5 a 1e-3 V; dividir dI por um dV tao pequeno da um s enorme e
% de sinal aleatorio, e o InC passaria a oscilar como o P&O.
if V <= 0
    % Capacitor de entrada descarregado na partida: I/V nao existe; sobe a tensao
    e.Vref = e.Vref + cfg.delta;
elseif e.dVref == 0 || dV == 0
    % Referencia mantida (ou a tensao ainda nao respondeu ao passo): se a
    % corrente mudou, foi a irradiancia que mudou. Mais luz (dI > 0)
    % desloca o MPP para a direita, entao sobe a tensao; menos luz, desce.
    if abs(dI) > cfg.epsI
        e.Vref = e.Vref + sign(dI) * cfg.delta;
    end
else
    s = dI / dV + I / V;                % = (dP/dV) / V
    if abs(s) > cfg.eps
        e.Vref = e.Vref + sign(s) * cfg.delta;
    end
end

e.Vref = min(max(e.Vref, cfg.Vmin), cfg.Vmax);
e.dVref = e.Vref - VrefAnt;             % passo efetivamente comandado
e.Vant = V;  e.Iant = I;
Vref = e.Vref;
end
```

Lógica passo a passo

1. Primeira chamada: dá um passo para ter dois pontos e poder calcular dI/dV.
2. $V \leq 0$ (capacitor descarregado na partida): I/V não existe; sobe a tensão.
3. **Referência mantida** (`e.dVref == 0`): se a corrente mudou mais que ϵ_I (0,2 % de I_{sc}), foi a irradiância: mais luz desloca o MPP para a direita, então sobe a tensão; menos luz, desce. O ramo é decidido pelo passo *comandado*, pois em simulação dinâmica uma referência mantida ainda gera dV de 10^{-5} a 10^{-3} V, e dividir dI por isso daria um s enorme e aleatório.
4. **Caso normal:** calcula s; se $|s| > \varepsilon$ anda δ no sentido de sign(s).

Escolha de ε (D14). Um passo δ muda s em cerca de $|ds/dV|\,\delta$. O ponto da grade mais próximo do MPP está a no máximo $\delta/2$ dele, então $|s|$ ali é no máximo metade desse salto; ε tem de ser pelo menos isso, ou o InC nunca para. Com $\varepsilon = 0{,}002$ S (valor inicial) isso nunca acontecia, pois o salto medido era de 0,09 a 0,10 S, e o InC oscilava como P&O. O código calcula $\varepsilon = \frac{1}{2}|ds/dV|\,\delta$ da curva de STC (ver `config_mppt`).

9.3 Enxame de partículas (PSO)

Cada partícula é uma tensão candidata x. O conversor só consegue impor uma tensão por vez; então as partículas são avaliadas **uma por período** T_a. Depois que as N foram medidas, o enxame se move:

$$v \leftarrow w\,v + c_1 r_1\,(p_{best} - x) + c_2 r_2\,(g_{best} - x), \qquad x \leftarrow x + v \tag{21}$$

Termo	Significado	Valor
$w\,v$	Inércia: continua andando no sentido em que ia	$w = 0{,}4$
$c_1 r_1 (p_{best} - x)$	Memória própria: volta ao melhor ponto que *esta* partícula achou	$c_1 = 1{,}2$
$c_2 r_2 (g_{best} - x)$	Memória do enxame: vai ao melhor ponto que *qualquer* partícula achou	$c_2 = 2{,}0$
r_1, r_2	Sorteios em $[0, 1]$	`RandStream` próprio

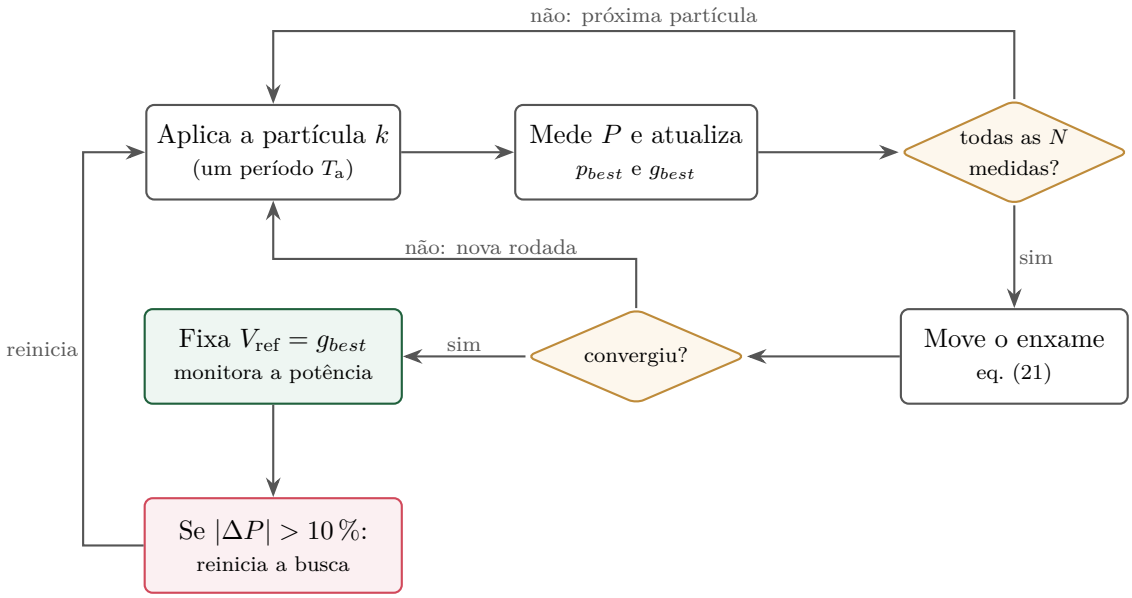


Figura 16: Máquina de estados do PSO. Reiniciar significa voltar às posições iniciais, com uma nova semente.

Código 24: `mppt/mppt_pso.m`: PSO com avaliação sequencial.

```
function [Vref, e] = mppt_pso(V, I, e, cfg)
%MPPT_PSO Enxame de particulas com avaliacao sequencial (uma particula por Ta).
%   [Vref, e] = mppt_pso(V, I, e, cfg)   (mesma interface de mppt_po)
%
%   Cada particula e uma tensao candidata x. O conversor so consegue impor
%   UMA tensao por vez, entao as particulas sao avaliadas uma por periodo
%   Ta: cada chamada recebe a medicao da particula aplicada na chamada
%   anterior. Depois que as N particulas foram medidas, o enxame se move:
%       v = w*v + c1*r1*(pbest - x) + c2*r2*(gbest - x)
%       x = x + v
%   - w*v       : inercia (continua andando no sentido em que ia)
%   - pbest - x : memoria propria (volta para o melhor ponto que ESTA
%                 particula ja achou)
%   - gbest - x : memoria do enxame (vai para o melhor ponto que QUALQUER
%                 particula ja achou)
%   r1, r2 sao sorteios em [0, 1]. Como as particulas comecam espalhadas
%   por toda a faixa de tensao, o enxame "ve" todos os picos da curva P-V e
%   tende a convergir para o global, ao contrario de P&O e InC.
%
%   Convergencia: quando todas as particulas estao a menos de tolV do
%   gbest (ou estourou maxIter), fixa a referencia no gbest.
%   Reinicio: convergido, monitora a potencia; se ela muda mais que
%   limiarReinicio (a sombra ou a irradiancia mudou), a curva P-V mudou e
%   a busca recomeca do zero. Cada reinicio usa a semente
%   cfg.seed + e.nReinicios: buscas independentes, mas reproduziveis.
%
%   Limitacoes conhecidas (versao basica do PSO, de proposito):
%   - Entre reinicios a tensao fica congelada no gbest; o Vmp real deriva
%     com a temperatura (~0,4 %/K) e essa pequena perda nao e corrigida.
%   - Qualquer mudanca de ~10 % na potencia dispara uma busca completa,
%     mesmo que o pico global nao tenha mudado de lugar (custo em energia).
%   - Durante uma busca nao ha teste de reinicio: sob rampas rapidas as
%     memorias (pbest, gbest) ficam velhas, de outra irradiancia.
```

```
34
35 P = V * I;
36 if isempty(e)
37     e = iniciar(cfg, 0);
38     Vref = e.x(1);                        % aplica a primeira particula
39     return
40 end
41
42 if e.convergido
43     if isnan(e.Pconv)
44         % Primeira medicao depois de convergir: vira a potencia de
45         % referencia do reinicio. Nao usa o gbestP da busca porque, sob
46         % rampa, ele pode ter sido medido com outra irradiancia.
47         e.Pconv = P;
48     elseif abs(P - e.Pconv) > cfg.limiarReinicio * max(e.Pconv, cfg.Pmin_ref)
49         e = iniciar(cfg, e.nReinicios + 1);   % a curva mudou: nova busca
50         Vref = e.x(1);
51         return
52     end
53     Vref = e.gbest;                       % fica parado no melhor ponto
54     return
55 end
56
57 % P e a potencia da particula k (aplicada no periodo anterior):
58 % atualiza a memoria propria e a do enxame
59 k = e.k;
60 if P > e.pbestP(k)
61     e.pbestP(k) = P;  e.pbest(k) = e.x(k);
62 end
63 if P > e.gbestP
64     e.gbestP = P;  e.gbest = e.x(k);
65 end
66
67 e.k = k + 1;
68 if e.k > cfg.N
69     % Todas as particulas foram medidas: move o enxame
70     e.k = 1;  e.iter = e.iter + 1;
71     r1 = rand(e.rs, cfg.N, 1);
72     r2 = rand(e.rs, cfg.N, 1);
73     e.v = cfg.w * e.v + cfg.c1 * r1 .* (e.pbest - e.x) + cfg.c2 * r2 .* (e.gbest - e.x);
74     e.x = min(max(e.x + e.v, cfg.Vmin), cfg.Vmax);
75     if max(abs(e.x - e.gbest)) < cfg.tolV || e.iter >= cfg.maxIter
76         e.convergido = true;
77         e.Pconv = NaN;                    % sera a proxima potencia medida
78         Vref = e.gbest;
79         return
80     end
81 end
82 Vref = e.x(e.k);                          % aplica a proxima particula
83 end
84
85 function e = iniciar(cfg, nReinicios)
86 % Estado inicial do enxame: particulas em cfg.x0, paradas, sem memoria.
87 % O gerador aleatorio e proprio (RandStream) para o resultado depender so
88 % de cfg.seed e do numero de reinicios, e nao do estado global do rand.
89 e.rs = RandStream('mt19937ar', 'Seed', cfg.seed + nReinicios);
90 e.nReinicios = nReinicios;
91 e.x = cfg.x0(:);
92 e.v = zeros(cfg.N, 1);
93 e.pbest = e.x;  e.pbestP = -inf(cfg.N, 1);
94 e.gbest = e.x(1);  e.gbestP = -inf;
95 e.k = 1;  e.iter = 0;
96 e.convergido = false;  e.Pconv = NaN;
97 end
```

Lógica passo a passo

1. **Primeira chamada:** `iniciar` cria o estado (partículas em `cfg.x0`, paradas, sem memória; gerador próprio com semente `seed + nReinicios`) e aplica a partícula 1.
2. **Convergido:** a primeira medição vira a potência de referência (`Pconv`); se depois $|P -$

$P_{conv}| > 10\,\% \cdot \max(P_{conv}, P_{min})$, a curva P-V mudou e a busca recomeça.

3. **Em busca:** a medição é da partícula `k` (aplicada no período anterior): atualiza `pbest` e `gbest`.
4. Quando todas as N foram medidas, move o enxame (eq. 21) com `rand(e.rs,…)` e limita à faixa.
5. Para (`convergido`) se todas estiverem a menos de `tolV` do `gbest` ou se `iter >= maxIter`; então a referência fica no `gbest`.

Posições iniciais (D14). Com N_{sub} diodos de bypass, os máximos locais ficam perto de múltiplos de V_{mp}/N_{sub} (cada submódulo ativo a mais soma $\sim$ uma tensão de MPP de submódulo; Patel e Agarwal, 2008). O código põe uma partícula em cada $k\,V_{mp}/N_{sub}$ ($k = 1, \ldots, N_{sub}$) mais uma em $0{,}9V_{oc}$: $N = N_{sub} + 1 = 4$ partículas. Antes, com partículas espaçadas linearmente, o PSO caía no pico local em cerca de 1/3 das sementes no padrão [1 0,6 0,3].

Limitações declaradas (versão básica, de propósito; ver D17): entre reinícios a tensão fica parada no `gbest`, mas o V_{mp} real deriva com a temperatura ($\sim 0{,}4\,\%$/K); qualquer mudança de $\sim 10\,\%$ na potência dispara uma busca completa, mesmo que o pico global não tenha mudado de lugar; durante a busca não há teste de reinício, e sob rampas rápidas as memórias ficam velhas.

9.4 Configuração comum

Código 25: `mppt/config_mppt.m`, linhas 44 a 63: parâmetros do PSO: inércia, atrações, partículas iniciais e critérios de convergência e reinício.

```
44 % --- PSO ---
45 cfg.w = 0.4;                    % inercia: quanto da velocidade anterior fica
46 cfg.c1 = 1.2;                   % atracao pela melhor posicao da propria particula
47 cfg.c2 = 2.0;                   % atracao pela melhor posicao do enxame
48 % Posicoes iniciais: com Nsub diodos de bypass, os maximos locais da curva
49 % P-V ficam perto de multiplos de Vmp/Nsub (cada submodulo ativo a mais
50 % soma ~uma tensao de MPP de submodulo; Patel e Agarwal 2008, Ishaque e
51 % Salam 2013). Uma particula em cada k*Vmp/Nsub (k = 1..Nsub) e mais uma
52 % em 0,9*Voc para cobrir o lado de alta tensao: N = Nsub + 1 particulas.
53 cfg.x0 = [(1:p.Nsub) * c.Vmp / p.Nsub, 0.90 * Voc].';
54 cfg.N = numel(cfg.x0);            % numero de particulas
55 cfg.tolV = 0.01 * Voc;            % convergiu quando todas estao a menos disso do gbest
56 cfg.maxIter = 15;                 % limite de iteracoes (cada uma custa N periodos Ta)
57 cfg.limiarReinicio = 0.10;        % reinicia se a potencia mudar mais de 10 %
58 % Piso do limiar de reinicio: 2 % da Pmp de STC. Sem ele, no amanhecer e
59 % no anoitecer (P de poucos W) qualquer variacao pequena passaria de 10 %
60 % e o PSO reiniciaria a busca repetidamente.
61 cfg.Pmin_ref = 0.02 * c.Pmp;
62 cfg.seed = 1;                     % semente do gerador aleatorio (reprodutibilidade)
63 end
```

Parâmetro	Valor	Motivo
$V_{\mathrm{ref},0}$	$0{,}85V_{oc}$	O conversor liga com o painel quase em aberto
Faixa de V_{ref}	$0{,}10V_{oc}$ a $0{,}95V_{oc}$	Abaixo de 10 % a potência é quase nula; acima de 95 % o painel está quase em aberto
Passo δ (P&O, InC)	$0{,}01V_{oc}$	Ajustado no E6
ε do InC	$\frac{1}{2}\lvert ds/dV\rvert\,\delta$	Ver acima
ϵ_I do InC	$0{,}002\,I_{sc}$	Distingue mudança de luz de ruído
`tolV`, `maxIter` (PSO)	$0{,}01V_{oc}$, 15	Critério de convergência
Limiar de reinício	10 % (piso 2 % de P_{mp})	Sem o piso, no amanhecer o PSO reiniciaria sem parar

9.5 Rastreamento estático (E5)

`rastrear_estatico` isola o algoritmo da dinâmica do conversor. Supõe que a malha de tensão assenta dentro de um período T_{a} (D13): a tensão do painel é exatamente a referência pedida e a

corrente vem direto da curva estática $I(V)$.

Código 26: `mppt/rastrear_estatico.m`: o laço do rastreamento estático.

```
function [r, e] = rastrear_estatico(alg, cfg, Gfun, T, p, nPassos)
%RASTREAR_ESTATICO MPPT sobre o mapa estatico (tensao assentada a cada passo).
%   [r, e] = rastrear_estatico(alg, cfg, Gfun, T, p, nPassos)
%
%   alg     : handle do algoritmo, ex. @mppt_po
%   cfg     : struct de config_mppt()
%   Gfun    : @(k) irradiancia 1 x Nsub no passo k [W/m^2]
%   T       : temperatura das celulas [K]
%   p       : struct de parametros_painel()
%   nPassos : numero de periodos Ta simulados
%
%   r.V, r.P : tensao aplicada e potencia medida em cada passo
%   e        : estado final do algoritmo (para inspecao nos testes)
%
%   Hipotese: a malha de tensao assenta dentro de um periodo Ta (D13), entao
%   a tensao do painel e exatamente a referencia pedida no passo anterior
%   e a corrente vem direto da curva estatica I(V). Isola o algoritmo de
%   MPPT da dinamica do conversor.

e = [];
V = cfg.Vref0;
z = [];
r.V = zeros(nPassos, 1);
r.P = zeros(nPassos, 1);
for k = 1:nPassos
    [I, s] = painel_sombreado(V, Gfun(k), T, p, z);
    z = s.z;                              % warm start para o proximo passo
    r.V(k) = V;
    r.P(k) = V * I;
    [V, e] = alg(V, I, e, cfg);           % nova referencia para o proximo passo
end
end
```

Lógica passo a passo

A cada passo: resolve o painel em V (com *warm start*), registra V e $P = VI$, e chama o algoritmo para obter o próximo V. Nada de EDO.

O PSO usa 10 sementes por padrão de sombra. A eficiência é $\eta = P_{rastreada}/P_{pico\ global}$:

Resultado

Padrão de irradiância	P&O	InC	PSO (10 sementes)
[1 1 1] (sem sombra)	99,9 %	99,97 %	100,0 %
[1 0,5 0,5]	99,9 %	99,98 %	100,0 %
[1 0,3 0,3]	99,9 %	100,0 %	100,0 %
[1 1 0,4]	**67,4 %**	**67,6 %**	99,99 a 100 %
[1 0,6 0,3]	**79,0 %**	**79,2 %**	99,99 a 100 %
[0,8 0,8 0,2]	**42,3 %**	**42,2 %**	99,99 a 100 %

Sem sombra, ou com sombra em que o pico global é o mais próximo de V_{oc} ([1 0,5 0,5] e [1 0,3 0,3]), os três acham o máximo. Quando o global fica abaixo do local mais perto de V_{oc}, P&O e InC ficam presos e perdem de 21 % até 58 %. O PSO acha o global em todas as 10 sementes de todos os padrões. O preço é o tempo: de 24 a 61 períodos para convergir, contra 1 a 7.

10 Simulação: dinâmica, quase-estática e oráculo

Arquivos: `simulacao/sim_dinamica.m`, `sim_quase_estatica.m`, `oraculo_pmax.m`, `oraculo_com_cache.m`. O sistema tem escalas de tempo muito diferentes e simular tudo no detalhe máximo é inviável. A solução foi uma **hierarquia de modelos**, cada um usado onde cabe.

10.1 As escalas de tempo

Fenômeno	Escala de tempo
Chaveamento do conversor	50 μs (f_s = 20 kHz)
Ressonância LC	~4 ms (240 Hz)
Laço do PI (acomodação)	~20 ms
Período do MPPT T_a	31 ms
Térmico do painel	~7 min
Dia	13 h de sol

O térmico é milhares de vezes mais lento que o MPPT, e o MPPT é mais lento que o PI. Essa separação permite os modelos reduzidos.

10.2 A hierarquia de modelos

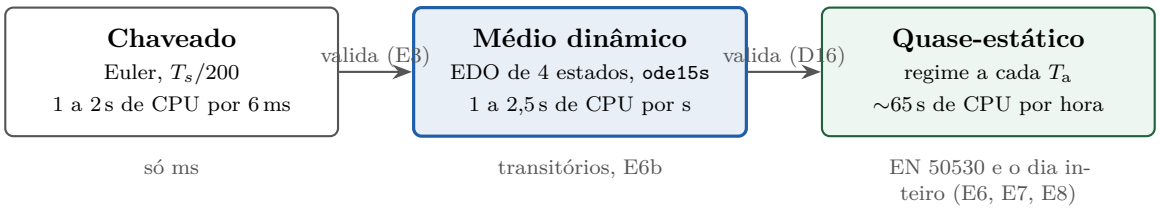


Figura 17: Cada modelo mais simples é conferido contra o mais detalhado, no intervalo em que ambos rodam.

10.3 Simulação dinâmica

Estado. $x = [v_{pv}; i_L; \xi; T]$: tensão do capacitor de entrada, corrente do indutor, integral do erro do PI e temperatura das células. A corrente do painel I *não* é estado: o painel não tem dinâmica própria nessa escala, e I é a solução das equações algébricas do módulo (Capítulo 5) para a tensão v_{pv} do momento. O `ode15s` integra a EDO reduzida $\dot{x} = f(x, I(x))$.

Atenção: por que EDO reduzida e não DAE (D15)

A formulação original era uma DAE com matriz de massa singular, como em `sim_medio_aberta`. Ela falhou em 9 de 36 casos de teste, nos degraus grandes do PSO sob sombra: o `ode15s` reaproveita o jacobiano e a condutância dos diodos varia exponencialmente (10^{-2} a 10^{2} S). A EDO reduzida, com o painel resolvido por Newton a cada avaliação, teve 0 falhas em 36 casos e eficiências iguais em 4 dígitos. O preço é ficar 2 a 4 vezes mais lenta.

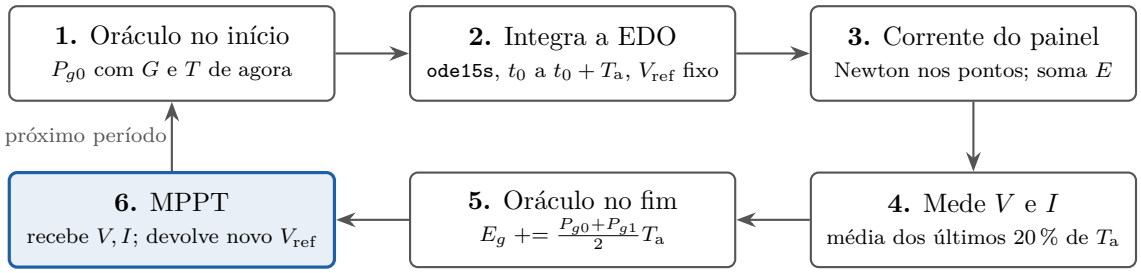


Figura 18: O laço de `sim_dinamica`, repetido a cada T_a. No fim, $\eta = E/E_g$.

Código 27: `simulacao/sim_dinamica.m`, linhas 132 a 193: o laço amostrado: um intervalo T_a por iteração.

```
132 for k = 1:nK
133     t0 = (k - 1) * Ta;
```

```
    t1 = t0 + Ta;
    % Oraculo na ponta inicial (G e T no inicio do intervalo). G e lida um
    % instante DENTRO do intervalo (dt = 1e-9 Ta): um degrau de G exatamente
    % na fronteira entre dois periodos conta inteiro para o periodo em que
    % ele vale, e nao pela metade
    dt = 1e-9 * Ta;
    [Pg0, cache] = oraculo_com_cache(cen.G(t0 + dt), x(4), p, cache);

    % Aviso de Newton sem convergencia: zera o ultimo aviso e confere no
    % fim do periodo (conta periodos com aviso, nao avisos individuais)
    lastwarn('', '');

    % V_ref fica constante dentro do intervalo (segurador de ordem zero)
    f = @(t, x) rhs(t, x, Vref, m);
    opts = odeset(base, 'Jacobian', @(t, x) jacobiano(t, x, Vref, m));
    [tt, xx] = ode15s(f, [t0, t1], x, opts);
    if tt(end) < t1 - 1e-9 * Ta
        error('sim_dinamica:ode15s', 'ode15s parou em t = %.6f s (periodo %d)', tt(end), k);
    end
    iLmin = min(iLmin, min(xx(:, 2)));
    x = xx(end, :).';
    % O diodo nao deixa i_L ficar negativa: corta o erro numerico residual
    x(2) = max(x(2), 0);

    % Corrente do painel em cada ponto aceito pelo integrador (pos-
    % processamento: aqui o warm start e seguro, os pontos estao em ordem)
    Ik = zeros(numel(tt), 1);
    z = [];
    for i = 1:numel(tt)
        [Ik(i), sol] = painel_sombreado(xx(i, 1), cen.G(tt(i)), xx(i, 4), p, z);
        z = sol.z;
    end

    % Energia tirada do painel no periodo: integral de v_pv * I
    E = E + trapz(tt, xx(:, 1) .* Ik);

    % Medida do MPPT: media nos ultimos 20 % do periodo
    Vm = media_janela(tt, xx(:, 1), t0 + 0.8 * Ta);
    Im = media_janela(tt, Ik, t0 + 0.8 * Ta);

    % Oraculo na ponta final e trapezio: Eg += (Pg0 + Pg1)/2 * Ta
    [Pg1, cache] = oraculo_com_cache(cen.G(t1 - dt), x(4), p, cache);
    Pg = (Pg0 + Pg1) / 2;
    Eg = Eg + Pg * Ta;

    [~, id] = lastwarn;
    if strcmp(id, 'painel_sombreado:naoConvergiu')
        nAvisos = nAvisos + 1;
    end

    % Conducao continua no fim do periodo (estado medio)
    dk = lei_pi(x(1), x(3), Vref, ctl, pc);
    r.ccm(k) = verifica_ccm(x(1), dk, x(2), pc);

    r.t(k) = t1;  r.V(k) = Vm;  r.I(k) = Im;  r.P(k) = Vm * Im;
    r.T(k) = x(4);  r.Vref(k) = Vref;  r.Pg(k) = Pg;  r.d(k) = dk;

    % MPPT: nova referencia para o proximo periodo
    [Vref, e] = alg(Vm, Im, e, cfg);
end
```

Lógica passo a passo

1. `t0`, `t1`: início e fim do período. `oraculo_com_cache` avalia P_{g0} na ponta inicial (G lida $10^{-9}T_{\mathrm{a}}$ *dentro* do intervalo: um degrau de G na fronteira conta para o período certo).
2. `ode15s(f, [t0 t1], x, opts)`: integra com V_{ref} constante (segurador de ordem zero), passando o jacobiano analítico.
3. Pós-processamento: para cada ponto aceito, resolve o painel com *warm start* (aqui os pontos estão em ordem, então é seguro) e soma $E = \int v_{\mathrm{pv}} I \, dt$ por trapézio.

4. `media_janela`: média de V e I nos últimos 20 % de T_{a}, esperando o PI acomodar (Femia) e filtrando a oscilação residual.
5. Oráculo na ponta final: $E_g \mathrel{+}= \frac{P_{g0}+P_{g1}}{2} T_{\mathrm{a}}$.
6. `lei_pi` e `verifica_ccm` registram d e se o conversor estava em CCM; por fim o MPPT devolve o novo `Vref`.

Código 28: `simulacao/sim_dinamica.m`, linhas 224 a 260: lado direito da EDO: PI, conversor, térmico e bloqueio do diodo.

```
224 function f = derivadas(t, x, I, Vref, m, modo, bloqueado)
225 % [dv_pv/dt; di_L/dt; dxi/dt; dT/dt] com a corrente do painel I dada.
226 % modo (PI) e bloqueado (diodo) opcionais: quando passados, ficam
227 % congelados (jacobiano por diferencas finitas, ver jacobiano).
228 v = x(1);  iL = x(2);  xi = x(3);  T = x(4);
229 if nargin < 6
230     [d, dxi] = lei_pi(v, xi, Vref, m.ctl, m.pc);
231 else
232     [d, dxi] = lei_pi(v, xi, Vref, m.ctl, m.pc, modo);
233 end
234 if m.termico
235     % G do balanco termico = media das irradiancias dos submodulos; a
236     % potencia eletrica v*I sai do modulo e nao vira calor
237     dT = termico_dTdt(T, mean(m.cen.G(t)), m.cen.Tamb(t), m.cen.vw(t), v * I, m.pt);
238 else
239     dT = 0;
240 end
241 dx = conversor_medio_rhs(v, iL, I, d, m.pc);
242 % Diodo bloqueia corrente reversa: o indutor so descarrega na bateria pelo
243 % diodo, entao a corrente media nao pode ficar negativa (a bateria nao
244 % empurra corrente de volta). Com i_L ja em 0 e a derivada apontando para
245 % baixo (pouca irradiancia: d*v_pv nao vence Vbat), i_L fica parada em 0.
246 if nargin < 7
247     bloqueado = iL <= 0 && dx(2) < 0;
248 end
249 if bloqueado
250     dx(2) = 0;
251 end
252 f = [dx; dxi; dT];
253 end
254
255 function f = rhs(t, x, Vref, m)
256 % Lado direito da EDO reduzida: resolve o painel em v_pv (sem warm start,
257 % ver cabecalho) e aplica derivadas.
258 I = painel_sombreado(x(1), m.cen.G(t), x(4), m.p);
259 f = derivadas(t, x, I, Vref, m);
260 end
```

Lógica passo a passo

`derivadas` monta $[\dot{v_{\mathrm{pv}}}; \dot{i}_L; \dot{\xi}; \dot{T}]$: `lei_pi` dá d e $\dot{\xi}$; `conversor_medio_rhs` dá $\dot{v_{\mathrm{pv}}}$ e $\dot{i}_L$; `termico_dTdt` dá $\dot{T}$ (com G médio dos submódulos e a potência elétrica $v_{\mathrm{pv}} I$ saindo do módulo). O diodo bloqueia corrente reversa: se $i_L \leq 0$ e a derivada aponta para baixo, é zerada. `rhs` só resolve o painel em v_{pv} e chama `derivadas`.

Cuidados numéricos.

- **Sem *warm start* no lado direito**, de propósito: o `ode15s` avalia fora de ordem e em passos que depois rejeita; guardar a última solução poderia dar resultados que dependem dessa ordem.
- **Jacobiano pela regra da cadeia:** $\partial f/\partial x = \partial f/\partial x|_I + \partial f/\partial I \cdot \partial I/\partial x$. dI/dv e dI/dT vêm de derivação implícita do resíduo do painel com o jacobiano analítico, mais preciso que diferenças finitas perto dos joelhos do bypass.
- **Descontinuidades** (saturação do PI, anti-windup, bloqueio do diodo): no jacobiano, o modo do PI e o estado do diodo são decididos uma vez e congelados nas avaliações perturbadas. Não

há funções de evento: o `ode15s` atravessa a descontinuidade rejeitando passos.

- **Passo máximo** $T_a/20$: vários pontos na janela de medida e menor erro do trapézio.
- **Pré-condição:** começa com $G > 0$; com $G = 0$ o conversor não tem equilíbrio e a partida é recusada (`sim_dinamica:inicioInviavel`).
- **Condição inicial:** módulo em equilíbrio térmico com o conversor desligado; integrador do PI no valor que dá exatamente o d de equilíbrio (partida sem tranco).

10.4 Oráculo do máximo global

A eficiência do MPPT é $\eta = E/E_g$, onde E_g é a energia do pico global. Um MPPT preso num pico local deve aparecer com $\eta < 1$.

Código 29: `simulacao/oraculo_pmax.m`: potência máxima global do painel.

```
function [Pg, Vg] = oraculo_pmax(G, T, p)
%ORACULO_PMAX Potencia maxima GLOBAL do painel (referencia da eficiencia do MPPT).
%   [Pg, Vg] = oraculo_pmax(G, T, p)   G: 1 x Nsub [W/m^2], T [K]
%
%   Com sombra a curva P-V tem varios maximos locais. A eficiencia de
%   rastreamento compara a energia obtida com a do pico GLOBAL: um MPPT
%   preso num pico local deve aparecer com eficiencia < 1.
%   1) Varredura grossa da curva (150 pontos): acha em qual pico esta o
%      maximo global (um otimizador local sozinho poderia parar no outro).
%   2) Refinamento com fminbnd entre os vizinhos do melhor ponto da grade:
%      dentro desse intervalo P(V) tem um maximo so.
%   Noite (todas as G < 1 W/m^2) ou curva sem potencia positiva: Pg = 0,
%   Vg = 0 (nao ha o que rastrear).
if max(G) < 1
    Pg = 0;  Vg = 0;
    return
end
c = curva_pv(G, T, p, 150);
[Pk, k] = max(c.P);
if isempty(Pk) || Pk <= 0
    Pg = 0;  Vg = 0;
    return
end
a = c.V(max(k - 1, 1));
b = c.V(min(k + 1, numel(c.V)));
if a >= b
    % Curva com um ponto so: nao ha intervalo para refinar
    Pg = Pk;  Vg = c.V(k);
    return
end
menosP = @(v) -v * painel_sombreado(v, G, T, p);
[Vg, nP] = fminbnd(menosP, a, b, optimset('TolX', 1e-6));
Pg = -nP;
% Garantia: o refinamento nunca devolve menos que o melhor ponto da grade
if Pk > Pg
    Pg = Pk;  Vg = c.V(k);
end
end
```

Lógica passo a passo

1. **Varredura grossa** da curva P-V (150 pontos): descobre em que pico está o máximo global. Um otimizador local sozinho poderia parar no outro.
2. **Refinamento com `fminbnd`** entre os vizinhos do melhor ponto da grade: dentro desse intervalo $P(V)$ tem um só máximo.
3. De noite ($\max G < 1$) devolve $P_g = 0$. O refinamento nunca devolve menos que o melhor ponto da grade.

Código 30: `simulacao/oraculo_com_cache.m`: o oráculo só recalcula se G ou T mudaram.

```
function [Pg, cache] = oraculo_com_cache(G, T, p, cache)
%ORACULO_COM_CACHE Potencia do oraculo com cache.
%   [Pg, cache] = oraculo_com_cache(G, T, p, cache)   G: 1 x Nsub [W/m^2], T [K]
%   cache: G, T e Pg da ultima chamada; comeca vazio com
%          struct('G', [], 'T', NaN, 'Pg', NaN)
%
%   oraculo_pmax so e recalculado se G ou T mudaram (0,01 K muda Pmp em
%   ~4e-5, desprezivel). Com G constante, a ponta final de um intervalo e a
%   inicial do seguinte caem no cache. Usada por sim_dinamica e
%   sim_quase_estatica.
if ~isequal(G, cache.G) || abs(T - cache.T) > 0.01
    cache.Pg = oraculo_pmax(G, T, p);
    cache.G = G;
    cache.T = T;
end
Pg = cache.Pg;
end
```

10.5 Simulação quase-estática

Em cada T_a, trata-se o sistema como em **regime permanente**: a tensão do painel já é igual a V_{ref} e a potência é $P = V_{\text{ref}} \cdot I(V_{\text{ref}})$ durante o período todo. Não há EDO elétrica para integrar.

Código 31: `simulacao/sim_quase_estatica.m`, linhas 142 a 222: o laço do modelo quase-estático.

```
for k = 1:nK
    t0 = op.t_ini + (k - 1) * Ta;
    t1 = t0 + Ta;
    Ga = cen.G(t0 + dt);      % irradiancia na ponta inicial
    Gb = cen.G(t1 - dt);      % irradiancia na ponta final

    % --- Ponta inicial: painel em V_ref (PI assentado) ---
    [Ia, sol] = painel_sombreado(Vref, Ga, T, p, z);
    z = sol.z;
    naoConvergiu = sol.res > tolNewton;
    % O diodo do conversor nao deixa corrente voltar para o painel
    Ia = max(Ia, 0);
    Pa = Vref * Ia;

    % --- Oraculo ---
    if passoOr == 1
        [Pga, cache] = oraculo_com_cache(Ga, T, p, cache);
    elseif mod(k - 1, passoOr) == 0
        % Inicio de um bloco: avalia no ponto medio do bloco e segura.
        % O ultimo bloco pode ter menos periodos (nK nao multiplo de passoOr):
        % o meio e o do bloco REAL, para a regra do ponto medio continuar valendo.
        nBloco = min(passoOr, nK - k + 1);
        tMeio = t0 + nBloco * Ta / 2;
        PgSegurado = oraculo_pmax(cen.G(tMeio), T, p);
    end

    % --- Termico: um passo de Euler de tamanho Ta ---
    % Forcantes (G, Tamb, vento) no mesmo instante, logo dentro do periodo
    if op.termico
        Tb = T + Ta * termico_dTdt(T, mean(Ga), cen.Tamb(t0 + dt), cen.vw(t0 + dt), Pa, pt);
    else
        Tb = T;
    end

    % --- Ponta final ---
    [Ib, sol] = painel_sombreado(Vref, Gb, Tb, p, z);
    z = sol.z;
    naoConvergiu = naoConvergiu || sol.res > tolNewton;
    Ib = max(Ib, 0);
    Pb = Vref * Ib;
    if naoConvergiu
        nAvisos = nAvisos + 1;      % conta periodos, como em sim_dinamica
    end

    % Energia do painel no periodo (trapezio, como a integral da dinamica)
    E = E + (Pa + Pb) / 2 * Ta;

```

```
    % Energia de referencia (oraculo)
    if passoOr == 1
        [Pgb, cache] = oraculo_com_cache(Gb, Tb, p, cache);
        Pg = (Pga + Pgb) / 2;
    else
        Pg = PgSegurado;
    end
    Eg = Eg + Pg * Ta;

    % Energia entregue a bateria (trapezio); DCM/inviavel na ponta final,
    % como a conducao continua em sim_dinamica
    Pouta = potencia_saida(Vref, Ia, pc, op.perdas);
    [Poutb, situacao] = potencia_saida(Vref, Ib, pc, op.perdas);
    Eout = Eout + (Pouta + Poutb) / 2 * Ta;
    if ~strcmp(situacao, 'semCorrente')
        nTransf = nTransf + 1;
    end
    if strcmp(situacao, 'dcm')
        nDCM = nDCM + 1;
    elseif strcmp(situacao, 'inviavel')
        nInviavel = nInviavel + 1;
    end

    T = Tb;
    % Series: so a cada n_log periodos
    if mod(k, op.n_log) == 0
        j = k / op.n_log;
        r.t(j) = t1;  r.Vref(j) = Vref;  r.I(j) = Ib;
        r.P(j) = Pb;  r.Pg(j) = Pg;  r.T(j) = T;
    end

    % MPPT: mede (V_ref, I) no fim do periodo e escolhe a proxima referencia
    [Vref, e] = alg(Vref, Ib, e, cfg);
end
```

Lógica passo a passo

1. **Ponta inicial:** resolve o painel em `Vref` com G e T do início; o diodo impede corrente negativa (`Ia = max(Ia,0)`).
2. **Oráculo:** se `passoOr == 1` avalia nas duas pontas; senão, no início de cada bloco, avalia *uma vez* no ponto médio do bloco e segura (ver abaixo).
3. **Térmico:** um passo de Euler explícito de tamanho T_a, `Tb = T + Ta*termico_dTdt(...)`. A potência elétrica sai do módulo e não vira calor.
4. **Ponta final:** resolve o painel em `Vref` com G e T do fim.
5. **Energia:** $E \mathrel{+}= (P_a + P_b)/2 \cdot T_a$ (trapézio, igual à dinâmica) e $E_g \mathrel{+}= P_g T_a$.
6. **Bateria:** `potencia_saida` usa `equilibrio_conversor` para obter P_{out} com perdas e classifica o ponto: CCM, DCM, inviável ou sem corrente.
7. O MPPT recebe (`Vref`, `Ib`) e escolhe o próximo `Vref`.

Opções. `op.termico` (obrigatória: com ou sem aquecimento), `op.perdas` (conversor com ou sem perdas), `op.dt_oraculo`, `op.t_ini` e `op.n_log` (guarda as séries só a cada n períodos, para economizar memória). As duas primeiras permitem a **ablação** do E7: ligar e desligar cada efeito e ver quanto ele pesa.

Oráculo em blocos (D16). O oráculo é a parte cara. Com `dt_oraculo`$= T_a$ custa $\sim$2500 s de CPU por hora simulada; com 1 s custa $\sim$65 s. Para isso os períodos são agrupados em blocos de round(1 s/T_a) períodos e o oráculo é avaliado **uma vez por bloco, no ponto médio**, e segurado. Numa rampa de G o erro de primeira ordem se cancela (o que falta numa metade sobra na outra), restando $\sim h^3/24\,|P_g''|$ por bloco de duração h; segurar o valor do início daría erro $\sim h^2/2\,|P_g'|$, isto é, um atraso. Com $dt = 60$ s o custo cai para $\sim$20 s por hora, mas o oráculo atrasa nas rampas ($\eta \approx 1{,}02$ numa rampa de $\sim 0{,}5\,\mathrm{W/m^2/s}$): por isso o valor de 1 s.

10.6 Validade do quase-estático contra a dinâmica (D16)

O que o quase-estático ignora é a energia do transitório depois de cada degrau de V_{ref}. O teste `test_quase_estatica` mede isso contra `sim_dinamica`:

Resultado

Algoritmo	Erro de energia do quase-estático
P&O e InC	$\leq 0{,}4\,\%$ (cerca de $+0{,}1\,\%$)
PSO buscando o tempo todo (janela de 4 s com reinício)	$-1{,}1\,\%$
PSO numa janela típica de 20 s (busca de 1 a 2 s e depois segura o g_{best})	cerca de $-0{,}3\,\%$

O sinal negativo para o PSO tem causa física. Depois de um degrau grande de V_{ref}, o transitório do PI passa por tensões *entre* as partículas, perto do pico, com mais potência que nas pontas; o quase-estático só conta a potência em V_{ref}. Logo o erro é **conservador contra o PSO**: o resultado do PSO no E7 é, se algo, um pouco pior que o real.

11 Cenários: EN 50530 e o dia em Florianópolis

Arquivos: `cenarios/cenario_constante.m`, `perfil_en50530.m`, `dia_florianopolis.m`, `aplicar_sombras.m`, `baixar_pvgis.m`, `data/en50530_rampas.csv`, `data/clima/pvgis_floripa_2020.csv`. Todo cenário é uma *struct* com quatro campos, chamados do mesmo jeito pelos simuladores: `cen.G(t)` (vetor $1 \times N_{sub}$ com a irradiância de cada submódulo), `cen.Tamb(t)`, `cen.vw(t)` (vento) e `cen.t_fim`. Isso permite trocar o cenário sem mudar a simulação.

11.1 Cenário constante

`cenario_constante(G, Tamb, vw, t_fim)` devolve sempre os mesmos valores. É usado em testes e em experimentos curtos, como o E5.

11.2 Perfil dinâmico da EN 50530

A EN 50530 é a norma que define como medir a eficiência dinâmica de um MPPT. Ela aplica ao inversor uma sequência de rampas trapezoidais de irradiância, de lentas ($0{,}5\,\text{W/m}^2/\text{s}$) a rápidas ($100\,\text{W/m}^2/\text{s}$). A irradiância é *uniforme*: a norma testa o MPPT sem sombreamento parcial.

A fonte (D5). O texto da norma (CENELEC) é pago e não foi consultado. As tabelas B.2, B.3 e B.4 foram transcritas do relatório de ensaio TÜV Rheinland n.° CN21VE2D 001 (tabela 4.4, "Dynamic MPPT Efficiency", p. 15 e 16). Os valores conferem aritmeticamente com as durações impressas: duração = repetições $\cdot$(2 rampa+2 patamar)+ espera. Uma inconsistência foi encontrada e documentada no CSV: na B.4 o relatório imprime rampa de 980 s, mas $(100-10)/0{,}1 = 900$ s, e o código calcula a rampa pela inclinação.

Sequência	Faixa de G [W/m^2]	Inclinações [W/m^2/s]
B.2	100 a 500	0,5 / 1 / 2 / 3 / 5 / 7 / 10 / 14 / 20 / 30 / 50
B.3	300 a 1000	10 / 14 / 20 / 30 / 50 / 100
B.4	10 a 100	0,1

Patamares de 10 s (30 s na B.4), espera de 300 s no início de cada linha e de 1 a 10 repetições; duração total de cerca de 7 h.

Código 32: `cenarios/perfil_en50530.m`: monta $G(t)$ como lista de pontos interpolada linearmente.

```
function cen = perfil_en50530(arq, Nsub, Tamb, vw)
%PERFIL_EN50530 Irradiancia uniforme G(t) dos ensaios dinamicos da EN 50530 (Anexo B).
%   Cada linha da tabela gera: espera em G_baixo e depois 'repeticoes' trapezios
%   (rampa de subida, patamar em G_alto, rampa de descida, patamar em G_baixo).
```

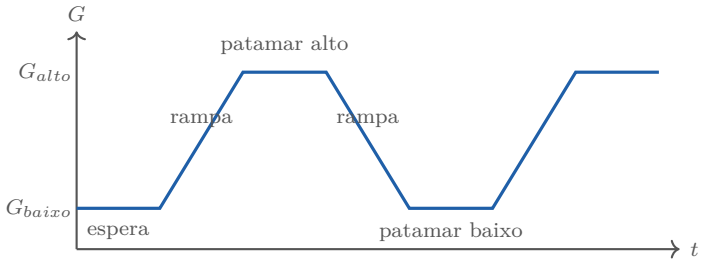


Figura 19: Estrutura de cada linha da tabela: espera em G_{baixo} e depois um número de trapézios (rampa, patamar alto, rampa, patamar baixo).

```
%   G(t) e montado como uma lista de pontos (tempo, irradiancia) interpolada
%   linearmente entre eles. A irradiancia e a mesma em todos os submodulos
%   (a norma testa o MPPT sem sombreamento parcial).
%   As funcoes do cenario aceitam t escalar (como os simuladores chamam).
if nargin < 3, Tamb = 298.15; end    % 25 C, condicao de ensaio
if nargin < 4, vw = 1; end
tab = readtable(arq, 'CommentStyle', '%', 'TextType', 'string');
% O interpolador exige instantes estritamente crescentes: a espera precisa ser maior
% que o degrau de 1 ms, e rampas e patamares precisam ter duracao positiva.
assert(all(tab.espera_s > 1e-3) && all(tab.patamar_s > 0) && all(tab.G_alto > tab.G_baixo), ...
    'Tabela EN 50530 invalida: espera > 1 ms, patamar > 0 e G_alto > G_baixo em todas as linhas');

t_pts = [];  G_pts = [];  t = 0;
for i = 1:height(tab)
    Gb = tab.G_baixo(i);  Ga = tab.G_alto(i);
    rampa = (Ga - Gb) / tab.inclinacao_Wm2s(i);      % duracao da rampa [s]
    patamar = tab.patamar_s(i);

    % Inicio da linha: degrau para G_baixo e espera (1 ms para o degrau ser vertical)
    t_pts(end + 1) = t + 1e-3;           G_pts(end + 1) = Gb;   %#ok<AGROW>
    t = t + tab.espera_s(i);
    t_pts(end + 1) = t;                  G_pts(end + 1) = Gb;   %#ok<AGROW>

    for rep = 1:tab.repeticoes(i)
        t = t + rampa;    t_pts(end + 1) = t;  G_pts(end + 1) = Ga;  %#ok<AGROW> sobe
        t = t + patamar;  t_pts(end + 1) = t;  G_pts(end + 1) = Ga;  %#ok<AGROW> fica no alto
        t = t + rampa;    t_pts(end + 1) = t;  G_pts(end + 1) = Gb;  %#ok<AGROW> desce
        t = t + patamar;  t_pts(end + 1) = t;  G_pts(end + 1) = Gb;  %#ok<AGROW> fica no baixo
    end
end
t_pts = [0, t_pts];  G_pts = [G_pts(1), G_pts];

interp_G = griddedInterpolant(t_pts, G_pts, 'linear', 'nearest');
cen.G = @(tq) interp_G(tq) * ones(1, Nsub);          % mesma irradiancia em todos
cen.Tamb = @(tq) Tamb;
cen.vw = @(tq) vw;
cen.t_fim = t;
end
```

Lógica passo a passo

1. Lê o CSV (`readtable` com comentários `%`) e valida: espera > 1 ms, patamar > 0, $G_{alto} > G_{baixo}$.
2. Para cada linha: o degrau inicial leva 1 ms (o interpolador exige instantes estritamente crescentes), depois a espera; a rampa dura $(G_{alto} - G_{baixo})$/inclinação.
3. Cada repetição adiciona 4 pontos (sobe, patamar alto, desce, patamar baixo).
4. `griddedInterpolant` interpola linearmente; G é a mesma em todos os submódulos. $T_{amb} = 25\,°\text{C}$ e vento 1 m/s, porque a norma separa o MPPT do efeito térmico.

11.3 O dia em Florianópolis

Para o dia real o trabalho usa dados do **PVGIS** (base PVGIS-SARAH3), no ponto de lat. −27,595°, lon. −48,548° e altitude 34 m. O módulo está inclinado a 27° e voltado para o norte. O dia escolhido foi **26/01/2020** (céu claro de verão). `dia_florianopolis` faz:

1. Lê o CSV do PVGIS (tem cabeçalho e rodapé de texto: só as linhas que começam com a data `yyyyMMdd:HHmm` são dados).
2. Converte de UTC para a hora local (UTC−3), para $t = 0$ ser a meia-noite local.
3. Usa as colunas $G(i)$ (irradiância no plano), T_{2m} (temperatura do ar) e WS_{10m} (vento a 10 m).
4. Interpola as amostras horárias de forma suave (`pchip`).
5. Converte o vento de 10 m para o vento junto ao módulo (∼2 m) pela lei de potência de expoente 1/7: fator $(2/10)^{1/7} \approx 0{,}79$.
6. Define a janela de sol: instantes com $G > 5\,\mathrm{W/m^2}$.

11.4 Nuvens e sombras sintéticas (D6)

Os dados horários não têm transitórios rápidos nem sombras. Por isso foram criados eventos **sintéticos**, declarados como tais no relatório e gravados em `resultados/e7_eventos.csv`.

Hora	Duração	Evento	Submódulo	Fator	Rampa
10:30 a 10:36	6 min	Nuvem	Todos	0,4	20 s
11:15 a 11:16:12	72 s	Nuvem	Todos	0,3	10 s
14:00 a 14:45	45 min	Sombra de poste	3	0,3	60 s
14:45 a 15:00	15 min	Sombra de poste	2	0,3	60 s
15:00 a 15:30	30 min	Poste (mais fraca)	3	0,5	60 s

As duas nuvens cobrem o módulo todo e testam o rastreamento sob mudança rápida de luz. A sombra de poste anda sobre o módulo e cria, com um submódulo sombreado, uma curva com dois picos: o trecho em que P&O e InC podem ficar presos e o PSO deve ganhar. Nas transições de 60 s os dois eventos se sobrepõem.

Código 33: `cenarios/aplicar_sombras.m`: multiplica G por um fator com rampas lineares.

```
function cen = aplicar_sombras(cen, ev)
%APLICAR_SOMBRAS Multiplica G por 'fator' entre t_ini e t_fim (rampas de rampa_s).
%   ev e uma tabela com colunas t_ini, t_fim, sub, fator, rampa_s.
%   ev.sub = 0 -> todos os submodulos (nuvem); sub = j -> so o submodulo j (sombra
%   de poste, folha etc.). O fator entra e sai linearmente em rampa_s segundos.
%   As funcoes do cenario aceitam t escalar (como os simuladores chamam).
%   Os dados horarios do PVGIS nao tem sombras nem nuvens rapidas; por isso estes
%   eventos sao SINTETICOS e devem ser declarados como tais no relatorio.
Gbase = cen.G;
cen.G = @(t) G_com_sombras(t);
    function G = G_com_sombras(t)
        G = Gbase(t);                    % Gbase e avaliada uma unica vez
        f = ones(size(G));
        for i = 1:height(ev)
            % a: (subida) - (descida). A subida vai de 0 a 1 em rampa_s a partir de
            % t_ini; a descida so comeca em t_fim e anula a subida: o resultado e um
            % trapezio no tempo (0 antes, rampa, 1 no patamar, rampa, 0 depois).
            a = min(max((t - ev.t_ini(i)) / ev.rampa_s(i), 0), 1) ...
              - min(max((t - ev.t_fim(i)) / ev.rampa_s(i), 0), 1);
            fi = 1 - (1 - ev.fator(i)) * a;
            if ev.sub(i) == 0
                f = f * fi;
            else
                f(ev.sub(i)) = f(ev.sub(i)) * fi;
            end
        end
        G = G .* f;
```

```
    end
end
```

Lógica passo a passo

A variável `a` é um trapézio no tempo: rampa de subida (de 0 a 1 em `rampa_s` segundos a partir de `t_ini`) menos rampa de descida (que começa em `t_fim`). O fator aplicado é $f = 1-(1-\text{fator})\,a$. Com `sub = 0` afeta todos os submódulos (nuvem); com `sub = j`, só o submódulo j (poste). `Gbase` é avaliada uma única vez por chamada.

12 Experimentos e resultados

Cada experimento é um script `matlab/experimentos/expNN_*.m` que grava tabelas em `resultados/` (CSV) e figuras em `relatorio/figs/`. Esta seção resume o que cada um mede e conclui.

12.1 Visão geral

Exp.	O que mede	Resultado principal	Cap.
E1	Validação do painel e identificabilidade	Erro de V_{oc} 0,03 %; γ_P −0,363 contra −0,4 %/K; eficiência em 200 W/m^2 entre 93,1 e 94,8 %	4
E2	Curvas I-V e P-V sob sombra	Até 3 picos; o global pode estar em qualquer um	5
E3	Chaveado contra médio	Concordam em CCM; separam-se em DCM	7
E4	Bode, margens e degrau do PI	$f_c = 56{,}6$ Hz, MG mínima 6,2 dB, $t_s = 20{,}2$ ms, $T_a = 31$ ms	8
E5	MPPT estático com sombra	P&O/InC chegam a 42 %; PSO $\geq$ 99,99 %	9
E6	MPPT na EN 50530	P&O 99,9 %, InC 99,6 %, PSO 98,6 %	12.2
E6b	Trecho da EN 50530 no modelo dinâmico	Dinâmico e quase-estático concordam	12.3
E7	Dia real com sombras e ablação	PSO 98,7 %; térmico pesa 9 %, perdas pesam 11 %	12.4
E8	Modelo anterior contra completo	O anterior superestima a energia em 21,9 %	12.5

12.2 E6: MPPT na EN 50530 (sem sombra)

O perfil de cerca de 7 h é simulado no quase-estático, com $T = 25\,^{\circ}$C fixa e oráculo a cada 1 s. A eficiência é $\eta = E/E_g$, com $E_g = 352{,}95$ Wh disponíveis. O passo δ do P&O e do InC é o compromisso clássico: δ grande segue rampas rápidas mas oscila mais em torno do MPP; δ pequeno oscila pouco mas fica para trás. Por isso varreu-se $\delta = 0{,}5$; 1 e 2 % de V_{oc} (ε do InC é reescalado junto).

Resultado

Algoritmo	δ [% V_{oc}]	η	Energia na bateria [Wh]	Reinícios
P&O	0,5	99,92 %	323,85	—
P&O	1	99,90 %	323,77	—
P&O	2	99,62 %	322,92	—
InC	0,5	98,79 %	319,93	—
InC	1	99,16 %	321,17	—
InC	2	99,58 %	322,64	—
PSO	—	98,62 %	319,67	2182

Como ler. Sem sombra, P&O e InC têm de 99,6 % a 99,9 % e o PSO é 1,3 ponto pior. O motivo: o PSO reinicia a busca a cada rampa (2182 reinícios no perfil) e, a cada busca, passa vários períodos avaliando partículas longe do pico. Quando a luz só varia de forma uniforme não há pico

global escondido, e esse custo não compensa. O conversor fica em DCM em cerca de 30 % do tempo, por causa das irradiâncias baixas do perfil (10 a 100 W/m^2).

12.3 E6b: conferência no modelo dinâmico

Um trecho curto da sequência B.3 (G de 300 a 1000 W/m^2, 50 W/m^2/s, 2 trapézios, 106 s) roda também na `sim_dinamica`, com a espera inicial encurtada para 10 s:

Resultado

Algoritmo	η dinâmico	η quase-estático
P&O	99,88 %	99,88 %
InC	99,49 %	99,49 %
PSO	97,79 %	97,62 %

Os dois modelos concordam em P&O e InC, e para o PSO o quase-estático fica 0,2 ponto abaixo, no sentido conservador do Capítulo 10. Os "dentes" do PSO na figura são as buscas.

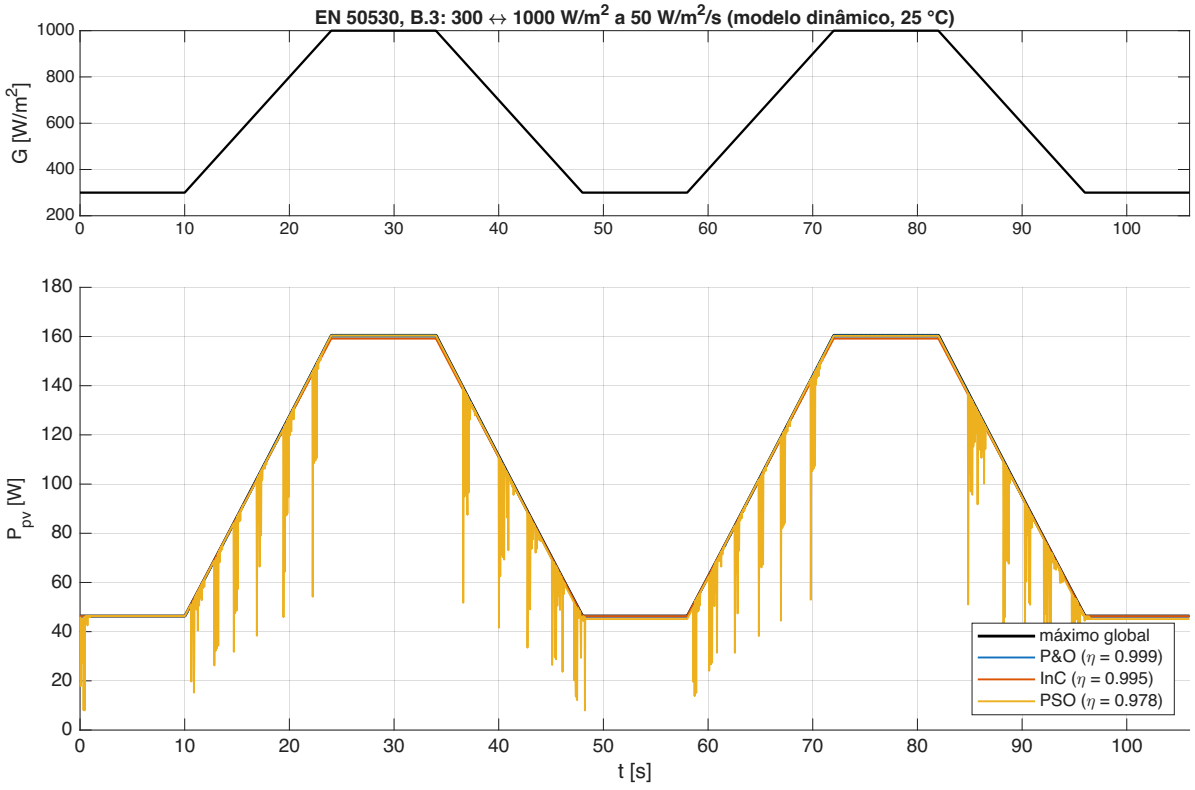


Figura 20: E6b. Como cada algoritmo segue as rampas no modelo dinâmico.

12.4 E7: dia em Florianópolis com sombras e ablação

O dia 26/01/2020 com os eventos sintéticos, no modelo completo, com 3 algoritmos × 3 variantes (completo, sem térmico, sem perdas) = 9 rodadas de 9 a 23 min de CPU.

Resultado

Variante	Algoritmo	E_{pv} [Wh]	E_{bat} [Wh]	η	ΔE_{bat}
Completo	P&O	1018,5	913,4	95,7 %	—
Completo	InC	997,6	896,5	93,8 %	—
Completo	PSO	1050,8	**941,7**	**98,7 %**	—
Sem térmico (25 °C)	P&O	1108,8	993,7	95,6 %	+8,8 %
Sem térmico (25 °C)	InC	1091,2	976,8	94,1 %	+9,0 %
Sem térmico (25 °C)	PSO	1143,7	1020,8	98,6 %	+8,4 %
Sem perdas	P&O	1018,5	1018,5	95,7 %	+11,5 %
Sem perdas	InC	997,6	997,6	93,8 %	+11,3 %
Sem perdas	PSO	1050,8	1050,8	98,7 %	+11,6 %

- **O PSO ganha na sombra.** Na sombra de poste (14:00 a 15:00), P&O e InC ficam no pico de alta tensão, com cerca de metade da potência disponível; o PSO reinicia, acha o global e o segue. A energia não captada, de ∼1064 Wh disponíveis, é de 13,8 Wh para o PSO (1,3 %), 45,7 Wh para o P&O (4,3 %) e 66,2 Wh para o InC (6,2 %).
- **Na bateria**, o PSO entrega 28,3 Wh (3,1 %) a mais que o P&O e 45,2 Wh (5,0 %) a mais que o InC.
- **O aquecimento pesa 9 %.** O módulo chega a 55,6–55,7 °C; ignorá-lo aumenta a energia disponível em 9,0 % e a da bateria em 8,4 a 9,0 %.
- **As perdas do conversor pesam 11 %.** Ignorá-las aumenta a energia na bateria em 11,3 a 11,6 %; a eficiência média do conversor no dia é de 89,6 a 89,9 %; o conversor fica em DCM em ∼13 % dos períodos.
- **Os dois efeitos são maiores que a diferença entre o melhor e o pior algoritmo** (5,0 %) e não mudam o ordenamento: um modelo isotérmico ou sem perdas erra a energia mais do que a escolha do algoritmo.
- O PSO reiniciou 104 vezes no dia. Os eventos de sombra reduzem a energia disponível de cerca de 1138 Wh (o mesmo dia sem eventos) para 1064 Wh.

12.4.1 Três limitações que só o dia real revela (D17)

Ficaram documentadas como resultado, sem alterar os algoritmos (corrigi-las exigiria rodar E6 e E7 de novo, horas de CPU):

1. **O PSO não acompanha a deriva térmica.** Depois de convergir ele fica parado no g_{best}. Quando o módulo esquenta de 43 para 53 °C depois da nuvem das 10:30, o V_{mp} cai, mas o PSO só reinicia se a potência mudar mais de 10 %: fica 3 a 4 % abaixo do máximo.
2. **O InC "congela".** Só sai do lugar se $\Delta I > 0{,}2\,\% \, I_{sc}$ por amostra (∼18 mA); variações lentas de luz não chegam a isso.
3. **O PSO erra o pico global com sombra fraca e módulo quente.** Com [1 1 0,5]·656 W/m^2 a 44 °C, as partículas iniciais erram o pico global (11,6 V) e o enxame converge no local (18,9 V).

12.5 E8: o trabalho anterior contra o modelo completo

O mesmo dia, sem os eventos de sombra, com P&O nos dois casos e o mesmo PI. O modelo "anterior" usa 1 diodo, conversor ideal e células a 25 °C; o "completo", 2 diodos, perdas e térmico.

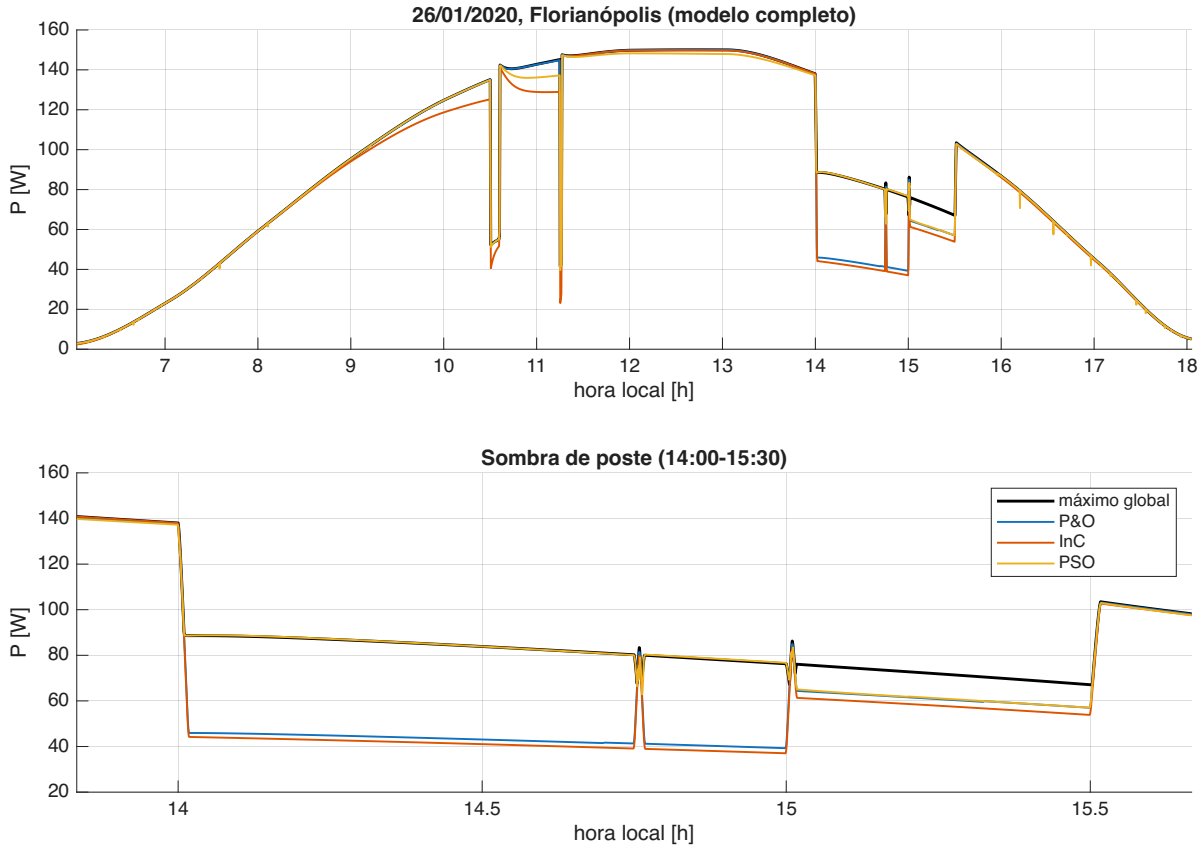


Figura 21: E7. Potência ao longo do dia (modelo completo). Preto: máximo global (oráculo); demais: potência extraída por P&O, InC e PSO. Embaixo, o detalhe da sombra de poste.

Resultado

Modelo	E_{pv} [Wh]	E_{bat} [Wh]	$T_{máx}$ [°C]
Anterior (1 diodo, ideal, 25 °C)	1238,4	1238,4	25,0
Completo (2 diodos, perdas, térmico)	1137,2	1015,8	55,6

O anterior **superestima a energia na bateria em 21,9 %** (1238,4/1015,8 = 1,219). A diferença se decompõe de forma multiplicativa: as perdas do conversor respondem por 1137,2/1015,8 = 1,120 (+12 %); o resto, 1238,4/1137,2 = 1,089 (+9 %), reúne a temperatura e a troca de 2 para 1 diodo. Como só a temperatura já aumenta a energia disponível em 9,0 % no E7, a contribuição do modelo de 1 diodo é praticamente nula, coerente com o E1 (os dois modelos são ajustados aos mesmos pontos e diferem pouco perto de STC, onde está quase toda a energia de um dia de verão). **O erro não estava no número de diodos, mas no que ficou fora do modelo.**

Atenção: ressalva

O E8 é uma *reconstrução*: o modelo "anterior" usa o ajuste de 1 diodo bem posto deste trabalho, para isolar o efeito das simplificações. A extração original do trabalho anterior dava ∼121 W com o MPP perto de 15 V, contra 160 W em 19,6 V aqui. Esse erro de parâmetros se somaria às simplificações e não está nos 21,9 %.

13 Testes, simplificação do código e verificação

13.1 O harness de testes

`matlab/run_tests.m` chama `setup_path` (que põe as pastas do projeto no *path*) e roda `runtests` em `matlab/tests/`; falha se algum teste falhar. São 14 arquivos com `matlab.unittest` e **78 testes** (eram 81 antes da simplificação e 80 depois dela; os 2 testes do LM saíram junto com ele). Cada teste prova uma propriedade física ou um contrato, e o comentário diz de onde vem o valor esperado.

Código 34: `tests/test_conversor.m`, linhas 11 a 37: três testes do conversor: ganho ideal, balanço de energia e derivadas nulas no equilíbrio.

```
11 function test_ganho_ideal(tc)
12 % Sem perdas, o ganho do buck-boost e d/(1-d) = Vbat/vpv e a eficiencia e 1.
13 pc = conversor_ideal(tc.TestData.pc);
14 eq = equilibrio_conversor(18, 7.7, pc);
15 verifyEqual(tc, eq.d / (1 - eq.d), 12 / 18, 'RelTol', 1e-9);
16 verifyEqual(tc, eq.eta, 1, 'AbsTol', 1e-9);
17 end
18
19 function test_balanco_de_energia(tc)
20 % Potencia de entrada = potencia na bateria + perdas de conducao, e
21 % Pout = Pin - Pcond - Psw; eta fica entre 0,85 e 1.
22 pc = tc.TestData.pc;
23 v = 18;  i = 7.7;
24 eq = equilibrio_conversor(v, i, pc);
25 verifyEqual(tc, v * i, (1 - eq.d) * eq.iL * pc.Vbat + eq.Pcond, 'RelTol', 1e-9);
26 verifyEqual(tc, eq.Pout, v * i - eq.Pcond - eq.Psw, 'RelTol', 1e-9);
27 verifyLessThan(tc, eq.eta, 1);
28 verifyGreaterThan(tc, eq.eta, 0.85);
29 end
30
31 function test_rhs_nulo_no_equilibrio(tc)
32 % No equilibrio calculado, as derivadas do modelo medio sao nulas.
33 pc = tc.TestData.pc;
34 eq = equilibrio_conversor(18, 7.7, pc);
35 dx = conversor_medio_rhs(18, eq.iL, 7.7, eq.d, pc);
36 verifyEqual(tc, dx, [0; 0], 'AbsTol', 1e-6);
37 end
```

Lógica passo a passo

1. `test_ganho_ideal`: sem perdas o ganho do buck-boost é $d/(1-d) = V_{bat}/v_{\mathrm{pv}}$ e $\eta = 1$ (física pura).
2. `test_balanco_de_energia`: a potência de entrada $v_{\mathrm{pv}} i_{\mathrm{pv}}$ deve ser igual à potência na bateria mais as perdas de condução; $P_{out} = P_{in} - P_{cond} - P_{sw}$; e $0{,}85 < \eta < 1$.
3. `test_rhs_nulo_no_equilibrio`: no equilíbrio calculado por `equilibrio_conversor`, as derivadas de `conversor_medio_rhs` são nulas. Confere *dois* códigos diferentes entre si.

Arquivo	O que verifica
`test_kcl_modulo`	O resíduo do módulo e a inclinação dI/dV (derivação implícita)
`test_extracao`	A extração no KC200GT contra o resultado publicado de Villalva
`test_ztp160m`, `test_ztp160m_extracao`	Pontos de STC do ZTP-160M, convergência dos modelos 1d e 2d, γ_P
`test_sombreamento`	P_{mp} e V_{mp} de referência, número de picos, comportamento do bypass
`test_termico`	Calibração pelo NMOT, efeito do vento e da extração de eletricidade
`test_conversor`	Balanço de energia, derivadas nulas no equilíbrio, ponto inviável, dimensionamento
`test_chaveado_medio`	Médio contra chaveado: desvio $< 0{,}5\,\%$ em v, $< 1{,}5\,\%$ em i_L, ripple dentro de $15\,\%$
`test_linearizacao`	A planta linearizada contra `gvd_analitico`
`test_controle`	Margens, tempo de acomodação, $T_\mathrm{a} \geq 1{,}5\,t_s$, e a falha da regra $\omega_c/5$
`test_mppt`	Cada algoritmo em curvas conhecidas, incluindo o preso em pico local
`test_sim_dinamica`	Energia, oráculo e partida da simulação dinâmica
`test_quase_estatica`	Quase-estático contra dinâmico (D16), blocos do oráculo e totais
`test_cenarios`	EN 50530, conversão de hora do PVGIS e eventos de sombra

Uma ideia recorrente é a de **regressão por comparativo**: um modelo mais detalhado serve de referência para um mais simples. O médio é conferido contra o chaveado, o quase-estático contra o dinâmico, o jacobiano analítico contra diferenças finitas e a planta numérica contra a analítica.

13.2 A etapa de simplificação do código

Depois do trabalho concluído foi feita uma auditoria para deixar o código mais simples, com um critério rígido: **nenhum número do relatório pode mudar**. Como a licença do MATLAB estava vencendo, os experimentos não foram rodados de novo; só entraram mudanças de risco baixo, que apenas reorganizam o código. O processo foi: spec, plano, achados (etapa 1: modelo e experimentos; etapa 2: testes), decisão item a item, aplicação em lotes e revisão independente. Os documentos estão em `docs/superpowers/simplificacao/`.

Etapa 1, 9 achados de risco baixo aplicados:

1. `levenberg_marquardt` (removido depois, ver Capítulo 4) sem o argumento opcional `opts` e a subfunção `campo`, que ninguém usava.
2. `curva_pv` sem o campo de saída `c.Vj`, que ninguém lia.
3. `sim_medio_aberta` sem o argumento `max_passo` e sem a generalidade de G como função do tempo.
4. `sim_chaveado` sem a saída `r.q`.
5. `projeto_pi` sem `rel.tabela` e sem o `ts_nom` só usado para preenchê-la (economiza 400 chamadas de `resposta_degrau_mf`).
6. Remoção do `mkdir` defensivo em 9 scripts de experimento, pois as pastas são versionadas.
7. Criação de `oraculo_com_cache.m`, antes uma subfunção duplicada nos dois simuladores.
8. `lei_pi` chamada uma vez, e não duas, dentro do jacobiano da `sim_dinamica`.
9. Remoção de comentário sobre uma opção (`op.V0`) que não existia no quase-estático.

O achado 10 (unificar as seis resoluções de `curva_pv`) tem risco médio: mudaria P_{mp}, V_{mp} e V_{oc} por pouco e, em cascata, L, C, o PI e todos os CSVs. Não foi aplicado.

Etapa 2 (testes). Achados 1, 2 e 5 aplicados: um teste duplicado de `test_conversor` foi fundido no de balanço de energia, um campo não usado do `setupOnce` saiu, e o *setup* de um caso do ZTP-160M foi movido para o único teste que o usa. Os achados 3 e 4 foram recusados, para o teste de extração continuar autossuficiente. O teste `test_fechada_igual_fzero` foi renomeado para `test_d_fechado_satisfaz_equilibrio`, pois o nome citava um `fzero` que ele não usa.

O que ficou de fora de propósito. A auditoria também registrou o que *não* foi mexido e por quê. O *setup* repetido entre arquivos de teste (2 a 5 linhas) ficou, porque uma função compartilhada esconderia de onde vem cada parâmetro, o que vai contra o estilo exigido (simples, explícito, legível por um júnior).

13.3 Resultado da simplificação

Pasta	Linhas antes	Depois	Variação
numerico	56	47	−9
painel	242	240	−2
termico	35	35	0
conversor	125	117	−8
controle	224	216	−8
mppt	140	140	0
simulacao	324	315	−9
cenarios	91	91	0
experimentos	480	466	−14
tests	812	806	−6
Total	**2548**	**2492**	**−56**

São linhas de código (sem comentário e sem branco). Os comentários (1494 antes, 1486 depois) foram mantidos porque explicam a física. A conclusão da auditoria foi que o código já estava enxuto: as ∼4400 linhas totais se explicam porque 34 % são comentários, os testes somam 806 linhas, os experimentos 466, o escopo é amplo e só havia o MATLAB base, sem toolboxes.

13.4 Como se verificou que nada quebrou

- **Testes:** 81 de 81 antes; 81 de 81 depois da etapa 1; 80 de 80 depois da etapa 2; 78 de 78 depois de trocar o LM pelo `fsolve` (10/11/2026, após a simplificação).
- `checkcode` (o analisador do MATLAB) nos arquivos alterados: nenhum aviso, antes nem depois.
- **Revisão independente da etapa 1:** conformidade e qualidade aprovadas; as duas observações menores, só de comentário, foram corrigidas.
- **Prova de que os testes enxutos não ficaram mais fracos:** a mesma perturbação no modelo faz o teste antigo e o novo falharem. Por exemplo, apagar `- eq.Psw` em `equilibrio_conversor.m` dá 1 falha no antigo e 1 no novo; trocar `r.E = E` por `1.02 * E` em `sim_quase_estatica.m` dá 4 falhas no antigo e 4 no novo.
- **Referências:** nenhuma referência órfã em `matlab/`, no relatório ou no README.

Atenção: ponto fraco a declarar

Os experimentos **não foram rodados de novo** depois da simplificação. A garantia de que os números não mudam vem de as mudanças serem só reorganização (o mesmo código, mudado de lugar ou sem as linhas não usadas). E6, E7 e E8 não foram reexecutados. O `git log` mostra a branch `simplificacao` já mesclada (commit `3f4f1ed`).

14 Conclusões, limitações, como rodar e glossário

14.1 Resposta à pergunta central

Em resumo

Sob sombreamento parcial estático, o P&O pode perder de 21 a 58 % da potência disponível (eficiências de 79 % a 42 %), porque para no pico local; o PSO recupera praticamente tudo ($\geq$ 99,99 %).
Num dia real de verão em Florianópolis, com uma hora de sombra de poste e duas nuvens, considerando o aquecimento do módulo (até 55,6 °C) e as perdas do conversor, o P&O deixa de captar 45,7 Wh (4,3 % da energia disponível) e o PSO, 13,8 Wh (1,3 %). Na bateria, o PSO entrega 3,1 % a mais que o P&O e 5,0 % a mais que o InC.
A busca global tem um custo: sem sombra, no perfil EN 50530, o PSO perde 1,3 ponto de eficiência para o P&O, porque reinicia a cada rampa. O PSO compensa quando há sombras que deslocam o pico global, e não compensa quando o céu só varia de forma uniforme.

Sobre a modelagem. Os resultados que mais pesam na energia não vêm dos algoritmos: ignorar o aquecimento superestima a energia em cerca de 9 % e ignorar as perdas do conversor, em cerca de 11 %; juntas, as simplificações do trabalho anterior dão +21,9 %. Do lado dos métodos, o trabalho mostrou que (i) o datasheet não identifica o segundo diodo e o gap precisa ser efetivo; (ii) $R_p(G)$ corrige uma perda de 26 % em baixa irradiância sem significado físico; (iii) a DAE com matriz de massa falha em 9 de 36 casos e a EDO reduzida resolve; e (iv) a hierarquia chaveado $\rightarrow$ médio $\rightarrow$ quase-estático, validada em cada degrau, permite simular as 13 h de sol em 9 a 23 min de CPU.

14.2 Limitações

- **Sem curvas I-V medidas.** O datasheet não as traz; a validação do painel é indireta (STC, γ_P, NMOT, KC200GT), e o γ_P do modelo (−0,363 %/K) fica abaixo do do datasheet (−0,4 %/K).
- **Diodos de bypass: hipótese.** O datasheet informa a associação "3×11" das 33 células, mas não quantos diodos há; adotaram-se 3 submódulos de 11 células, a configuração usual.
- **Ruptura reversa não modelada.** O bypass conduz muito antes dela, então o efeito na curva é pequeno, mas o aquecimento local (*hot spot*) fica fora.
- **Temperatura uniforme.** Na realidade, um submódulo sombreado fica mais frio que os outros.
- **Bateria ideal.** A tensão fixa em 12 V não varia com o estado de carga.
- **Sombras e nuvens sintéticas.** Os dados do PVGIS são horários; os resultados de energia valem para esse cenário, não para um dia típico.
- **Quase-estático conservador contra o PSO**, cerca de 1 % enquanto ele busca; no perfil EN 50530 a eficiência do PSO pode estar subestimada nessa ordem.
- **Margem de ganho apertada.** MG de 6,2 dB no pior caso, a 0,2 dB do requisito.
- **DCM não modelado.** O modelo médio de CCM não descreve os 13 a 32 % do tempo em DCM, onde as perdas de condução são subestimadas.
- **Perdas de catálogo**, não de um protótipo: a ablação mostra o tamanho do efeito, não o valor exato.
- **A EN 50530 foi transcrita de um relatório de ensaio**, não da norma; a data do relatório TÜV ainda aparece como "s.d." na bibliografia (pendência opcional).

Trabalho futuro: reverificação periódica e posições iniciais corrigidas pela temperatura no PSO; limiar de corrente proporcional à irradiância no InC; passo do InC além de 2 % de V_{oc}; ganho escalonado no PI; modelo de DCM; temperatura por submódulo; e validação experimental com curvas I-V medidas do ZTP-160M.

14.3 Como rodar

Requer o **MATLAB** com a **Optimization Toolbox** (só para o `fsolve` da extração de parâmetros). Testado no R2026b.

```
cd matlab
run_tests                % todos os testes
exp01_validacao_painel   % cada experimento grava em resultados/ e relatorio/figs/
```

Experimento	Tempo aproximado de CPU
`exp01` a `exp05`, `exp06b`	Minutos
`exp06_en50530`	∼1 h
`exp07_dia`	∼2 a 3 h (se `resultados/e7_series.mat` existir, só redesenha a figura)
`exp08_baseline`	∼30 min

Rode um experimento longo de cada vez: processos MATLAB em paralelo esgotam a memória. Os dados do PVGIS já estão versionados; para baixá-los de novo, `baixar_pvgis(2020)`.

Para compilar os documentos:

```
cd relatorio && tectonic -X compile relatorio.tex
cd guia      && tectonic -X compile guia_implementacao.tex
cd proposta  && python3 gerar_figuras.py && tectonic -X compile proposta.tex
```

14.4 Glossário

Termo	Significado
STC	Condições padrão de teste: 1000 W/m^2 e 25 °C de célula
NMOT	Temperatura nominal de operação do módulo: 800 W/m^2, ar a 20 °C, vento de 1 m/s
MPP / MPPT	Ponto de máxima potência / seu rastreamento
P&O, InC, PSO	Perturba e Observa; Condutância Incremental; Otimização por Enxame de Partículas
$V_{oc}, I_{sc}, V_{mp}, I_{mp}$	Tensão de circuito aberto, corrente de curto, tensão e corrente no MPP
Submódulo	Grupo de células em série protegido por um diodo de bypass
Diodo de bypass	Diodo em antiparalelo que desvia a corrente em torno de um submódulo sombreado
Pico local e global	Máximos da curva P-V com sombra; o global é o maior
Oráculo	Função que calcula o máximo global verdadeiro, para medir η
η MPPT	Energia rastreada dividida pela energia do máximo global
CCM e DCM	Condução contínua e descontínua (i_L zera a cada período)
Razão cíclica d	Fração do período em que a chave está ligada
Ripple	Oscilação pico a pico de uma grandeza por causa do chaveamento
Margem de fase e de ganho	Folgas de estabilidade da malha; usual: MF $\geq 45°$ e MG ≥ 6 dB
Anti-windup	Congelar o integrador do PI quando o atuador satura
Quase-estático	Tratar conversor e PI como em regime a cada período do MPPT
EDO e DAE	Equações diferenciais ordinárias; diferenciais e algébricas
`fsolve`	Resolve sistemas não lineares $r(y) = 0$ (Optimization Toolbox); aqui, por região de confiança
PVGIS	Base de dados de irradiância e clima da Comissão Europeia